

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_b7f012f8-5c3\agent-a7b7b6ae5c3e3f027.jsonl`  
- SHA-256: `e44e942d23bfdab2982274541664f65a8065fb37dee1cb2b50bf1c263e3a`  
- Source modified: `2026-06-18T01:31:54+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_b7f012f8-5c3`  
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T01:27:48.616Z)

You are an adversarial bug-hunter in a mature Python repo (badminton-highlight-indexer: FastAPI + SQLite + ffmpeg + GPU CV/AI rally detector). Read code READ-ONLY in this worktree (pinned to origin/master ? DO NOT modify anything):

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base

Hunt ONLY for REAL, REACHABLE defects a maintainer would agree is a bug and want fixed:

- crash on a reachable input (unguarded dict/index/attr access, None deref, KeyError/IndexError),
- wrong logic / off-by-one / inverted condition / wrong operator,
- incorrect error handling (catching too much/little, swallowing, wrong exception type),
- resource leak (unclosed file/db/subprocess), TOCTOU, mutable-default-arg,
- contract mismatch between a function and its callers/tests,
- wrong default / unit / sign that produces incorrect output.

HARD RULES:

- PRECISION over recall. Report a finding ONLY if you traced the actual code and a concrete trigger reaches it. No style, no "could be cleaner", no speculative "might", no perf-nits, no missing-test observations, no naming. If you are not sure it is a real bug, DROP it.
- For each finding give an EXACT file path + line, the bug_class, a CONCRETE trigger (the input or call path that hits it), why it is wrong, severity, and a minimal proposed fix.
- This repo has 848 passing tests and several hardening sweeps ? easy bugs are already gone. Expect FEW or ZERO real bugs in your area. Returning an empty candidates list is a VALID, respected answer. Do NOT manufacture findings to look productive.
- EXCLUDE these files entirely (just fixed / fix already in flight): backend/eval/ablation.py, backend/eval/fusion_compare.py, backend/eval/fusion_audio.py.
- Verify your trigger by reading the surrounding code and any related test, not from memory.

YOUR AREA: backend/pipeline/ ?

segmenters/, detectors/, stitcher/, validators/, audio/, personalization/. Focus on the registry/ABC plumbing, ffmpeg/subprocess invocation + error handling, video/frame indexing, padding/watermark/stitch math, and any unguarded access on detector/segment outputs.

2. assistant (2026-06-18T01:27:51.233Z)

I'll hunt for real, reachable defects in `backend/pipeline/`. Let me start by exploring the structure.

3. user (2026-06-18T01:27:52.917Z)

```
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)  
backend/pipeline/audio/hit_detector.py  
backend/pipeline/audio/__init__.py  
backend/pipeline/detectors/audio_features.py  
backend/pipeline/detectors/base.py
```

```
backend/pipeline/detectors/fusion_features.py
backend/pipeline/detectors/person_detector.py
backend/pipeline/detectors/rally_gate.py
backend/pipeline/detectors/tracknet_runner.py
backend/pipeline/detectors/wasb_infer.py
backend/pipeline/detectors/wasb_runner.py
backend/pipeline/detectors/__init__.py
backend/pipeline/personalization/serving.py
backend/pipeline/personalization/__init__.py
backend/pipeline/segmenters/base.py
backend/pipeline/segmenters/fusion_hybrid.py
backend/pipeline/segmenters/gemini.py
backend/pipeline/segmenters/motion.py
backend/pipeline/segmenters/tracknet_hybrid.py
backend/pipeline/segmenters/trajectory_hybrid.py
backend/pipeline/segmenters/wasb_hybrid.py
backend/pipeline/segmenters/yolo_hybrid.py
backend/pipeline/segmenters/__init__.py
backend/pipeline/stitcher/compiler.py
backend/pipeline/validators/base.py
backend/pipeline/validators/blur.py
backend/pipeline/validators/format.py
backend/pipeline/validators/pts_continuity.py
backend/pipeline/validators/relevance.py
backend/pipeline/validators/resolution_fps.py
backend/pipeline/validators/scene_cut.py
backend/pipeline/validators/__init__.py
```

4. assistant (2026-06-18T01:27:56.458Z)

Let me read the core files. Starting with the stitcher (padding/watermark/stitch math), detectors, and registry plumbing.

5. user (2026-06-18T01:27:57.410Z)

```
1     import os
2     import subprocess
3     import tempfile
4     import logging
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     logger = logging.getLogger(__name__)
9
10    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
11    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
12    # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
13    # backend/pipeline/stitcher/, so parents[2] == backend/.
14    _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
Regular.ttf"
15
16    class VideoCompiler:
17        def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
18                    watermark: Optional[Any] = None):
19            self.output_dir = output_dir
20            self.end_padding = end_padding
21            self.begin_padding = begin_padding
22            # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
23            # normalized to a dict (or None) so this module stays config-package-agnostic.
24            self.watermark = self._normalize_watermark(watermark)
25            os.makedirs(output_dir, exist_ok=True)
```

```

26
27 @staticmethod
28 def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
29     if watermark is None:
30         return None
31     if hasattr(watermark, "model_dump"):
32         return watermark.model_dump()
33     if isinstance(watermark, dict):
34         return dict(watermark)
35     return None
36
37 @staticmethod
38 def _ff_quote(path: str) -> str:
39     """Make a filesystem path safe to embed inside single quotes in an ffmpeg
40     filtergraph operand (drawtext fontfile/textfile, movie= source): forward
41     slashes (Windows-friendly), escaped single quotes, AND escaped colons.
42
43     The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
44     option parser treats ':' as the option separator, so a Windows drive-letter
45     path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
46     the encode fails with "Invalid argument". This silently disabled the
47     on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
48     return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
49
50 @staticmethod
51 def _parse_time(val: Union[int, float, str]) -> float:
52     """Normalizes a timestamp value to float seconds.
53     Handles: float/int (pass-through), plain numeric strings ("12.4"),
54     and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
55     This mirrors the parse_time logic used in gemini.py to ensure
56     the stitcher can always consume whatever format the indexer produced."""
57     if isinstance(val, (int, float)):
58         return float(val)
59     if isinstance(val, str):
60         val = val.strip()
61         if ":" in val:
62             # Handle MM:SS, HH:MM:SS, etc.
63             parts = val.split(":")
64             parts.reverse()
65             total = 0.0
66             for i, p in enumerate(parts):
67                 try:
68                     total += float(p) * (60 ** i)
69                 except ValueError:
70                     pass
71             return total
72         try:
73             return float(val)
74         except ValueError:
75             logger.warning(f"Could not parse timestamp value '{val}' as a number.
76             Defaulting to 0.0.")
77             return 0.0
78             logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
79             '{val}'. Defaulting to 0.0.")
80             return 0.0
81
82 @staticmethod
83 def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float,
84 float]]:
85     """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
86
87     A highlight reel must never replay the same footage. The same rally can land in
88     the segment list more than once ? e.g. two detector runs accumulated for one
89     video (add_video is an UPSERT, so re-running without a prune leaves both sets),
90     or

```

```

87         chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
rally
88         twice. Merging any range that overlaps or abuts the previous one collapses true
89         duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
90         Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
91         """
92         parsed = []
93         for raw_s, raw_e in segments:
94             s = VideoCompiler._parse_time(raw_s)
95             e = VideoCompiler._parse_time(raw_e)
96             if e > s:
97                 parsed.append((s, e))
98         parsed.sort()
99         merged: List[Tuple[float, float]] = []
100         for s, e in parsed:
101             if merged and s <= merged[-1][1]: # overlaps/abuts previous -> extend it
102                 merged[-1] = (merged[-1][0], max(merged[-1][1], e))
103             else:
104                 merged.append((s, e))
105         return merged
106
107     def _get_video_duration(self, video_path: str) -> float:
108         """Probes the video file to get its total duration in seconds.
109         Returns 0.0 if it cannot be determined."""
110         try:
111             result = subprocess.run(
112                 [
113                     "ffprobe", "-v", "error",
114                     "-show_entries", "format=duration",
115                     "-of", "default=noprint_wrappers=1:nokey=1",
116                     video_path
117                 ],
118                 capture_output=True, text=True, check=True
119             )
120             return float(result.stdout.strip())
121         except Exception as e:
122             logger.warning(f"Could not determine video duration via ffprobe: {e}")
123             return 0.0
124
125     def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
126         """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
127         and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
128         watermarking is disabled or has nothing to draw. The concat stage stays -c copy
129         because the mark is already baked into every clip identically."""
130         wm = self.watermark
131         if not wm or not wm.get("enabled"):
132             return None
133
134         text = (wm.get("text") or "").strip()
135         logo_path = (wm.get("logo_path") or "").strip()
136         if logo_path and not os.path.exists(logo_path):
137             logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo
overlay.")
138             logo_path = ""
139         if not text and not logo_path:
140             return None
141
142         margin = int(wm.get("margin", 24))
143         opacity = float(wm.get("opacity", 0.85))
144         position = str(wm.get("position", "bottom-right"))
145         # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
146         dt_xy = {
147             "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
148             "bottom-left": f"x={margin}:y=h-th-{margin}",

```

```

149         "top-right": f"x=w-tw-{margin}:y={margin}",
150         "top-left": f"x={margin}:y={margin}",
151     }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
152     ov_xy = {
153         "bottom-right": f"W-w-{margin}:H-h-{margin}",
154         "bottom-left": f"{margin}:H-h-{margin}",
155         "top-right": f"W-w-{margin}:{margin}",
156         "top-left": f"{margin}:{margin}",
157     }.get(position, f"W-w-{margin}:H-h-{margin}")
158
159     drawtext = ""
160     if text:
161         ratio = float(wm.get("font_size_ratio") or 0.035)
162         divisor = max(1, round(1.0 / ratio))
163         # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
164         strings.
165         tf_path = os.path.join(tmp_dir, "watermark.txt")
166         with open(tf_path, "w", encoding="utf-8") as fh:
167             fh.write(text)
168         # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
169         # without fontconfig) ? fontconfig family (last-resort, environment-
170         dependent).
171         font_path = (wm.get("font_path") or "").strip()
172         if not font_path and _BUNDLED_FONT.exists():
173             font_path = str(_BUNDLED_FONT)
174         if font_path:
175             font_opt = f"fontfile='{self._ff_quote(font_path)}'"
176         else:
177             font_opt = f"font='{wm.get('font', 'Sans')}'"
178         box = ""
179         if wm.get("box", True):
180             box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
181         drawtext = (
182             f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
183             f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
184         )
185
186     if logo_path and drawtext:
187         return
188     f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
189     if logo_path:
190         return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
191     return drawtext
192
193     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
194     float]], output_filename: str) -> str:
195         """
196         Extracts selected segments from a raw video and stitches them together
197         into a seamless highlights file, applying end_padding to prevent abrupt endings.
198         """
199         if not segments:
200             raise ValueError("No highlight segments provided to compile.")
201
202         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
203         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
204         original_count = len(segments)
205         segments = self._merge_overlapping(segments)
206         if len(segments) < original_count:
207             logger.info("Merged %d overlapping/duplicate segment(s) before stitching: %d
208             -> %d.",
209                 original_count - len(segments), original_count, len(segments))
210         if not segments:
211             raise ValueError("No valid (start < end) highlight segments after merge.")
212
213         if not os.path.exists(raw_video_path):

```

```

209         raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
210
211         # Defense-in-depth: never let the output name escape output_dir (the API also
212         # validates this at the request boundary).
213         output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
214
215         # Probe video duration so we can detect out-of-range segments
216         video_duration = self._get_video_duration(raw_video_path)
217         if video_duration > 0:
218             logger.info(f"Source video duration: {video_duration:.2f}s")
219         else:
220             logger.warning("Video duration unknown. Cannot validate segment boundaries
against video length.")
221
222         # We will create temporary clips and stitch them using FFmpeg concat demuxer
223         temp_clips = []
224         skipped_segments = []
225
226         # Create a temp directory within output_dir
227         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
228             logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
229             # Build the (optional) watermark filtergraph once; it is applied during
every
230             # per-clip re-encode so the mark is baked into each clip identically.
231             watermark_filter = self._watermark_filter(tmp_dir)
232             if watermark_filter:
233                 logger.info("[watermark] applying branding overlay to each clip.")
234             for idx, (raw_start, raw_end) in enumerate(segments):
235                 # Normalize timestamps to float seconds ? handles float, int,
236                 # plain numeric strings, and colon-separated formats like "MM:SS"
237                 start = self._parse_time(raw_start)
238                 end = self._parse_time(raw_end)
239                 segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
{end:.2f}s]"
240
241                 # Validate the segment times
242                 if start < 0:
243                     logger.warning(f"{segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.")
244                     start = 0.0
245
246                 if start >= end:
247                     logger.error(f"{segment_label}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s). Skipping invalid segment.")
248                     skipped_segments.append((idx, start, end, "start >= end"))
249                     continue
250
251                 # Check if segment extends beyond video duration
252                 if video_duration > 0:
253                     if start >= video_duration:
254                         logger.error(f"{segment_label}: start_time ({start:.2f}s) is
beyond the video duration ({video_duration:.2f}s). "
255                                     f"This segment cannot be extracted ? the video ran
out. Skipping.")
256                         skipped_segments.append((idx, start, end, "start beyond video
end"))
257                         continue
258
259                 if end > video_duration:
260                     original_end = end
261                     end = video_duration
262                     logger.warning(f"{segment_label}: end_time ({original_end:.2f}s)
exceeds video duration ({video_duration:.2f}s). "

```

```

263                                     f"Clamping end_time to {video_duration:.2f}s. The
segment may be truncated.")
264
265         clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
266
267         # Precise sub-second trim using fast re-encoding to preserve stream
headers
268         padded_start = max(0.0, start - self.begin_padding)
269         padded_end = end + self.end_padding
270
271         # Clamp padded_end to video duration if known
272         if video_duration > 0 and padded_end > video_duration:
273             padded_end = video_duration
274             logger.info(f"{segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
275                       f"Clamping to {video_duration:.2f}s (end padding
partially applied).")
276
277         trim_duration = padded_end - padded_start
278         logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")
279
280         base_cmd = [
281             "ffmpeg", "-y",
282             "-ss", str(round(padded_start, 3)),
283             "-to", str(round(padded_end, 3)),
284             "-i", raw_video_path,
285             "-c:v", "libx264",
286             "-preset", "superfast",
287             "-crf", "22",
288             "-c:a", "aac",
289             "-b:a", "128k",
290         ]
291         vf_args = ["-vf", watermark_filter] if watermark_filter else []
292
293         # Try with the watermark first; if that encode fails (e.g. a bad
font/logo
294         # path), retry once WITHOUT it so a branding misconfig can never nuke
the
295         # whole reel. Clips stay codec-identical either way, so concat -c copy
holds.
296         attempts = [vf_args, []] if vf_args else [[]]
297         produced = False
298         last_exc: Optional[subprocess.CallledProcessError] = None
299         cmd = base_cmd + [clip_path]
300         for attempt_vf in attempts:
301             cmd = base_cmd + attempt_vf + [clip_path]
302             try:
303                 subprocess.run(cmd, capture_output=True, check=True)
304             except subprocess.CallledProcessError as e:
305                 last_exc = e
306                 if attempt_vf and vf_args:
307                     _err = (e.stderr.decode(errors="replace").strip()[-300:]
308                             if getattr(e, "stderr", None) else "")
309                     logger.warning(
310                         f"{segment_label}: watermark encode failed; retrying
without it. "
311                         f"ffmpeg: {_err or '(no stderr captured)'}"
312                     )
313                     continue
314                 break
315         if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
316             produced = True
317             if not attempt_vf and vf_args:
318                 logger.warning(f"{segment_label}: encoded WITHOUT the

```

```

watermark (the watermark filter failed).")
319             break
320             last_exc = None # ffmpeg succeeded but produced nothing
321             break
322
323         if produced:
324             temp_clips.append(clip_path)
325         elif last_exc is not None:
326             stderr_msg = last_exc.stderr.decode(errors='replace').strip()
327             logger.error(f"{segment_label}: FFmpeg trim failed.")
328             print(f"  Command: {' '.join(cmd)}")
329             print(f"  FFmpeg stderr: {stderr_msg[-500:]}") # Last 500 chars to
avoid flooding
330             skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
331             continue
332         else:
333             logger.error(f"{segment_label}: FFmpeg exited successfully but
output clip is missing or empty. Skipping.")
334             skipped_segments.append((idx, start, end, "empty output clip"))
335             continue
336
337         # Report summary of skipped segments
338         if skipped_segments:
339             logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
340             for seg_idx, seg_start, seg_end, reason in skipped_segments:
341                 logger.info(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
342                 logger.info("")
343
344         if not temp_clips:
345             raise RuntimeError(
346                 f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
347                 f"All {len(segments)} segments were skipped. "
348                 f"Check the errors above for details. "
349                 f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
350             )
351
352         logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")
353
354         # Create a list file for FFmpeg concat demuxer
355         list_file_path = os.path.join(tmp_dir, "clips.txt")
356         with open(list_file_path, "w") as f:
357             for clip in temp_clips:
358                 # Escape single quotes in path
359                 safe_path = clip.replace("'", "'\\'")
360                 f.write(f"file '{safe_path}'\n")
361
362         logger.info("Stitching clips together...")
363         # Stitch them together without re-encoding (since they were encoded with
identical settings)
364         concat_cmd = [
365             "ffmpeg", "-y",
366             "-f", "concat",
367             "-safe", "0",
368             "-i", list_file_path,
369             "-c", "copy",
370             output_path
371         ]
372
373         try:
374             subprocess.run(concat_cmd, capture_output=True, check=True)

```



```

375
376         # Final verification
377         if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
378             raise RuntimeError(
379                 f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
380                 f"but output file is missing or empty: {output_path}"
381             )
382
383         output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
384         logger.info(f"Dynamic highlight reel saved: {output_path}
385         ({output_size_mb:.1f} MB)")
386         return output_path
387     except subprocess.CalledProcessError as e:
388         stderr_msg = e.stderr.decode(errors='replace').strip()
389         raise RuntimeError(
390             f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
391             f"Command: {' '.join(concat_cmd)}\n"
392             f"FFmpeg stderr: {stderr_msg[-500:]}"
393         ) from e

```

6. user (2026-06-18T01:27:58.004Z)

```

1     from abc import ABC, abstractmethod
2     from typing import List, Dict, Any, Type, Optional
3     from backend.infrastructure.database import Database
4     from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6     class BasePlaySegmenter(ABC):
7         def __init__(self, db: Database, config: Dict[str, Any]):
8             self.db = db
9             self.config = config
10
11         @property
12         @abstractmethod
13         def name(self) -> str:
14             """Unique identifier for this segmenter."""
15             pass
16
17         @property
18         @abstractmethod
19         def description(self) -> str:
20             """Human-readable explanation of this segmenter."""
21             pass
22
23         @abstractmethod
24         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25             """
26             Processes a video to detect play segments, populates SQLite database,
27             and returns a Result.
28
29             Use isinstance(result, Failure) (or result.is_success()) to distinguish
30             "no segments found" from "something went wrong" (missing key, healthcheck
31             failure, etc.). This is the start of the standardized error/result contract.
32             """
33             pass
34
35     class SegmenterRegistry:
36         def __init__(self):
37             self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39         def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40             """Registers a new segmenter class under its name property."""

```

```

41         try:
42             temp_inst = segmenter_cls(None, {})
43             name = temp_inst.name
44         except Exception as e:
45             raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46         self._segmenters[name] = segmenter_cls
47
48     def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49         return self._segmenters.get(name)
50
51     def list_available(self) -> Dict[str, str]:
52         """Returns a dict of registered segmenter names and their descriptions."""
53         res = {}
54         for name, cls in self._segmenters.items():
55             try:
56                 temp_inst = cls(None, {})
57                 desc = temp_inst.description
58             except Exception:
59                 desc = "No description available."
60             res[name] = desc
61         return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

7. assistant (2026-06-18T01:28:05.515Z)

Let me look at the detectors and other segmenters which often have more complex indexing math.

8. user (2026-06-18T01:28:06.752Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4     1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5     2. copies wasb_infer.py into the WASB repo's src/ dir,
6     3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7     4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess
16    from dataclasses import dataclass
17    from typing import Dict, Any, Optional, Tuple
18
19    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21    from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24    logger = logging.getLogger(__name__)
25
26    # Windows path to the inference wrapper we stage into WSL.
27    _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30    @dataclass
31    class WasbConfig:

```

```

32     """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33     distro: str = "Ubuntu"
34     repo_dir: str = "~/models/WASB-SBDT"
35     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36     conda_env: str = "wasb"
37     weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38     sport: str = "badminton"
39     wsl_stage_dir: str = "~/clips"
40     timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41     keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42     min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43     # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44     # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45     # extraction that dominates a long clip. Output is bit-identical to the PNG path
46     # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47     stream_video: bool = False
48
49     @classmethod
50     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51         w = (idx_cfg or {}).get("wasb", {}) or {}
52         return cls(
53             distro=w.get("wsl_distro", "Ubuntu"),
54             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56             conda_env=w.get("conda_env", "wasb"),
57             weights_path=w.get("weights_path",
58                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59             sport=w.get("sport", "badminton"),
60             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61             timeout_sec=int(w.get("timeout_sec", 1800)),
62             keep_frames=bool(w.get("keep_frames", False)),
63             min_free_gb=float(w.get("min_free_gb", 0.0)),
64             stream_video=bool(w.get("stream_video", False)),
65         )
66
67
68     class WasbRunner(WslCommandMixin, DetectorRunner):
69         def __init__(self, cfg: WasbConfig):
70             self.cfg = cfg
71
72         def healthcheck(self) -> Tuple[bool, str]:
73             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74             cmd = (f"conda activate {self.cfg.conda_env} && "
75                   f"python -c \"import torch; print('torch', torch.__version__, "
76                   f"'cuda', torch.cuda.is_available())\"")
77             try:
78                 res = self._wsl_bash(cmd)
79             except FileNotFoundError:
80                 return False, "`wsl` not found ? Windows + WSL2 required."
81             except subprocess.TimeoutExpired:
82                 return False, "WSL healthcheck timed out."
83             out = (res.stdout or "").strip()
84             if res.returncode != 0:
85                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86             return True, out
87
88         def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89             src_mnt = to_wsl_mnt_path(video_win_path)
90             reason = self.wsl_source_unreadable_reason(src_mnt)

```

```

91         if reason:
92             logger.error("Cannot stage video into WSL: %s", reason)
93             return None
94         dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
96         {wsl_tilde_quote(dest)} && echo OK"
97         try:
98             res = self._wsl_bash(cmd)
99         except subprocess.TimeoutExpired:
100             logger.error("Staging video into WSL timed out.")
101             return None
102         if res.returncode != 0 or "OK" not in (res.stdout or ""):
103             logger.error("Failed to stage video into WSL: %s", (res.stderr or
104 res.stdout).strip())
105             return None
106         return dest
107
108 def _stage_infer_script(self) -> bool:
109     """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
110     src_mnt = to_wsl_mnt_path(_INFER_WIN)
111     cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
112 OK"
113     res = self._wsl_bash(cmd)
114     return res.returncode == 0 and "OK" in (res.stdout or "")
115
116 def run_predict(self, video_win_path: str, output_win_dir: str,
117                 video_id: Optional[str] = None) -> Optional[str]:
118     """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
119
120     All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
121     two videos that share a filename (e.g. two ``match.mp4`` from different folders)
122     can never reuse each other's staged frames, resumable cache, or CSV.
123
124     """
125     # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
126     # translation: to_wsl_mnt_path passes relative paths through unchanged, so
127     # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
128     # the CSV landed inside WSL while Windows polled the repo-relative path.
129     output_win_dir = os.path.abspath(output_win_dir)
130     os.makedirs(output_win_dir, exist_ok=True)
131     # Normalize for cross-platform basename (tests use Windows paths on Linux).
132     video_win_path = str(video_win_path).replace("\\", "/")
133     base = os.path.basename(video_win_path)
134     stem, ext = os.path.splitext(base)
135     # Unique, content-derived key: same filename + different bytes -> different key.
136     key = f"{stem}__{video_id[:12]}" if video_id else stem
137     wsl_out_dir = to_wsl_mnt_path(output_win_dir)
138     expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
139
140     if not self._stage_infer_script():
141         logger.error("Could not stage wasb_infer.py into WSL.")
142         return None
143     staged_video = self._stage_video(video_win_path, f"{key}{ext}")
144     if staged_video is None:
145         return None
146     frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
147
148     # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
149     # WSL stage filesystem is already low so the user can clean up before it fills.
150     # (Streaming never materializes the PNGs, so the guard only matters off the fast
151     path.)
152
153     if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
154         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
155         if free is not None and free < self.cfg.min_free_gb:
156             logger.warning(
157                 "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame

```

```

extraction "
152             "may fill the disk ? consider running tools/cleanup_caches.py.",
153             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
154
155     # Fast path: stream-decode the video in-process (no PNG extraction). Output is
156     # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
157     if self.cfg.stream_video:
158         frame_args = "--stream-video "
159     else:
160         frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
161     cmd = (
162         f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
163         f"python wasb_infer.py "
164         f"--video {wsl_tilde_quote(staged_video)} "
165         f"{frame_args}"
166         f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
167         f"--sport {shlex.quote(self.cfg.sport)} "
168         f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv'}}"
169     )
170     logger.info("Running WASB inference on %s ...", base)
171     try:
172         res = self._wsl_bash(cmd)
173     except subprocess.TimeoutExpired:
174         logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
175         return None
176     if res.returncode != 0:
177         logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
178         return None
179     if not os.path.exists(expected_csv_win):
180         logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
181         return None
182     logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184     # Success ? the trajectory CSV is the durable output; the staged video copy and
185     # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186     # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187     # so the frame cache + detector cache remain for a resumable re-run.
188     # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189     if not self.cfg.keep_frames:
190         stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191         logger.info("Cache hygiene: removing staged video%s for %s "
192                     "(set indexing.wasb.keep_frames=true to retain).",
193                     "" if self.cfg.stream_video else " + frame cache", key)
194         self._wsl_rm_rf(stale)
195         return expected_csv_win
196
197     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199     trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200

```

9. user (2026-06-18T01:28:06.979Z)

```

1     """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3     WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4     the opponent for service after losing a point, that single trajectory looks like a
5     rally. The discriminator is structural, not duration-based: a real rally has the

```

```

6 shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7 single service feed crosses -once.
8
9 This module is camera-agnostic and needs NO new model ? it works on the trajectory
10 we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11 axis with the larger shuttle spread) and the net position (median shuttle coord on
12 that axis ? the shuttle clusters at the two ends and crosses the net region often,
13 so the median sits near the net), then counts sign changes of (coord - net) across
14 the visible points inside a candidate window, with a deadband to ignore jitter.
15
16 Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17 Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18 windows from trajectory_to_action_windows, or to fold into it later.
19 """
20 from __future__ import annotations
21
22 from dataclasses import dataclass
23 from typing import List, Optional, Sequence, Tuple
24
25 Interval = Tuple[float, float]
26
27
28 @dataclass
29 class GatedWindow:
30     start: float
31     end: float
32     crossings: int
33     visible_points: int
34     kept: bool
35
36
37 def _spread(vals: Sequence[float]) -> float:
38     """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39     if len(vals) < 2:
40         return 0.0
41     s = sorted(vals)
42     lo = s[max(0, int(0.05 * (len(s) - 1)))]
43     hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44     return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48     if not vals:
49         return 0.0
50     s = sorted(vals)
51     n = len(s)
52     return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56     """Return (axis 'x'/'y', net_value, span) from visible points.
57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the

```

```

71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87 def _points_in(points, fps: float, start: float, end: float):
88     return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91 def gate_windows(points, windows: List[Interval], fps: float,
92                 min_crossings: int = 2, deadband_frac: float = 0.06,
93                 estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94     """Annotate each (start,end) window with its shuttle net-crossing count and a
95     keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96     axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98     `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99     `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
100     per window ? same result, avoids the per-window re-estimation."""
101     axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102     deadband = deadband_frac * span
103     out: List[GatedWindow] = []
104     for (s, e) in windows:
105         wp = _points_in(points, fps, s, e)
106         c = count_net_crossings(wp, axis, net, deadband)
107         vis = sum(1 for p in wp if p.visible)
108         out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                               kept=c >= min_crossings))
110     return out
111
112
113 def apply_gate(points, windows: List[Interval], fps: float,
114               min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115     """Convenience: return only the windows that pass the crossing gate."""
116     return [(g.start, g.end) for g in
117             gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

10. assistant (2026-06-18T01:28:09.483Z)

Let me read the tracknet_runner (shared windowing logic), and the hybrid segmenters where indexing math lives.

11. user (2026-06-18T01:28:10.825Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6     1. Translate the Windows video path to its WSL-visible form (and optionally
7     stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).

```

```

8         2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9         3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10        4. Convert that trajectory into high-motion "action windows" ? the same
11            candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12            pipeline (AI handoff, DB population) is unchanged.
13
14    Nothing here imports TensorFlow; all model code stays in WSL.
15    """
16
17    import os
18    import csv
19    import shlex
20    import logging
21    import subprocess
22    from dataclasses import dataclass
23    from typing import List, Dict, Any, Optional, Tuple
24
25    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27    logger = logging.getLogger(__name__)
28
29
30    @dataclass
31    class TrackNetConfig:
32        """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33        distro: str = "Ubuntu"
34        repo_dir: str = "~/models/TrackNetV4" # WSL path to the cloned repo
35        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36        conda_env: str = "TrackNetV4"
37        weights_path: str = "" # WSL path to the .h5/.keras weights
(REQUIRED)
38        queue_length: int = 5
39        stage_in_wsl: bool = True # copy video into WSL fs first
(perf)
40        wsl_stage_dir: str = "~/clips" # where staged videos/outputs live
in WSL
41        timeout_sec: int = 1800 # 30 min; kills a hung call (0 = no
timeout)
42        keep_frames: bool = False # retain staged video after success
(debug only)
43        min_free_gb: float = 0.0 # warn if WSL free space below this
(0 = off)
44
45    @classmethod
46    def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47        tn = (idx_cfg or {}).get("tracknet", {}) or {}
48        return cls(
49            distro=tn.get("wsl_distro", "Ubuntu"),
50            repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51            conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52            conda_env=tn.get("conda_env", "TrackNetV4"),
53            weights_path=tn.get("weights_path", ""),
54            queue_length=int(tn.get("queue_length", 5)),
55            stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56            wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57            timeout_sec=int(tn.get("timeout_sec", 1800)),
58            keep_frames=bool(tn.get("keep_frames", False)),
59            min_free_gb=float(tn.get("min_free_gb", 0.0)),
60        )
61
62
63    def to_wsl_mnt_path(win_path: str) -> str:
64        """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).

```



```

65
66     Already-POSIX paths (starting with / or ~) are returned unchanged so the
67     same helper is safe to call on either kind of path.
68     """
69     p = str(win_path)
70     if p.startswith("/") or p.startswith("~"):
71         return p
72     p = p.replace("\\", "/")
73     if len(p) >= 2 and p[1] == ":":
74         drive = p[0].lower()
75         return f"/mnt/{drive}{p[2:]}"
76     return p
77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU')))\n"
107         )
108         try:
109             res = self._wsl_bash(cmd)
110         except FileNotFoundError:
111             return False, "`wsl` not found ? is this running on Windows with WSL2
112             installed?"
113         except subprocess.TimeoutExpired:
114             return False, "WSL healthcheck timed out."
115         out = (res.stdout or "").strip()
116         if res.returncode != 0:
117             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
118         return True, out
119
120 # ----- inference
121
122 def run_predict(self, video_win_path: str, output_win_dir: str,
123                 video_id: Optional[str] = None) -> Optional[str]:
124     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
125     None.
126     ``output_win_dir`` is a Windows directory (under the repo's output/) that
127     WSL writes into via its /mnt view, so the windows process can read the CSV
128     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and

```

```

127     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
128     a filename cannot collide on the output CSV.
129     """
130     if not self.cfg.weights_path:
131         logger.error(
132             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
133             "in config.json to the WSL path of your trained TrackNetV4 weights."
134         )
135     return None
136
137     # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
138     # relative paths through unchanged, so WSL would resolve them against the
139     # predict.py cwd instead of the repo (same bug class as wasb_runner).
140     output_win_dir = os.path.abspath(output_win_dir)
141     os.makedirs(output_win_dir, exist_ok=True)
142     # Normalize separators so basename/splitext work when tests (or collector)
143     # pass Windows-style paths on a Linux host.
144     video_win_path = str(video_win_path).replace("\\", "/")
145     base = os.path.basename(video_win_path)
146     stem, ext = os.path.splitext(base)
147
148     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
149     if ext.lower() not in (".mp4", ".avi"):
150         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
151     return None
152
153     wsl_out = to_wsl_mnt_path(output_win_dir)
154     # Namespace by video_id so two same-filename videos don't collide on the CSV.
155     # predict.py derives its output name from the (staged) video stem, so staging
156     # under the namespaced name propagates into "<key>_predict.csv".
157     key = f"{stem}_{video_id[:12]}" if video_id else stem
158     expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160     # Resolve the video path WSL will read.
161     if self.cfg.stage_in_wsl:
162         staged = self._stage_video(video_win_path, f"{key}{ext}")
163         if staged is None:
164             return None
165         wsl_video = staged
166     else:
167         # No staging: predict.py reads /mnt directly and writes
168         "<stem>_predict.csv".
169         # We cannot rename its output, so fall back to the un-namespaced name.
170         wsl_video = to_wsl_mnt_path(video_win_path)
171         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
172
173     if self.cfg.min_free_gb > 0:
174         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
175         if free is not None and free < self.cfg.min_free_gb:
176             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
177                             "consider running tools/cleanup_caches.py.",
178                             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
179
180     cmd = (
181         f"conda activate {self.cfg.conda_env} && "
182         f"cd {self.cfg.repo_dir}/src && "
183         f"python predict.py "
184         f"--video_path {wsl_tilde_quote(wsl_video)} "
185         f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
186         f"--output_dir {wsl_tilde_quote(wsl_out)} "
187         f"--queue_length {self.cfg.queue_length}"
188     )
189     logger.info("Running TrackNet inference on %s ...", base)
190     try:
191         res = self._wsl_bash(cmd)

```

```

191         except subprocess.TimeoutExpired:
192             logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
193             return None
194
195         if res.returncode != 0:
196             logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
197             return None
198
199         if not os.path.exists(expected_csv_win):
200             logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
201             return None
202             logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
203
204         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
205         # regenerable intermediate). On earlier failure we return above without
cleaning.
206         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207             logger.info("Cache hygiene: removing staged video for %s "
208                         "(set indexing.tracknet.keep_frames=true to retain).", key)
209             self._wsl_rm_rf([wsl_video])
210             return expected_csv_win
211
212     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215         Returns the WSL path to the staged copy, or None on failure.
216         """
217         src_mnt = to_wsl_mnt_path(video_win_path)
218         reason = self.wsl_source_unreadable_reason(src_mnt)
219         if reason:
220             logger.error("Cannot stage video into WSL: %s", reason)
221             return None
222         dest = f"{self.cfg.wsl_stage_dir}/{base}"
223         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
224         try:
225             res = self._wsl_bash(cmd)
226         except subprocess.TimeoutExpired:
227             logger.error("Staging video into WSL timed out.")
228             return None
229         if res.returncode != 0 or "OK" not in (res.stdout or ""):
230             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
231             return None
232         return dest
233
234     # ----- CSV -> windows
235
236     @staticmethod
237     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239         points: List[TrajectoryPoint] = []
240         with open(csv_win_path, newline="") as f:
241             reader = csv.DictReader(f)
242             for row in reader:
243                 try:
244                     points.append(TrajectoryPoint(
245                         frame=int(row["Frame"]),
246                         visible=int(row["Visibility"]) == 1,
247                         x=float(row["X"]),
248                         y=float(row["Y"]),
249

```

```

250         except (KeyError, ValueError):
251             continue
252     points.sort(key=lambda p: p.frame)
253     return points
254
255     @staticmethod
256     def trajectory_to_action_windows(
257         points: List[TrajectoryPoint],
258         fps: float,
259         frame_width: float,
260         chunk_start: float = 0.0,
261         velocity_thresh: float = 0.02,
262         merge_gap: float = 2.0,
263         min_window_duration: float = 2.0,
264         chunk_index: Optional[int] = None,
265         max_window_duration: float = 0.0,
266         min_split_gap: float = 0.5,
267         window_hard_cap: bool = False,
268         window_inactivity_end: float = 0.0,
269         inactivity_min_density: float = 0.34,
270         inactivity_rally_thresh: float = 0.08,
271         window_blob_fallback: bool = False,
272         window_blob_factor: float = 4.0,
273     ) -> List[Dict[str, Any]]:
274         """Convert a shuttle trajectory into candidate rally windows.
275
276         A frame is "active" when the shuttle is visible AND its inter-frame
277         displacement (normalised by frame width, per second) exceeds
278         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
279         ``merge_gap`` seconds of each other are grouped into one window; short
280         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
281         shape feeding the AI-handoff phase is identical.
282
283         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
284         against the *over-merge* failure where "shuttle moving" is conflated with "rally in
285         progress" and a single window swallows several rallies + the dead time between them. When
286         set, any window longer than this is recursively SPLIT at its largest internal inactivity
287         gap (the longest stretch with no in-flight shuttle ? the most likely rally boundary),
288         provided that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
289         fix from ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
290         recall cannot drop, and it is config-gated default-OFF until measured on the golden set.
291
292         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
293         an over-long window has NO internal inactivity gap to split on (a continuously-
294         active trajectory ? the shuttle never stops moving, e.g. the real-footage
295         ``mahadevpura-1`` blob where the gap-splitter alone left a single 174 s window), it is recursively
296         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
297         cuts (never merges), so recall cannot drop; gated default-OFF until measured.
298
299         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
300         unlike ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
301         every gap-less over-cap window and over-segments normal clips), this runs AFTER the R4

```

```

inactivity trim and
302         force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` x
303         ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
304         their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally
is left
305         alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
306         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
307         rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
308         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
309         """
310         if fps <= 0:
311             fps = 30.0
312         if frame_width <= 0:
313             frame_width = 1280.0
314
315         # Compute per-frame normalised velocity from the last *visible* point.
316         active = [] # (frame_idx, velocity)
317         prev: Optional[TrajectoryPoint] = None
318         for p in points:
319             if not p.visible:
320                 prev = None # break the trajectory; absent shuttle = not in flight
321                 continue
322             if prev is not None:
323                 dframes = p.frame - prev.frame
324                 if dframes > 0:
325                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
326                     velocity = (dist / frame_width) * (fps / dframes)
327                     if velocity > velocity_thresh:
328                         active.append((p.frame, velocity))
329             prev = p
330
331         if not active:
332             return []
333
334         max_gap_frames = int(merge_gap * fps)
335
336         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
337             vels = [v for _, v in group]
338             return {
339                 "start": group[0][0] / fps + chunk_start,
340                 "end": group[-1][0] / fps + chunk_start,
341                 "active_frame_count": len(group),
342                 "peak_velocity": max(vels),
343                 "mean_velocity": sum(vels) / len(vels),
344                 "track_id_count": 1, # single shuttle; kept for window-shape parity
345                 "chunk_index": chunk_index,
346             }
347
348         groups: List[List[Tuple[int, float]]] = []
349         group = [active[0]]
350         for af in active[1:]:
351             if af[0] - group[-1][0] <= max_gap_frames:
352                 group.append(af)
353             else:
354                 groups.append(group)
355                 group = [af]
356         groups.append(group)
357
358         if max_window_duration > 0:

```

```

359         groups = TrackNetRunner._split_overlong_groups(
360             groups, int(max_window_duration * fps), int(min_split_gap * fps),
361             hard_cap=window_hard_cap)
362
363     # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
364     # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
365     # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
366     # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
367     # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
368     if window_inactivity_end > 0:
369         tw = int(window_inactivity_end * fps)
370         groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
371                                                         inactivity_rally_thresh) for g
in groups]
372
373     # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
374     # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
375     # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
376     # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
377     forced_flags = [False] * len(groups)
378     if window_blob_fallback and max_window_duration > 0:
379         groups, forced_flags = TrackNetRunner._blob_fallback_chop(
380             groups, int(max_window_duration * fps), window_blob_factor)
381
382     windows = []
383     for g, forced in zip(groups, forced_flags):
384         w = _finalize(g)
385         if forced:
386             w["forced_cut"] = True
387         windows.append(w)
388     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
389
390     @staticmethod
391     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
392                             min_density: float, rally_thresh: float) -> List[Tuple[int,
float]]:
393         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
394         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
395         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
396         (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
397         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
398         LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
399         can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
400         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
401         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
402         if trail_frames <= 0 or len(group) < 2:
403             return group
404         frames = [f for f, _ in group]

```

```

405         fast = [1 if v > rally_thresh else 0 for _, v in group]
406         lo = 0
407         fast_sum = 0
408         last_dense = -1
409         for i in range(len(frames)):
410             fast_sum += fast[i]
411             while frames[lo] <= frames[i] - trail_frames:
412                 fast_sum -= fast[lo]
413                 lo += 1
414             win_count = i - lo + 1
415             if win_count > 0 and fast_sum / win_count >= min_density:
416                 last_dense = i
417         return group[: last_dense + 1] if last_dense >= 0 else group
418
419     @staticmethod
420     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
421                               min_split_frames: int,
422                               hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
423         """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
424         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
425         boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
426         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
427         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
428         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
429         bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
430         if max_win_frames <= 0:
431             return groups
432         out: List[List[Tuple[int, float]]] = []
433         work = list(groups)
434         while work:
435             g = work.pop()
436             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
437                 out.append(g)
438                 continue
439             best_i, best_gap = -1, -1
440             for i in range(1, len(g)):
441                 gap = g[i][0] - g[i - 1][0]
442                 if gap > best_gap:
443                     best_gap, best_i = gap, i
444             if best_i < 0 or best_gap < min_split_frames:
445                 # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
446                 # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
447                 if hard_cap:
448                     mid = (g[0][0] + g[-1][0]) / 2.0
449                     split_i = min(range(1, len(g)), key=lambda i: abs(g[i][0] - mid))
450                     work.append(g[:split_i])
451                     work.append(g[split_i:])
452                 else:
453                     out.append(g)
454                     continue
455             work.append(g[:best_i])
456             work.append(g[best_i:])
457         out.sort(key=lambda grp: grp[0][0])
458         return out
459

```

```

460     @staticmethod
461     def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
462                             blob_factor: float = 4.0
463                             ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
464         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
465         inactivity trim have each had their chance, a group still longer than
``blob_factor`` x
466         ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
467         signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
468         11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2x the cap)
469         is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
470         midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
471         degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
472         rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
473         can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
474         that piece came from a forced chop. Iterative worklist to bound stack depth."""
475         if max_win_frames <= 0:
476             return groups, [False] * len(groups)
477         blob_frames = max_win_frames * max(1.0, blob_factor)
478         out: List[List[Tuple[int, float]]] = []
479         flags: List[bool] = []
480         forced = 0
481         for g in groups:
482             if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
483                 out.append(g)
484                 flags.append(False)
485                 continue
486             pieces: List[List[Tuple[int, float]]] = []
487             work = [g]
488             while work:
489                 h = work.pop()
490                 if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
491                     pieces.append(h)
492                     continue
493                 mid = (h[0][0] + h[-1][0]) / 2.0
494                 split_i = min(range(1, len(h)), key=lambda i: abs(h[i][0] - mid)) #
noqa: B023
495                 work.append(h[:split_i])
496                 work.append(h[split_i:])
497             pieces.sort(key=lambda grp: grp[0][0])
498             out.extend(pieces)
499             flags.extend([True] * len(pieces))
500             forced += len(pieces) - 1
501         if forced:
502             logger.warning(
503                 "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
504                 "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
505                 "motion), not a bigger cap", forced)
506         return out, flags
507

```

12. assistant (2026-06-18T01:28:14.011Z)

Let me look at the hybrid segmenters and the personalization/serving and validators.

13. user (2026-06-18T01:28:15.004Z)

```
1      """WASB-based hybrid play segmenter.
2
3      Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4      WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5      badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7      For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8      candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9      min_window_duration) are what get tuned against golden rally labels.
10
11     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13     WASB runner and identity.
14     """
15     from typing import Any, Dict, Tuple
16
17     from backend.pipeline.segmenters.base import segmenter_registry
18     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19     from backend.pipeline.detectors.base import DetectorRunner
20     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23     class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24         family = "wasb"
25         display_label = "WASB"
26
27         @property
28         def name(self) -> str:
29             return "wasb_hybrid"
30
31         @property
32         def description(self) -> str:
33             return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
34                     "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
35
36         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
37             cfg = WasbConfig.from_indexing_cfg(idx_cfg)
38             return WasbRunner(cfg), {"weights_path": cfg.weights_path}
39
40
41     segmenter_registry.register(WasbHybridSegmenter)
42
```

14. user (2026-06-18T01:28:15.668Z)

```
1      """Per-user serving bridge ? resolve a user's fusion config + classifier from the v5
store.
2
3      PERSONALIZATION_PLAN.md §2 (Phase 1, the "#1 wall"): a personalized model is a per-user
4      ``Variant`` (cue_flags + ``min_crossings`` / ``min_players`` + an optional
``LogisticRegression``
5      + threshold). It does NOT require a segmenter rewrite ? it is a per-user
``FusionConfig`` overlay
6      resolved UPSTREAM of detection, plus a classifier the decision seam re-scores persisted
features
7      with. This module is that resolution step, and **only** that step.
8
9      What this module does (additive, pure-ish ? one DB read, no writes):
10     - ``resolve_user_fusion_config`` ? overlay the active ``user_models`` row's
``fusion_config``
```

```

11         onto the served fusion config dict, returning the config the served path WOULD use.
Falls
12         back to the served baseline (a copy, unchanged) when there is no active model, or
the model
13         is below the promotion-confidence floor (`n_videos_at_fit < min_n`), or it carries
no
14         fusion overlay. The min-N floor here is the SAME `per_user_gate` promotion floor:
below it
15         a per-user A/B is not trusted enough to override a baseline default (PLAN §1.3).
16         - `load_user_classifier` ? rehydrate the stored `classifier_json` into a
17         `LogisticRegression` via `from_dict` (None when the user has no active model or
it is a
18         config-only model). `LogisticRegression.load` is never called in
`backend/pipeline/`
19         today ? this is the load half of the serving bridge.
20
21         ? NOT WIRED INTO THE SERVED PATH. `fusion_hybrid._select_for_handoff` still reads only
the
22         static `idx_cfg["fusion"]` ? it does NOT call anything here. Wiring this resolver into
that
23         seam (so a pinned, promoted per-user model actually changes the served handoff) is the
NEXT,
24         **owner-gated** PR, because it is the first change to served behavior. Until then this
is pure
25         groundwork: import it, unit-test it, but nothing in production reads it. With NULL
`user_id`
26         (the local install) and no active model, `resolve_user_fusion_config` returns the
served
27         config unchanged ? byte-identical to today.
28         """
29         from __future__ import annotations
30
31         import copy
32         from typing import Any, Dict, Optional
33
34         from backend.eval.classifier import LogisticRegression
35
36         __all__ = [
37             "DEFAULT_MIN_N",
38             "resolve_user_fusion_config",
39             "load_user_classifier",
40         ]
41
42         #: Promotion-confidence floor (PERSONALIZATION_PLAN.md §1.3, owner-confirmed `min_n =
5`)).
43         #: A per-user model fit on fewer than this many of the user's videos is NOT trusted
enough to
44         #: override a baseline fusion default ? we fall back to the served baseline. This
mirrors
45         #: `backend.eval.per_user_gate.should_promote_for_user`'s `min_n` default.
46         DEFAULT_MIN_N = 5
47
48
49         def resolve_user_fusion_config(
50             db: Any,
51             user_id: Optional[str],
52             served_cfg: Dict[str, Any],
53             *,
54             min_n: int = DEFAULT_MIN_N,
55         ) -> Dict[str, Any]:
56             """Resolve the fusion config the served path should use for `user_id`.
57
58             Overlays the active `user_models` row's `fusion_config` (a partial per-user
59             `FusionConfig` dict ? e.g. raised `min_crossings` / `min_players` / flipped
60             `cue_flags`) onto a COPY of `served_cfg` (the baseline served fusion config

```

```

dict, i.e.
61     ``idx_cfg["fusion"]``). The original ``served_cfg`` is never mutated.
62
63     Falls back to the served baseline (an unmodified deep copy) when ANY of:
64     - the user has no active model;
65     - the active model was fit below the promotion floor (``n_videos_at_fit < min_n``)
? too
66         little of the user's own footage to trust a per-user override (PLAN $1.3);
67     - the active model carries no ``fusion_config`` overlay (a classifier-only model).
68
69     So a brand-new / NULL (local) user with no active model gets the served baseline
unchanged
70     ? never denied service, never altered. The overlay is a shallow key-level merge:
per-user
71     keys win, every key the user did not set keeps the baseline value.
72     """
73     base = copy.deepcopy(served_cfg)
74     model = db.get_active_user_model(user_id)
75     if not model:
76         return base
77     if int(model.get("n_videos_at_fit") or 0) < min_n:
78         # Below the trust floor ? keep the baseline (the conservative default).
79         return base
80     overlay = model.get("fusion_config")
81     if not overlay:
82         return base # config-only fallback: a classifier-only model has nothing to
overlay
83     base.update(overlay)
84     return base
85
86
87     def load_user_classifier(db: Any, user_id: Optional[str]) ->
Optional[LogisticRegression]:
88     """Rehydrate the active per-user classifier for ``user_id``, or None.
89
90     Returns a ``LogisticRegression`` rebuilt from the stored ``classifier_json`` (via
91     ``from_dict``) when the user has an active model that carries one; None when there
is no
92     active model or it is a config-only (no-classifier) model. The caller (the FUTURE
served
93     wiring) would feed this model's ``predict_proba`` over the persisted
``window_stats`` ?
94     the same math ``experiment.predict`` runs offline. Nothing calls this on the served
path
95     yet; it is the load half of the serving bridge.
96     """
97     model = db.get_active_user_model(user_id)
98     if not model:
99         return None
100     blob = model.get("classifier_json")
101     if not blob:
102         return None
103     return LogisticRegression.from_dict(blob)
104

```

15. assistant (2026-06-18T01:28:18.314Z)

Let me look at the trajectory_hybrid (the engine), motion segmenter, and gemini segmenter (parse_time, indexing).

16. user (2026-06-18T01:28:19.289Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a

```

```

4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27
28    from backend.pipeline.segmenters.base import BasePlaySegmenter
29    from backend.utils.video import get_video_duration, extract_video_chunk
30    from backend.sports import sport_registry
31    from backend.providers import provider_registry
32    from backend.pipeline.detectors.base import DetectorRunner
33
34    logger = logging.getLogger(__name__)
35
36
37    def _fmt_ts(seconds: float) -> str:
38        seconds = max(0, int(seconds))
39        h, rem = divmod(seconds, 3600)
40        m, s = divmod(rem, 60)
41        return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44    class TrajectoryHybridSegmenter(BasePlaySegmenter):
45        """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46        boundaries."""
47
48        #: config section under `indexing` (velocity_threshold lives there), the
49        #: output subdir for trajectories, and the metadata detector-tag prefix.
50        family: str = ""
51        #: human-readable label used in log lines.
52        display_label: str = ""
53
54        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55        Any]]:
56
57            """Return (runner, detector-specific detection_params extras)."""
58            raise NotImplementedError
59
60        @staticmethod
61        def _probe_fps_width(video_path: str) -> tuple:
62            """Return (fps, frame_width) for a video, with sensible fallbacks."""
63            cap = cv2.VideoCapture(video_path)
64            fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
65            width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
66            cap.release()
67            return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
68
69        @staticmethod

```

```

67     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
68         """Provider-reported exchange count, else a duration-based estimate.
69
70         One canonical implementation ? the twins had drifted (wasb relied on
71         ``int(None)`` raising into the fallback; same result, by accident).
72         """
73         shot_count = segment.get("shuttle_exchange_count")
74         if shot_count is None:
75             return max(3, int(duration / 0.8))
76         try:
77             return int(shot_count)
78         except (ValueError, TypeError):
79             return max(3, int(duration / 0.8))
80
81     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
82     -----
83     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
84                                frame_width: float, video_path: str,
85                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
86         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
87         BASE
88         returns exactly the 4 motion keys, so the trajectory twins persist byte-
89         identical
90         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
91         features). ``points`` are the whole-video trajectory points
92         (TrajectoryPoint)."""
93         return {
94             "peak_velocity": cw["peak_velocity"],
95             "mean_velocity": cw["mean_velocity"],
96             "active_frame_count": cw["active_frame_count"],
97             "track_id_count": cw["track_id_count"],
98         }
99
100     def _select_for_handoff(self, windows: List[Dict[str, Any]],
101                            window_stats: List[Dict[str, Any]],
102                            idx_cfg: Dict[str, Any]) -> List[int]:
103         """Indices of the candidate windows that proceed to the AI handoff (and thus may
104         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
105         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
106         Persisted candidates are NOT affected ? they remain the full superset for
107         offline
108         re-scoring."""
109         return list(range(len(windows)))
110
111     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
112                      player_pool: Optional[List[str]] = None,
113                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
114         # override-ignore: the ABC declares the Result contract (Success|Failure)
115         # but every live segmenter still returns a bare list ? the documented
116         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
117         label = self.display_label
118         # --- Sport profile + AI provider wiring (identical across segmenters) ---
119         sport_name = self.config.get("sport", "badminton")
120         try:
121             sport_profile = sport_registry.get(sport_name)
122         except KeyError as e:
123             logger.error(str(e))
124             return []
125
126         idx_cfg = self.config.get("indexing", {})
127         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
128         the
129         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
130         is
131         # needed and nothing is uploaded. Used to run the local detector standalone

```

```

(e.g. to
125         # measure it against the Gemini path, or to avoid the cloud entirely). With
126         # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
127         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
128         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
129         if skip_ai:
130             model_name = "local-noai"
131             logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
132                         "be emitted as rallies; no %s handoff, no API key needed.",
133                         label, provider_name)
134         else:
135             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
136             api_key_env_var = f"{provider_name.upper()}_API_KEY"
137             api_key = os.environ.get(api_key_env_var)
138             if not api_key:
139                 logger.error(
140                     f"{api_key_env_var} environment variable is not set! "
141                     f"To run the {provider_name} handoff, set your API key. "
142                     f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
143                 )
144             return []
145         try:
146             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
147         except KeyError as e:
148             logger.error(str(e))
149             return []
150
151         video_duration = get_video_duration(video_path)
152         if video_duration <= 0:
153             logger.error("Invalid video duration.")
154             return []
155         fps, frame_width = self._probe_fps_width(video_path)
156
157         # --- Runner config + windowing thresholds ---
158         runner, runner_params = self._build_runner(idx_cfg)
159
160         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
161         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
162         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
163         # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
164         max_window_duration = idx_cfg.get("max_window_duration", 0.0)
165         window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
166         window_hard_cap = idx_cfg.get("window_hard_cap", False)
167         window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
168         inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
169         inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
170         window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
171         window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
172         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
173         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
174         log_candidates = idx_cfg.get("log_yolo_candidates", True)
175         store_candidates = idx_cfg.get("store_yolo_candidates", True)
176
177         detection_params = {
178             "detector": self.name,
179             "velocity_thresh": velocity_thresh,
180             "merge_gap": merge_gap,
181             "min_action_window_duration": min_window_duration,
182             **runner_params,

```

```

183     }
184
185     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
186     ok, msg = runner.healthcheck()
187     if not ok:
188         logger.error("%s WSL environment not ready: %s", label, msg)
189         return []
190     logger.info("%s WSL env OK: %s", label, msg)
191
192     # --- Phase 2: trajectory -> candidate rally windows ---
193     # Trajectory detectors process the whole video in one pass (they carry
194     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
195     t_start = time.time()
196     out_dir = os.path.join("output", self.family)
197     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
198     if not csv_path:
199         logger.error("%s produced no trajectory; aborting.", label)
200         return []
201
202     points = runner.parse_trajectory_csv(csv_path)
203     logger.info("Parsed %d trajectory points (%d visible).",
204                 len(points), sum(1 for p in points if p.visible))
205
206     all_candidate_windows = runner.trajectory_to_action_windows(
207         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
208         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
209         min_window_duration=min_window_duration, chunk_index=0,
210         max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
211         window_hard_cap=window_hard_cap,
212         window_inactivity_end=window_inactivity_end,
213         inactivity_rally_thresh=inactivity_rally_thresh,
214         inactivity_min_density=inactivity_min_density,
215         window_blob_fallback=window_blob_fallback,
216         window_blob_factor=window_blob_factor,
217     )
218     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
219                 label, time.time() - t_start, len(all_candidate_windows))
220
221     if log_candidates:
222         for cw in all_candidate_windows:
223             logger.info(f"[{label} rally]
224 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
225                        f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
226                        f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")
227
228     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
229     # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
230     # persistence and the handoff gate below.
231     window_stats = [
232         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
233         for cw in all_candidate_windows
234     ]
235
236     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
237     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
238     if store_candidates:
239         for i, cw in enumerate(all_candidate_windows):
240             candidate_ids[i] = self.db.add_candidate_segment(
241                 video_id, detector=self.name,
start_time=cw["start"], end_time=cw["end"],

```

```

242             chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"])),
243             stats=window_stats[i], detection_params=detection_params,
244         )
245
246     if not all_candidate_windows:
247         return []
248
249     # --- Phase 3: AI provider handoff over padded candidate clips ---
250     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
251     # candidates above stay the full superset ? only the served play_segments are
gated.
252     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
253
254     ai_segments: List[dict] = []
255     if skip_ai:
256         # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
257         # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
258         # falls back to the duration-based estimate (no AI exchange count).
259         ai_segments = [
260             {
261                 "start_time": all_candidate_windows[wi]["start"],
262                 "end_time": all_candidate_windows[wi]["end"],
263                 "shuttle_exchange_count": None,
264                 "shot_count_confidence": "LocalNoAI",
265                 "_candidate_id": candidate_ids[wi],
266             }
267             for wi in handoff_indices
268         ]
269         logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
270                     "(no AI handoff).", len(ai_segments))
271     else:
272         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
273         os.makedirs(handoff_dir, exist_ok=True)
274         logger.info("Phase 3: %s validation on %d candidate windows...",
275                     provider_name, len(handoff_indices))
276
277         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
278             w_start = max(0, window["start"] - handoff_padding)
279             w_end = min(video_duration, window["end"] + handoff_padding)
280             w_dur = w_end - w_start
281             w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
282             if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
283                 return []
284             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
285             segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
286                                             label=f"Handoff {wi+1}")
287             valid = []
288             for seg in segments:
289                 try:
290                     seg["start_time"] = float(seg["start_time"]) + w_start
291                     seg["end_time"] = float(seg["end_time"]) + w_start
292                     seg["_candidate_id"] = candidate_ids[wi]
293                     valid.append(seg)
294                 except (ValueError, TypeError, KeyError) as e:
295                     logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
296             try:

```



```

297         os.remove(w_path)
298     except OSError:
299         pass
300     return valid
301
302     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
303         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
304                     for wi in handoff_indices]
305         for future in futures:
306             ai_segments.extend(future.result())
307         try:
308             os.rmdir(handoff_dir)
309         except OSError:
310             pass
311         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
312
313     # --- Phase 4: DB population + candidate linking ---
314     segments_data = []
315     for segment in ai_segments:
316         try:
317             start = float(segment["start_time"])
318             end = float(segment["end_time"])
319             dur = end - start
320             metadata = {
321                 "shot_count": self._shot_count_from(segment, dur),
322                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
323                 "detector": f"{self.family}-hybrid-{model_name}",
324             }
325             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
326             if segment_id:
327                 candidate_id = segment.get("_candidate_id")
328                 if candidate_id is not None:
329                     self.db.link_candidate_to_segment(candidate_id, segment_id)
330                 segments_data.append({
331                     "id": segment_id, "start_time": start, "end_time": end,
332                     "duration": dur, "metadata": metadata,
333                 })
334         except Exception as e:
335             logger.warning("Skipping invalid segment struct: %s", e)
336
337     return segments_data
338

```

17. assistant (2026-06-18T01:28:21.813Z)

Now let me look at the remaining detector/segmenter files, particularly fusion_hybrid, fusion_features, person_detector, audio, and the validators.

18. user (2026-06-18T01:28:23.633Z)

```

1     """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3     NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4     math is unit-testable in a GPU-free dev container. The detection (person boxes,
5     shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6     module only turns those signals into a handful of scale/translation-invariant
7     features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8     memorise this court/camera and fail to generalise off a small labelled set).
9

```

```

10     Conventions:
11     - A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
needs it.
12     - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13         index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14     - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15     - ``court_polygon`` is normalised 0-1 ``(x, y), ...`` or ``None`` (None ? no mask,
every
16         detection counts ? the player-count-only mode).
17     - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19     Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20     floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21     """
22     from __future__ import annotations
23
24     from typing import Dict, List, Optional, Sequence, Tuple
25
26     BBox = Tuple[float, float, float, float]
27     Net = Tuple[str, float]
28
29
30     def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31         """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32         (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33         if polygon is None or len(polygon) < 3:
34             return False
35         x, y = point
36         inside = False
37         n = len(polygon)
38         j = n - 1
39         for i in range(n):
40             xi, yi = polygon[i]
41             xj, yj = polygon[j]
42             # Does the horizontal ray at y cross edge (i, j)?
43             if (yi > y) != (yj > y):
44                 x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                 if x < x_cross:
46                     inside = not inside
47             j = i
48         return inside
49
50
51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, _y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65

```

```

66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                          court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
71         point meaningless, so a masked check returns False rather than testing a bogus
72         (0,0)."""
73         if court_polygon is None:
74             return True
75         if frame_width <= 0 or frame_height <= 0:
76             return False
77         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
78
79     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
80                        frame_height: float,
81                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
82     List[int]:
83         """Per-frame count of in-court persons (one entry per sampled frame)."""
84         return [
85             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
86 court_polygon))
87             for _frame_idx, dets in sorted(per_frame.items())
88         ]
89
90     def _median(vals: Sequence[float]) -> float:
91         if not vals:
92             return 0.0
93         s = sorted(vals)
94         n = len(s)
95         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
96
97     def players_in_court(counts: Sequence[int]) -> float:
98         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99         return _median(list(counts))
100
101
102     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
103         """Fraction of sampled frames with >= `min_players` in court (track stability vs
104 flicker)."""
105         if not counts:
106             return 0.0
107         return sum(1 for c in counts if c >= min_players) / len(counts)
108
109     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
110                           frame_width: float, frame_height: float) -> float:
111         """Mean per-track centre displacement between consecutive sampled frames, with x
112         normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
113         not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
114         if frame_width <= 0 or len(per_frame) < 2:
115             return 0.0
116         frames = sorted(per_frame.keys())
117         steps: List[float] = []
118         for a, b in zip(frames, frames[1:]):
119             prev = {tid: box for tid, box in per_frame[a]}
120             for tid, box in per_frame[b]:
121                 if tid in prev:
122                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
123                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
124                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
125         if not steps:
126             return 0.0

```

```

127         return sum(steps) / len(steps)
128
129
130     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
131                          frame_height: float, net: Optional[Net],
132                          court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
133         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
134
135         A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
136         The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
137         0.0.
138
139         """
140         if net is None or not per_frame:
141             return 0.0
142         axis, pos = net
143         both = 0
144         for _frame_idx, dets in per_frame.items():
145             near, far = False, False
146             for _tid, box in dets:
147                 if not person_in_court(box, frame_width, frame_height, court_polygon):
148                     continue
149                 fx, fy = _foot_norm(box, frame_width, frame_height)
150                 coord = fx if axis == "x" else fy
151                 if coord < pos:
152                     near = True
153                 else:
154                     far = True
155             if near and far:
156                 both += 1
157         return both / len(per_frame)
158
159     def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
160 float,
161                      pad_frac: float) -> bool:
162         """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
163         width/height (the 'adjacent / in-hand' tolerance)?"""
164         x1, y1, x2, y2 = box
165         px = pad_frac * frame_width
166         py = pad_frac * frame_height
167         return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
168
169     def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
170 shuttle_points,
171                                frame_width: float, frame_height: float,
172                                pad_frac: float = 0.03) -> float:
173         """Fraction of (visible shuttle frames that were also person-sampled) in which the
174         shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
175         ~0**
176
177         ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
178         """
179         if not per_frame or frame_width <= 0 or frame_height <= 0:
180             return 0.0
181         aligned = 0
182         colocated = 0
183         for p in shuttle_points or []:
184             if not getattr(p, "visible", False):
185                 continue
186             dets = per_frame.get(int(p.frame))
187             if dets is None:
188                 # shuttle frame wasn't person-sampled ? can't judge
189                 continue
190             aligned += 1
191             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
192                    for _tid, box in dets):

```

```

188         colocated += 1
189     return colocated / aligned if aligned else 0.0
190
191
192     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
193     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
194     PERSON_FEATURE_NAMES = [
195         "players_in_court", "two_sided_motion", "mean_player_motion",
196         "in_court_ratio", "shuttle_player_colocation",
197     ]
198
199
200     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
201 shuttle_points,
202         frame_width: float, frame_height: float,
203         court_polygon: Optional[Sequence[Tuple[float, float]]] =
204         None,
205         net: Optional[Net] = None,
206         min_players: int = 2) -> Dict[str, float]:
207         """Aggregate the per-window person features from per-frame detections + the in-
208 window
209 shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
210         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
211         return {
212             "players_in_court": players_in_court(counts),
213             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
214 court_polygon),
215             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
216             "in_court_ratio": in_court_ratio(counts, min_players),
217             "shuttle_player_colocation": shuttle_player_colocation(
218                 per_frame, shuttle_points, frame_width, frame_height),
219         }

```

19. user (2026-06-18T01:28:23.888Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11    (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
13    the
14    torchvision/supervision model ? so the default (person-OFF) path never loads a
15    detector
16    model nor touches the GPU. (torch/cv2 themselves are imported by the registry
17    regardless,
18    via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
19    - ``select_for_handoff`` ? optional served Gate A/B: when
20    ``indexing.fusion.min_crossings``
21    (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
22    are
23    dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
24    **Persisted candidates remain the full superset** regardless, so the offline
25    experiment
26    harness can re-score any variant.
27
28    Defaults reproduce the substrate exactly: net_crossings/person features are persisted
29    but

```

```

23     nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26     experiment harness (EXPERIMENT_HARNESS.md), never silently.
27     """
28     from typing import Any, Dict, List, Optional, Tuple
29
30     from backend.pipeline.segmenters.base import segmenter_registry
31     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32     from backend.pipeline.detectors.base import DetectorRunner
33     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35     from backend.pipeline.detectors import rally_gate
36     from backend.pipeline.detectors import fusion_features as ff
37
38
39     class FusionSegmenter(TrajectoryHybridSegmenter):
40         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42         family = "fusion"
43         display_label = "Fusion"
44
45         def __init__(self, db: Any, config: Any):
46             super().__init__(db, config)
47             # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48             self._person: Optional[Any] = None
49             # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50             # over the whole trajectory once per candidate window (it depends only on
`points`).
51             self._axis_cache_key: Optional[int] = None
52             self._axis_cache: Optional[tuple] = None
53
54             @property
55             def name(self) -> str:
56                 return "fusion_hybrid"
57
58             @property
59             def description(self) -> str:
60                 return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                         "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                         "AI provider sets semantic boundaries.")
63
64             def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65                 substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66                 if substrate == "tracknet":
67                     cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68                     return TrackNetRunner(cfg), {
69                         "weights_path": cfg.weights_path,
70                         "queue_length": cfg.queue_length,
71                         "fusion_substrate": "tracknet",
72                     }
73                 wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74                 return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76             def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,

```

```

77             frame_width: float, video_path: str,
78             idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82         # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83         gated = rally_gate.gate_windows(points, [[cw["start"], cw["end"]]], fps,
84             min_crossings=0,
estimate=self._axis_estimate(points))
85         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87         fcfg = idx_cfg.get("fusion", {})
88         if fcfg.get("cue_flags", {}).get("person", False):
89             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90         return stats
91
92         # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
93         # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
94         # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
95         # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
96         # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
97         # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
98         # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
99
100         def _axis_estimate(self, points: List[Any]) -> tuple:
101             """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
102             computed once per video, not once per candidate window (it depends only on
points)."""
103             key = id(points)
104             if self._axis_cache_key != key or self._axis_cache is None:
105                 self._axis_cache_key = key
106                 self._axis_cache = rally_gate.estimate_play_axis(points)
107             est = self._axis_cache
108             assert est is not None
109             return est
110
111         def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
112             frame_width: float, video_path: str,
113             fcfg: Dict[str, Any]) -> Dict[str, float]:
114             """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
115             frame range and compute the scale/translation-invariant person features. Lazily
builds
116             ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
117             # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
118             from backend.pipeline.detectors.person_detector import PersonDetector
119             if self._person is None:
120                 self._person = PersonDetector(self.config)
121
122             frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
123             # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a

```

```

124         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
125         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
126         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
127         fw = det["frame_width"] or frame_width
128         fh = det["frame_height"]
129         # If we can't trust the frame dims, every normalised feature would silently
collapse
130         # to -0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
131         if fw <= 0 or fh <= 0:
132             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
133
134         poly = fcfg.get("court_polygon")
135         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
136         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
137         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
138         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
139                                         court_polygon=polygon, net=net)
140
141     def _select_for_handoff(self, windows: List[Dict[str, Any]],
142                           window_stats: List[Dict[str, Any]],
143                           idx_cfg: Dict[str, Any]) -> List[int]:
144         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
145         (0) keep every window ? identical to the substrate. Persisted candidates are
146         untouched (full superset).
147
148         NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
149         thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
150         where no segment clears ``min_crossings``), the gate falls back to the full
ungated
151         set rather than emit a video with zero rallies. This makes Gate A/B strictly
152         *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
153         the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
154         below it always wins on an otherwise-empty video. Observed on real footage
155         (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
156         gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
157         fcfg = idx_cfg.get("fusion", {})
158         min_crossings = int(fcfg.get("min_crossings", 0))
159         min_players = int(fcfg.get("min_players", 0))
160         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
161
162         keep: List[int] = []
163         for i, st in enumerate(window_stats):
164             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
165                 continue
166             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
167                 continue
168             keep.append(i)
169
170         if not keep and window_stats:
171             # Gating emptied a non-empty video ? fall back to the ungated set.
172             return list(range(len(window_stats)))
173         return keep
174
175
176     segmenter_registry.register(FusionSegmenter)
177

```


20. assistant (2026-06-18T01:28:27.905Z)

Let me examine the validators, motion segmenter, gemini, person_detector, audio hit_detector, and the wasb_infer.

21. user (2026-06-18T01:28:28.992Z)

```
1     import subprocess
2     from backend.pipeline.validators.base import VideoValidator, ValidationResult
3     from typing import Dict, Any
4
5     class PTSContinuityValidator(VideoValidator):
6         @property
7         def name(self) -> str:
8             return "pts_continuity_check"
9
10        @property
11        def description(self) -> str:
12            return "Analyzes Presentation Time Stamps (PTS) of keyframes to check for
13            timeline gaps, jumps, or editing splices."
14
15        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
16        -> ValidationResult:
17            # Command to extract packet PTS and flags (extremely fast, works on H.264 &
18            H.265)
19            cmd = [
20                "ffprobe",
21                "-v", "error",
22                "-select_streams", "v:0",
23                "-show_entries", "packet=pts_time,flags",
24                "-of", "csv=p=0",
25                video_path
26            ]
27
28            try:
29                result = subprocess.run(cmd, capture_output=True, text=True, check=True)
30                output = result.stdout.strip()
31
32                if not output:
33                    return ValidationResult(
34                        validator_name=self.name,
35                        passed=False,
36                        message="No packet timestamps could be extracted from the video
37                        stream."
38                    )
39
40                timestamps = []
41                for line in output.split("\n"):
42                    line = line.strip()
43                    if line:
44                        parts = line.split(",")
45                        if len(parts) == 2:
46                            pts_str, flags = parts
47                            if "K" in flags: # Check if keyframe
48                                try:
49                                    timestamps.append(float(pts_str))
50                                except ValueError:
51                                    continue
52
53                if len(timestamps) < 2:
54                    return ValidationResult(
55                        validator_name=self.name,
56                        passed=True,
57                        message="Video is too short to perform a PTS continuity check.",
58                        details={"keyframes_count": len(timestamps)}
```

```

55         )
56
57     # Analyze gaps
58     max_gap = 0.0
59     backward_jumps = 0
60     gaps = []
61
62     for i in range(1, len(timestamps)):
63         gap = timestamps[i] - timestamps[i-1]
64         gaps.append(gap)
65         if gap < 0:
66             backward_jumps += 1
67         if gap > max_gap:
68             max_gap = gap
69
70     # An continuous GoPro recording usually has keyframe intervals (GOP size) of
71     1.0 to 4.0 seconds.
72     # If there's a keyframe gap larger than 8.0 seconds or a negative gap
73     (backward jump), it's highly likely spliced.
74     details = {
75         "keyframes_count": len(timestamps),
76         "max_keyframe_gap_seconds": round(max_gap, 2),
77         "backward_jumps_detected": backward_jumps,
78         "first_keyframe": timestamps[0],
79         "last_keyframe": timestamps[-1]
80     }
81
82     if backward_jumps > 0:
83         return ValidationResult(
84             validator_name=self.name,
85             passed=False,
86             message="Video integrity compromise: Backward timeline jumps
87             (splicing) detected.",
88             details=details
89         )
90
91     # Threshold for sudden gap (e.g. 10.0 seconds)
92     if max_gap > 10.0:
93         return ValidationResult(
94             validator_name=self.name,
95             passed=False,
96             message=f"Video integrity compromise: Detected a timestamp gap of
97             {round(max_gap, 2)}s, indicating edited or missing content.",
98             details=details
99         )
100
101     return ValidationResult(
102         validator_name=self.name,
103         passed=True,
104         message="PTS timeline is continuous with no abnormal gaps or backward
105         jumps.",
106         details=details
107     )
108
109 except subprocess.CalledProcessError as e:
110     return ValidationResult(
111         validator_name=self.name,
112         passed=False,
113         message="Failed to execute ffprobe PTS analysis.",
114         details={"error": e.stderr}
115     )
116
117 except Exception as e:
118     return ValidationResult(
119         validator_name=self.name,
120         passed=False,

```

```

115         message=f"Unexpected error in PTS validation: {str(e)}"
116     )
117

```

22. user (2026-06-18T01:28:29.627Z)

```

1     import cv2
2     from backend.pipeline.validators.base import VideoValidator, ValidationResult
3     from typing import Dict, Any
4
5     class SceneCutDensityValidator(VideoValidator):
6         @property
7         def name(self) -> str:
8             return "scene_cut_check"
9
10        @property
11        def description(self) -> str:
12            return "Analyzes visual frames to detect abrupt scene cuts, which indicate an
13            edited or compiled video."
14
15        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
16            cap = cv2.VideoCapture(video_path)
17            if not cap.isOpened():
18                return ValidationResult(
19                    validator_name=self.name,
20                    passed=False,
21                    message="Failed to open video file using OpenCV."
22                )
23
24            fps = cap.get(cv2.CAP_PROP_FPS)
25            total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
26            duration = total_frames / fps if fps > 0 else 0
27
28            if duration <= 0:
29                cap.release()
30                return ValidationResult(
31                    validator_name=self.name,
32                    passed=False,
33                    message="Invalid video duration."
34                )
35
36            # Sample at 2 frames per second to keep it fast
37            sample_interval = max(1, int(fps / 2))
38
39            cuts_detected = 0
40            prev_hist = None
41
42            frame_idx = 0
43            sampled_count = 0
44
45            # Adjust histogram comparison thresholds
46            # Using correlation: 1.0 is perfect match, < 0.6 indicates a sudden massive
scene change
47            cut_threshold = 0.6
48
49            while True:
50                cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
51                ret, frame = cap.read()
52                if not ret:
53                    break
54
55                # Calculate color histogram
56                hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
57                hist = cv2.calcHist([hsv], [0, 1], None, [50, 60], [0, 180, 0, 256])

```

```

57         cv2.normalize(hist, hist, alpha=0, beta=1, norm_type=cv2.NORM_MINMAX)
58
59         if prev_hist is not None:
60             # Compare correlation
61             score = cv2.compareHist(prev_hist, hist, cv2.HISTCMP_CORREL)
62             if score < cut_threshold:
63                 cuts_detected += 1
64
65         prev_hist = hist
66         sampled_count += 1
67         frame_idx += sample_interval
68
69         # Cap check at 100 frames max (e.g. first 50 seconds or evenly spread) to
keep validation instant
70         if frame_idx >= total_frames or sampled_count > 100:
71             break
72
73         cap.release()
74
75         # Compute cuts per minute
76         sampled_duration = (sampled_count * sample_interval) / fps if fps > 0 else 1
77         cuts_per_minute = (cuts_detected / sampled_duration) * 60 if sampled_duration >
0 else 0
78
79         details = {
80             "cuts_detected": cuts_detected,
81             "sampled_frames": sampled_count,
82             "cuts_per_minute": round(cuts_per_minute, 3)
83         }
84
85         val_cfg = config.get("validation", {})
86         max_cuts_rate = val_cfg.get("max_scene_cuts_per_minute", 0.5)
87
88         if cuts_per_minute > max_cuts_rate:
89             return ValidationResult(
90                 validator_name=self.name,
91                 passed=False,
92                 message=f"Video splicing compromise: Detected {cuts_detected} cuts
({round(cuts_per_minute, 2)} cuts/min), which exceeds the continuous raw file limit of
{max_cuts_rate} cuts/min.",
93                 details=details
94             )
95
96         return ValidationResult(
97             validator_name=self.name,
98             passed=True,
99             message="Scene continuity check passed. No abnormal visual cuts detected.",
100             details=details
101         )
102

```

23. assistant (2026-06-18T01:28:36.329Z)

Let me look at the person_detector, audio hit_detector, motion segmenter, and the base detector mixin.

24. user (2026-06-18T01:28:37.269Z)

```

1         """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3         Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4         the person cue is a swappable producer**, not logic baked into one segmenter. Today
5         `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6         next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and

```

```

7      later the learned classifier reads its persisted features. Keeping detection here (not
in
8      the segmenter) is what makes "add a cue = register a producer" true.
9
10     Behaviour is unchanged from the in-lined version ? only the home moved:
11     - `lazy_load_model`           ? load a permissively-licensed torchvision detector
12     - `make_tracker`             ? a fresh supervision ByteTrack per chunk
13     - `detect_and_track`         ? one frame ? confident persons with persistent IDs
14     - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15       with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16       that feed the candidate `stats` JSON and the learned fusion features.
17
18     The model libs (torchvision, supervision) are imported lazily inside the methods, and
the
19     detector weights load only on first detection (`lazy_load_model`) ? so importing the
20     segmenter registry never loads a detector model, and tests can mock the seams. (torch +
cv2
21     are top-level imports here; CI's import-smoke stubs them, and they're present on the GPU
22     machine.) Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
23     (ADR-005; ultralytics' AGPL YOLO was dropped).
24     """
25     import logging
26     from typing import Any, Callable, Dict, List, Optional, Tuple
27
28     import cv2
29     import torch
30
31     from backend.utils.geometry import get_velocity_normalised
32
33     logger = logging.getLogger(__name__)
34
35     # torchvision detection models are trained on COCO where the "person" category is
36     # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
37     # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
38     _PERSON_CLASS_ID = 1
39
40     # Permissively-licensed torchvision detectors selectable via config
41     # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
42     _DETECTOR_FACTORIES = {
43         "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
44         "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
45         "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
46     }
47     _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
48
49
50     class PersonDetector:
51         """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
52
53         Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
54         ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
55         min_action_window_duration}``); the model loads lazily on first use.
56         """
57
58         def __init__(self, config: Dict[str, Any]):
59             self.config = config
60             # Dynamic torchvision module (None until lazy_load_model); typed Any so the
61             # `self.model([tensor])` inference call type-checks without a quarantine.
62             self.model: Any = None
63             self.device: Optional[str] = None
64
65         def lazy_load_model(self) -> None:
66             if self.model is not None:
67                 return
68

```

```

69         # Imported lazily so the module imports without torchvision present and so
70         # the test suite (which pre-sets self.model) never pulls in model weights.
71         from torchvision.models import detection as tv_detection
72
73         idx_cfg = self.config.get("indexing", {})
74         use_gpu = idx_cfg.get("use_gpu", True)
75         self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
76         if use_gpu and not torch.cuda.is_available():
77             logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
78
79         model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
80         if model_name not in _DETECTOR_FACTORIES:
81             logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
82             model_name = _DEFAULT_DETECTOR
83
84         builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
85         logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
86         # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
87         self.model = builder(weights="DEFAULT")
88         self.model.eval()
89         self.model.to(self.device)
90
91     def make_tracker(self, fps: float) -> Any:
92         """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
93
94         Imported lazily so tests can mock this method without supervision installed.
95         """
96         import supervision as sv
97         frame_rate = max(1, int(round(fps)))
98         return sv.ByteTrack(frame_rate=frame_rate)
99
100     def detect_and_track(self, frame: Any, tracker: Any,
101                         score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
102         """Detect persons in a frame and assign persistent track IDs.
103
104         Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
105         This replaces the old ultralytics ``model.track(...)`` call, which bundled
106         detection AND association: torchvision does the detection, supervision's
107         ByteTrack does the association.
108         """
109         import supervision as sv
110
111         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
112         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
113         tensor = torch.from_numpy(rgb).permute(2, 0,
1).float().div(255.0).to(self.device)
114         with torch.no_grad():
115             outputs = self.model([tensor])
116
117         out = outputs[0]
118         boxes = out["boxes"].detach().cpu().numpy()
119         labels = out["labels"].detach().cpu().numpy()
120         scores = out["scores"].detach().cpu().numpy()
121
122         # Keep only confident person detections.
123         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
124         if not mask.any():
125             return []
126
127         # ByteTrack requires confidence to be populated (it filters on it internally).
128         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])

```

```

129         tracked = tracker.update_with_detections(dets)
130
131     result: List[Tuple[int, Tuple[float, float, float, float]]] = []
132     for i in range(len(tracked)):
133         tid = tracked.tracker_id[i]
134         if tid is None:
135             continue
136         x1, y1, x2, y2 = tracked.xyxy[i]
137         result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
138     return result
139
140     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
141                                         velocity_thresh: float, merge_gap: float,
142                                         frame_skip: int = 3,
143                                         chunk_index: Optional[int] = None) ->
144     List[Dict[str, Any]]:
145         """
146         Runs person detection + tracking on a video chunk and extracts high-motion
147         action windows. Returns a list of dicts in the full video's timeframe, each
148         with:
149             start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
150             track_id_count, chunk_index.
151
152         Args:
153             frame_skip: Process every Nth frame (1 = every frame). Higher values
154                         dramatically reduce inference cost with minimal quality loss.
155             chunk_index: Index of the source chunk (recorded on each window for
156                         traceability).
157         """
158         cap = cv2.VideoCapture(chunk_path)
159         if not cap.isOpened():
160             logger.error(f"Could not open {chunk_path}")
161             return []
162
163         fps = cap.get(cv2.CAP_PROP_FPS)
164         if fps <= 0:
165             fps = 30.0
166
167         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
168         if frame_width <= 0:
169             frame_width = 1280.0 # Sensible fallback
170
171         # Effective sampling rate after frame-skipping
172         effective_fps = fps / max(frame_skip, 1)
173
174         # A fresh tracker per chunk ? track IDs only need to be consistent within a
175         # chunk (windows are computed per chunk, then merged across chunks by time).
176         tracker = self.make_tracker(effective_fps)
177         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
178         0.5)
179
180         # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
181         # so windows can be enriched with motion stats for logging + fine-tuning.
182         active_frames = []
183         frame_idx = 0
184         track_history: Dict[int, Tuple[float, float, float, float]] = {}
185
186         while True:
187             ret, frame = cap.read()
188             if not ret:
189                 break
190
191             # Skip frames for performance ? at 30fps, every 3rd frame still
192             # gives ~10 samples/second, plenty for human-scale velocity.
193             if frame_idx % max(frame_skip, 1) != 0:

```

```

190         frame_idx += 1
191         continue
192
193     # Detect persons + assign persistent track IDs.
194     detections = self.detect_and_track(frame, tracker, score_thresh)
195
196     frame_peak_velocity = 0.0
197     frame_active_ids = set()
198     current_ids = set()
199
200     for track_id, bbox in detections:
201         current_ids.add(track_id)
202
203         if track_id in track_history:
204             prev_bbox = track_history[track_id]
205             velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
206
207             if velocity > velocity_thresh:
208                 frame_peak_velocity = max(frame_peak_velocity, velocity)
209                 frame_active_ids.add(track_id)
210
211             track_history[track_id] = bbox
212
213     # Prune stale IDs that are no longer visible ? prevents false
214     # velocity spikes when the tracker reassigns an old ID.
215     track_history = {k: v for k, v in track_history.items() if k in current_ids}
216
217     if frame_active_ids:
218         active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
219
220     frame_idx += 1
221
222     cap.release()
223
224     # Group active frames into continuous windows
225     if not active_frames:
226         return []
227
228     max_gap_frames = int(merge_gap * fps)
229
230     def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
231         velocities = [v for _, v, _ in group]
232         track_ids = set().union(*[ids for _, _, ids in group])
233         return {
234             "start": group[0][0] / fps + chunk_start,
235             "end": group[-1][0] / fps + chunk_start,
236             "active_frame_count": len(group),
237             "peak_velocity": max(velocities),
238             "mean_velocity": sum(velocities) / len(velocities),
239             "track_id_count": len(track_ids),
240             "chunk_index": chunk_index,
241         }
242
243     windows = []
244     group = [active_frames[0]]
245     for af in active_frames[1:]:
246         if af[0] - group[-1][0] <= max_gap_frames:
247             group.append(af)
248         else:
249             windows.append(_finalize(group))
250             group = [af]
251     windows.append(_finalize(group))
252
253     # Filter out extremely short action windows
254     min_window_duration = self.config.get("indexing",

```



```

309         """Per-window person detections via ONE sequential pass over the video.
310
311         Opens the video ONCE and reads forward ? ``grab()`` to advance, ``retrieve()`` +
312         detect only on sampled frames inside a window ? instead of
313         ``detections_per_frame``'s
314         per-window open + ``POS_FRAMES`` seek, which on a long video degrades to
315         decode-from-start per window (0(window_index) ? the fusion-P3 extraction
316         bottleneck;
317         172 windows took ~80 min). ``windows`` = ``[(start_frame, end_frame), ...]``;
318         returns a
319         list aligned to it, each ``{"frames": {idx: [(tid, bbox)]}, "fps", "frame_width",
320         "frame_height"}``` (same shape as ``detections_per_frame``). Each window gets its
321         own
322         window-local ByteTrack. ``progress_cb(frames_done, total_frames)`` (called every
323         ~256
324         frames) drives progress logging.
325         """
326         self.lazy_load_model()
327         cap = cv2.VideoCapture(video_path)
328         n = len(windows)
329         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
330         if fps <= 0:
331             fps = 30.0
332         fw = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
333         fh = cap.get(cv2.CAP_PROP_FRAME_HEIGHT) if cap.isOpened() else 0.0
334         base = {"fps": fps, "frame_width": float(fw), "frame_height": float(fh)}
335         if not cap.isOpened() or n == 0:
336             cap.release()
337             return [dict(frames={}, **base) for _ in range(n)]
338
339         step = max(frame_skip, 1)
340         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
341         0.5)
342         frames_per_win: List[Dict[int, Any]] = [{ } for _ in range(n)]
343         trackers = [self.make_tracker(fps / step) for _ in range(n)]
344         max_end = max(ef for _, ef in windows)
345
346         frame_idx = 0
347         while frame_idx <= max_end:
348             if not cap.grab():
349                 break
350             # Windows that want THIS frame (inside the range AND on that window's
351             sampling grid).
352             wis = [wi for wi, (sf, ef) in enumerate(windows)
353                     if sf <= frame_idx <= ef and (frame_idx - sf) % step == 0]
354             if wis:
355                 ok, frame = cap.retrieve()
356                 if ok:
357                     for wi in wis:
358                         frames_per_win[wi][frame_idx] = self.detect_and_track(
359                             frame, trackers[wi], score_thresh)
360             frame_idx += 1
361             if progress_cb is not None and frame_idx % 256 == 0:
362                 progress_cb(frame_idx, max_end)
363         cap.release()
364         return [dict(frames=frames_per_win[wi], **base) for wi in range(n)]
365
366

```

25. user (2026-06-18T01:28:37.682Z)

```

1     """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3     This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4     of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5     the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.

```

```

6
7 The ABC lets us:
8 - Keep the current WSL adapters working unchanged for Windows users.
9 - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10 the hybrid segmenters or the model-agnostic windowing brain.
11 - Provide a single place to evolve the healthcheck / error contract and thread
12 detector readiness into per-video reports.
13
14 Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15 - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16   Run detector, return the Windows-side path to the produced trajectory CSV
17   (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18   The caller (hybrid) is responsible for output dir; impls may stage the video
19   into WSL fs for perf.
20
21 - healthcheck() -> tuple[bool, str]
22   Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23   Message is logged (and can be surfaced to reports or UX in future).
24   Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26 Existing behavior is 100% preserved. The only additions are the inheritance and
27 the explicit common type for future implementers.
28
29 How to add a new runner (for docs / future contributors):
30 1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31 2. Implement run_predict and healthcheck (return (bool, str) tuple).
32 3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33   TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34   deliberately static and model-agnostic). If the new detector needs different
35   windowing, it can implement its own or we generalize later.
36 4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37   registry never pulls torch/tf on a machine that doesn't have them).
38 5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
39   DetectorRunner instances (they already do the right thing via duck-typing + the
40   type hints we add here).
41 6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
42   returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
43 7. No change to eval/calibration code paths (they consume the CSV + the static
44   windowing helpers directly).
45
46 Optional Linux-native path (the main de-risk goal):
47 A LinuxNativeRunner (or ONNXRunner) would:
48 - Skip all wsl / _stage / conda activate.
49 - Load weights locally (torch or onnx) or call a native binary.
50 - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
51 - healthcheck can just check "weights file exists and torch importable + CUDA?".
52 This removes the WSL requirement for Linux users / future SaaS workers while
53 keeping the exact same candidate quality characteristics (or better, once native
54 weights are available).
55
56 Error surfacing into reports:
57 The healthcheck message is already logged by the hybrids on failure. The reporting
58 harvest sees the outcome as "0 candidates" + detector name. When we wire richer
59 segmenter Results (or pass detector_health into emit_indexer_report context), the
60 (ok, msg) can be recorded explicitly under segmentation.details or a new detector
61 stage. The ABC makes that future change local to one place.
62
63 See also:
64 - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
65 - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
66 - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
67 ""
68
69 import logging
70 import shlex

```

```

71 import subprocess
72 from abc import ABC, abstractmethod
73 from typing import Any, Callable, List, Optional, Tuple
74
75 logger = logging.getLogger(__name__)
76
77
78 def wsl_tilde_quote(path: str) -> str:
79     """shlex.quote that still lets bash expand a leading `~/`.
80
81     `shlex.quote("~/clips/x.mp4")` single-quotes the whole path, so bash
82     treats `~` literally and `cp ~/mkdir` fail with "No such file or
83     directory" ? this broke live WSL staging for the default `~/clips`
84     stage dir (found by the 2026-06-11 wasb_hybrid smoke run; the BACKLOG
85     "shlex-quote breaks ~ expansion" item). Quoting only the part after
86     `~/` keeps expansion AND protects spaces/specials in the remainder.
87     """
88     if path == "~":
89         return path
90     if path.startswith("~/"):
91         return "~/ " + shlex.quote(path[2:])
92     return shlex.quote(path)
93
94
95 class WslCommandMixin:
96     """Shared `wsl -d <distro> bash -c 'source <conda_sh> && <cmd>'` invocation.
97
98     Lives beside (not on) DetectorRunner: a future Linux-native runner must not
99     inherit WSL plumbing. Requires `self.cfg` with `distro`, `conda_sh`
100     and `timeout_sec` attributes (TrackNetConfig and WasbConfig both qualify).
101     `timeout_sec == 0` maps to None = wait forever (configs default 1800s).
102     """
103
104     cfg: Any
105
106     def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
107         full = f"source {self.cfg.conda_sh} && {inner_cmd}"
108         argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
109         logger.debug("WSL exec: %s", full)
110         return subprocess.run(argv, capture_output=True, text=True,
111                               timeout=(self.cfg.timeout_sec or None))
112
113     # ---- cache hygiene helpers (shared by both WSL runners) -----
114     def _wsl_free_gb(self, wsl_path: str) -> Optional[float]:
115         """Best-effort free space (GB) on the WSL filesystem holding `wsl_path`.
116
117         Returns None if it can't be determined (never raises). Used as a pre-run
118         disk guard so a long extraction doesn't fill the disk mid-flight.
119         """
120         try:
121             res = self._wsl_bash(f"df -P -B1 {wsl_tilde_quote(wsl_path)} | awk
122 'NR==2{{print $4}}'")
123         except (subprocess.TimeoutExpired, OSError):
124             return None
125         try:
126             return int((res.stdout or "").strip()) / (1024 ** 3)
127         except (ValueError, AttributeError):
128             return None
129
130     def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
131         """Best-effort recursive delete of WSL paths (post-success cleanup; never
132         raises)."""
133         paths = [p for p in wsl_paths if p]
134         if not paths:
135             return

```

```

134         quoted = " ".join(wsl_tilde_quote(p) for p in paths)
135         try:
136             self._wsl_bash(f"rm -rf {quoted}")
137         except (subprocess.TimeoutExpired, OSError) as e:
138             logger.warning("WSL cache cleanup failed (non-fatal): %s", e)
139
140     def wsl_source_unreadable_reason(self, src_mnt: str) -> Optional[str]:
141         """Actionable message if ``src_mnt`` is NOT readable from inside WSL, else None.
142
143         Google Drive (``G:``/Drive File Stream), UNC and many network/virtual
144         filesystems are invisible to WSL's ``/mnt`` ? staging a video from such a path
145         fails with a cryptic ``cp: cannot stat`` and the whole run silently produces no
146         trajectory (i.e. "0 rallies", a wasted run that looks like a quality result).
147         A cheap ``test -r`` preflight turns that into one clear, fixable error BEFORE
148         any
149         frame extraction. Best-effort: a flaky/timed-out probe returns None so the real
150         ``cp`` still gets its chance (we never block a genuinely-readable source).
151         """
152         try:
153             res = self._wsl_bash(f"test -r {shlex.quote(src_mnt)} && echo READABLE")
154         except (subprocess.TimeoutExpired, OSError):
155             return None
156         if "READABLE" in (res.stdout or ""):
157             return None
158         return (
159             f"video source is not readable from WSL at '{src_mnt}'. Google Drive, UNC, "
160             f"and other network/virtual filesystems are invisible to WSL's /mnt ? copy "
161             f"the video to a local disk (e.g. C:\\ or the WSL filesystem) and retry."
162         )
163
164     class DetectorRunner(ABC):
165         """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""
166
167         @abstractmethod
168         def run_predict(self, video_win_path: str, output_win_dir: str,
169                        video_id: Optional[str] = None) -> Optional[str]:
170             """Run inference and return path to trajectory CSV (or None on failure).
171
172             ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two
173             videos that happen to share a filename cannot reuse each other's frames or CSV.
174             It is optional for back-compat; callers in the live pipeline always pass it.
175             """
176             pass
177
178         @abstractmethod
179         def healthcheck(self) -> Tuple[bool, str]:
180             """Quick check if the runner (env, weights, etc.) is usable.
181
182             Returns (ok, message). Never raises. Cheap to call before a long run.
183             """
184             pass
185
186         # De facto trajectory contract: the trajectory hybrids call these
187         # unconditionally after run_predict. Concrete runners reuse/re-export the
188         # model-agnostic TrackNetRunner staticmethods (same CSV schema + windowing);
189         # a runner with custom windowing supplies its own with the same shapes.
190         parse_trajectory_csv: Callable[..., Any]
191         trajectory_to_action_windows: Callable[..., Any]
192

```

26. assistant (2026-06-18T01:28:41.558Z)

Now let me look at the audio hit_detector, motion segmenter, gemini segmenter, and yolo_hybrid. Let me check the `detections\_over\_windows` more closely first ? there's a potential issue with

`max\_end` when windows might have negative or the progress callback. But let me focus on reading the remaining files.

27. user (2026-06-18T01:28:42.520Z)

```
1      """Audio shuttle-hit ("thwack") detection ? local, dependency-light (numpy + ffmpeg).
2
3      Stages 0?1 of docs/AUDIO_RALLY_DETECTION_PLAN.md: extract the audio track, build a
4      spectral-flux onset envelope (with pre-emphasis to boost the broadband HF transient
5      a racket?shuttle contact makes), threshold it adaptively, and return (timestamp,
6      strength) hit candidates. Pure numpy + stdlib `wave` (NO scipy); ffmpeg (subprocess)
7      only for extraction. Consumed by the E1 go/no-go probe; later feeds rally fusion
8      (ADR-007) if E1 passes. WASB is never modified ? this is a separate signal.
9      """
10     from __future__ import annotations
11
12     import os
13     import wave
14     import tempfile
15     import subprocess
16     from dataclasses import dataclass
17     from typing import List, Optional, Tuple
18
19     import numpy as np
20
21
22     @dataclass
23     class HitEvent:
24         t: float          # seconds
25         strength: float    # onset-envelope peak value (arbitrary units)
26
27
28     def extract_audio(video_path: str, out_wav: Optional[str] = None, sr: int = 16000) ->
str:
29         """ffmpeg ? mono 16-bit PCM WAV at `sr`. Returns the wav path (temp if not given).
30         Audio-only decode, so it's fast even for multi-GB video."""
31         if out_wav is None:
32             fd, out_wav = tempfile.mkstemp(suffix=".wav")
33             os.close(fd)
34             cmd = ["ffmpeg", "-y", "-i", video_path, "-vn", "-ac", "1", "-ar", str(sr),
35                   "-c:a", "pcm_s16le", out_wav]
36             subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
37             return out_wav
38
39
40     def load_wav(path: str) -> Tuple[np.ndarray, int]:
41         """Read a PCM WAV ? (float32 samples in [-1,1], sample_rate). Mono-downmixes."""
42         with wave.open(path, "rb") as w:
43             sr, n, sw, ch = w.getframerate(), w.getnframes(), w.getsampwidth(),
w.getnchannels()
44             raw = w.readframes(n)
45             if sw == 2:
46                 a = np.frombuffer(raw, dtype=np.int16).astype(np.float32) / 32768.0
47             elif sw == 4:
48                 a = np.frombuffer(raw, dtype=np.int32).astype(np.float32) / 2147483648.0
49             elif sw == 1:
50                 a = np.frombuffer(raw, dtype=np.uint8).astype(np.float32) / 128.0 - 1.0
51             else:
52                 raise ValueError(f"unsupported sample width {sw}")
53             if ch > 1:
54                 a = a.reshape(-1, ch).mean(axis=1)
55             return a, sr
56
57
```

```

58     def _preemphasis(x: np.ndarray, coef: float = 0.97) -> np.ndarray:
59         if len(x) < 2:
60             return x.copy()
61         return np.append(x[0], x[1:] - coef * x[:-1])
62
63
64     def onset_envelope(samples: np.ndarray, sr: int, frame: int = 1024,
65                       hop: int = 256, band: Optional[Tuple[float, float]] = None) ->
66     Tuple[np.ndarray, float]:
67         """Spectral-flux onset strength per hop (pure-numpy STFT). Returns (env,
68         hop_seconds).
69
70         `band=(lo_hz, hi_hz)` restricts the flux to a frequency band ? a shuttle "thwack"
71         is HF-rich while voices/footsteps live lower, so band-limiting (e.g. 3-9 kHz)
72         suppresses non-shuttle transients. Needs sr high enough for hi_hz (Nyquist =
73         sr/2)."""
74         x = _preemphasis(np.asarray(samples, dtype=np.float32))
75         hop_sec = hop / float(sr)
76         if len(x) < frame:
77             return np.zeros(0, dtype=np.float32), hop_sec
78         win = np.hanning(frame).astype(np.float32)
79         n_frames = 1 + (len(x) - frame) // hop
80         idx = np.arange(frame)[None, :] + hop * np.arange(n_frames)[:, None]
81         frames = x[idx] * win
82         mag = np.abs(np.fft.rfft(frames, axis=1))
83         if band is not None:
84             freqs = np.fft.rfftfreq(frame, d=1.0 / sr)
85             keep = (freqs >= band[0]) & (freqs <= band[1])
86             if keep.any():
87                 mag = mag[:, keep] # band-limited flux
88         diff = np.diff(mag, axis=0) # frame-to-frame change
89         flux = np.sqrt((np.maximum(diff, 0.0) ** 2).sum(axis=1)) # positive (onset) flux
90         env = np.concatenate([[0.0], flux]).astype(np.float32) # align to n_frames
91         return env, hop_sec
92
93     def _moving_stats(env: np.ndarray, win: int) -> Tuple[np.ndarray, np.ndarray]:
94         """Sliding-window mean and std (reflect-padded, via cumsum). win forced odd."""
95         if win % 2 == 0:
96             win += 1
97         pad = win // 2
98         p = np.pad(env.astype(np.float64), pad, mode="reflect")
99         c1 = np.cumsum(np.insert(p, 0, 0.0))
100        c2 = np.cumsum(np.insert(p * p, 0, 0.0))
101        s1 = c1[win:] - c1[:-win]
102        s2 = c2[win:] - c2[:-win]
103        mean = s1 / win
104        var = np.maximum(s2 / win - mean * mean, 0.0)
105        return mean[:len(env)], np.sqrt(var)[:len(env)]
106
107     def detect_hits(samples: np.ndarray, sr: int, k: float = 2.5, win_frames: int = 41,
108                   min_gap_sec: float = 0.08, band: Optional[Tuple[float, float]] = None)
109     -> List[HitEvent]:
110         """Detect impact transients: local maxima of the onset envelope that exceed an
111         adaptive threshold (moving mean + k*std), keeping the strongest within
112         `min_gap_sec`.
113         `k` is the sensitivity knob (lower = more hits); `band` band-limits the onset flux
114         to the shuttle-click frequencies (e.g. (3000, 9000))."""
115         env, hop_sec = onset_envelope(samples, sr, band=band)
116         if len(env) < 3:
117             return []
118         mean, std = _moving_stats(env, win_frames)
119         thr = mean + k * std
120         is_peak = (env[1:-1] >= env[:-2]) & (env[1:-1] > env[2:]) & (env[1:-1] > thr[1:-1])

```

```

118     peak_idx = np.nonzero(is_peak)[0] + 1
119     if peak_idx.size == 0:
120         return []
121     # enforce min inter-peak gap: greedily keep strongest first
122     min_gap = max(1, int(round(min_gap_sec / hop_sec)))
123     order = peak_idx[np.argsort(-env[peak_idx])]
124     taken = np.zeros(len(env), dtype=bool)
125     kept = []
126     for i in order:
127         lo, hi = max(0, i - min_gap), min(len(env), i + min_gap + 1)
128         if taken[lo:hi].any():
129             continue
130         taken[i] = True
131         kept.append(i)
132     kept.sort()
133     return [HitEvent(t=float(i * hop_sec), strength=float(env[i])) for i in kept]
134
135
136 def detect_hits_in_video(video_path: str, sr: int = 16000, **kw) -> List[HitEvent]:
137     """Convenience: extract audio from a video and detect hits. Cleans up the temp
138     wav."""
139     wav = extract_audio(video_path, sr=sr)
140     try:
141         samples, real_sr = load_wav(wav)
142         return detect_hits(samples, real_sr, **kw)
143     finally:
144         try:
145             os.remove(wav)
146         except OSError:
147             pass

```

28. user (2026-06-18T01:28:43.406Z)

```

1     import cv2
2     import time
3     import random
4     from typing import List, Dict, Any, Tuple
5     import logging
6     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
7     from backend.results import Success, Failure # standardized contract
8
9     logger = logging.getLogger(__name__)
10
11     class MotionPlaySegmenter(BasePlaySegmenter):
12         @property
13         def name(self) -> str:
14             return "motion"
15
16         @property
17         def description(self) -> str:
18             return "Traditional CV pixel-motion heuristics tracking frame differences."
19
20         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
21             """
22             Processes a video to detect play segments using motion heuristics,
23             populates SQLite with detected segments, and indexes detailed events.
24             """
25             if not player_pool:
26                 player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
27
28             cap = cv2.VideoCapture(video_path)
29             if not cap.isOpened():
30                 logger.error(f"Segmenter could not open video: {video_path}")

```



```

31         return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
32
33         fps = cap.get(cv2.CAP_PROP_FPS)
34         total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
35         duration = total_frames / fps if fps > 0 else 0
36
37         # Sub-sample settings (5 FPS for analysis)
38         analysis_fps = 5.0
39         frame_interval = max(1, int(fps / analysis_fps))
40
41         idx_cfg = self.config.get("indexing", {})
42         motion_threshold = idx_cfg.get("motion_threshold", 0.08)
43         min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
44         dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
45
46         prev_gray = None
47         motion_signals = []
48         timestamps = []
49
50         frame_idx = 0
51         processed_count = 0
52         t_start = time.time()
53         # Emit a progress line roughly every 5% of the video so long runs are
54         # observable instead of silent (a full 1080p match is ~tens of minutes).
55         progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
56         next_progress = progress_step
57         logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
58                   f"({duration/60:.1f} min), analyzing every {frame_interval}th
frame")
59         while True:
60             # Skip decoding for intermediate frames using cap.grab() for high
performance
61             for _ in range(frame_interval - 1):
62                 if not cap.grab():
63                     break
64                 frame_idx += 1
65
66             ret, frame = cap.read()
67             if not ret:
68                 break
69
70             # Process frame
71             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
72             # Apply Gaussian Blur to smooth noise
73             gray = cv2.GaussianBlur(gray, (21, 21), 0)
74
75             t = frame_idx / fps
76             timestamps.append(t)
77
78             if prev_gray is not None:
79                 # Frame absolute difference (highly optimized motion tracking)
80                 diff = cv2.absdiff(prev_gray, gray)
81                 _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
82
83                 # Dilate to fill gaps
84                 thresh = cv2.dilate(thresh, None, iterations=2)
85
86                 # Motion ratio
87                 motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
88                 motion_signals.append(motion_ratio)
89             else:
90                 motion_signals.append(0.0)
91

```

```

92         prev_gray = gray
93         frame_idx += 1
94         processed_count += 1
95
96         if progress_step and frame_idx >= next_progress:
97             pct = 100.0 * frame_idx / total_frames
98             elapsed = time.time() - t_start
99             eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
100            logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames,
101            "
102                        f"{processed_count} analyzed, {elapsed:.0f}s elapsed,
103            ~{eta:.0f}s left")
104            next_progress += progress_step
105
106            if max_frames is not None and processed_count >= max_frames:
107                logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer
108            frame parsing.")
109                break
110
111            if frame_idx >= total_frames:
112                break
113
114        cap.release()
115
116        # Smooth signal using a moving average
117        window_size = 5
118        smoothed = []
119        for i in range(len(motion_signals)):
120            start = max(0, i - window_size // 2)
121            end = min(len(motion_signals), i + window_size // 2 + 1)
122            smoothed.append(sum(motion_signals[start:end]) / (end - start))
123
124        # Detect active play boundaries
125        in_segment = False
126        segment_start = None
127        raw_segments: List[Tuple[float, float]] = []
128
129        for i, t in enumerate(timestamps):
130            signal = smoothed[i]
131            if signal > motion_threshold:
132                if not in_segment:
133                    in_segment = True
134                    segment_start = t
135            else:
136                if in_segment:
137                    in_segment = False
138                    raw_segments.append((segment_start, t))
139
140        # Handle video ending while in active segment
141        if in_segment:
142            raw_segments.append((segment_start, timestamps[-1]))
143
144        # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
145        merged_segments: List[Tuple[float, float]] = []
146        if raw_segments:
147            curr_start, curr_end = raw_segments[0]
148            for next_start, next_end in raw_segments[1:]:
149                if next_start - curr_end < dead_time_merge_gap:
150                    # Merge
151                    curr_end = next_end
152            else:
153                merged_segments.append((curr_start, curr_end))
154                curr_start, curr_end = next_start, next_end
155            merged_segments.append((curr_start, curr_end))

```

```

154     # Post-Processing: 2. Filter segments shorter than min_segment_duration
155     final_segments: List[Tuple[float, float]] = [
156         r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
157     ]
158
159     # Populate SQLite DB with segments and metadata events
160     segments_data = []
161     for start, end in final_segments:
162         # Assign Server and Receiver randomly from pool
163         server, receiver = random.sample(player_pool, 2)
164
165         # Formulate dynamic rich tags
166         dur = end - start
167         metadata = {
168             "shot_count": random.randint(4, 25),
169             "intensity": "high" if dur > 12 else "medium",
170             "avg_speed_kmh": round(random.uniform(40, 85), 1)
171         }
172
173         segment_id = self.db.add_play_segment(video_id, start, end, server,
174 receiver, metadata)
175
176         if segment_id:
177             # Add sub-segment events
178             # A: Serve Event (always at start)
179             serve_foul = random.random() < 0.15 # 15% chance of serve foul
180             self.db.add_event(
181                 segment_id=segment_id,
182                 timestamp=start + 0.5,
183                 event_type="serve",
184                 player=server,
185                 details={"status": "foul" if serve_foul else "valid", "type":
186 "low_serve" if random.random() > 0.5 else "high_serve"}
187             )
188
189             # B: Smash Events (1 to 3 random smashes per segment)
190             smashes_count = random.randint(0, 3)
191             for _ in range(smashes_count):
192                 smash_time = random.uniform(start + 1.5, end - 1.0)
193                 smasher = random.choice([server, receiver])
194                 self.db.add_event(
195                     segment_id=segment_id,
196                     timestamp=round(smash_time, 2),
197                     event_type="smash",
198                     player=smasher,
199                     details={"speed_kmh": round(random.uniform(120, 250), 1)}
200                 )
201
202             # C: Segment Termination Event (foul/winner)
203             foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
204             is_foul = random.random() > 0.4
205             error_player = random.choice([server, receiver])
206
207             self.db.add_event(
208                 segment_id=segment_id,
209                 timestamp=end - 0.2,
210                 event_type="foul" if is_foul else "winner",
211                 player=error_player,
212                 details={"type": random.choice(foul_types)} if is_foul else
213 {"shot_type": "drop" if random.random() > 0.5 else "smash"}
214             )
215
216             # Store segment representation to return to caller
217             segments_data.append({
218                 "id": segment_id,

```

```

216             "start_time": start,
217             "end_time": end,
218             "duration": dur,
219             "server": server,
220             "receiver": receiver,
221             "metadata": metadata
222         })
223
224     return Success(value=segments_data)
225
226     # Register the segmenter
227     segmenter_registry.register(MotionPlaySegmenter)
228

```

29. assistant (2026-06-18T01:28:47.548Z)

I found a contract mismatch in `motion.py`. The `process\_video` method has a type signature `-> List[Dict[str, Any]]` but returns `Failure(...)` and `Success(value=...)` objects. Let me check how the return value of this method is consumed? whether returning `Success`/`Failure` here vs. a bare list (as other segmenters return) causes a real problem. The base contract docstring says it should return a `Result`. But `trajectory\_hybrid` returns a bare list. Let me check `main.py` and how callers handle the return.

30. user (2026-06-18T01:28:48.355Z)

<persisted-output>

Output too large (26.9KB). Full output saved to: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\tool-results\toolu_019nkTd1RMAR9oEasmfbF6Ve.txt

Preview (first 2KB):

```

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-188-           # if the run
yields segments; otherwise drop the new partial rows and keep the old.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-189-           play_wm, cand_wm
= db_instance.segment_watermarks(video_id)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-190-           segmenter =
segmenter_cls(db_instance, global_config)
backend\main.py-191-           try:
backend\main.py:192:               result = segmenter.process_video(video_id, req.video_path,
req.player_pool, req.max_frames)
backend\main.py-193-           except Exception:
backend\main.py-194-               # A raising segmenter must not leave half-written rows: restore
the pre-run
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-195-               # state
(same destroy-after-confirm contract as the Failure branch), then let
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-196-               # the job
worker record the failure.
--
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-343-_MAX_DRAIN_WAKE_SEC =
60.0
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-344-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-345-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\main.py-346-@app.post("/api/process", status_code=202)
backend\main.py:347:def process_video(req: ProcessRequest):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-348-    """Submit a
processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-349-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py-350-    Fast-fail checks
(bad path, missing file, unknown segmenter) happen here so the
...
</persisted-output>

```

31. assistant (2026-06-18T01:28:52.986Z)

`motion.py` returning `Success`/`Failure` is its established intended behaviour (it imports `Success, Failure`), and main.py handles both. This is a known, deliberate inconsistency. Not a bug. Let me look at the `detections\_over\_windows` `max\_end` issue and the gemini/yolo segmenters and remaining validators.

32. user (2026-06-18T01:28:54.411Z)

```
1      """Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
2      (Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).
3
4      Stage-1 detection now lives in `backend/pipeline/detectors/person_detector.py`
5      (`PersonDetector`, a reusable rally-cue producer ? fusion P1) so the SAME person cue
6      can be consumed by the future fusion segmenter; this segmenter just orchestrates
7      chunking, the AI-provider handoff, and DB population around it. Behaviour is unchanged.
8
9      Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
10     service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
11     detection uses torchvision's permissively-licensed (BSD) detectors and the
12     association/tracking that YOLO.track() used to bundle in is provided by
13     supervision's MIT-licensed ByteTrack.
14
15     The registry key stays "yolo_hybrid" for back-compat with existing configs and the
16     DB `detector` tags, even though no YOLO is involved anymore.
17     """
18     import os
19     import time
20     import logging
21     import concurrent.futures
22     from typing import List, Dict, Any, Tuple
23
24     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
25     from backend.pipeline.detectors.person_detector import PersonDetector, _DEFAULT_DETECTOR
26     from backend.utils.video import get_video_duration, extract_video_chunk
27     from backend.sports import sport_registry
28     from backend.providers import provider_registry
29
30     logger = logging.getLogger(__name__)
31
32
33     def _fmt_ts(seconds: float) -> str:
34         """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
35         seconds = max(0, int(seconds))
36         h, rem = divmod(seconds, 3600)
37         m, s = divmod(rem, 60)
38         return f"{h:02d}:{m:02d}:{s:02d}"
39
40
41     class HybridYoloSegmenter(BasePlaySegmenter):
42         def __init__(self, db, config):
43             super().__init__(db, config)
44             # Stage-1 person cue (detection + tracking + motion windows) ? a reusable
45             # (fusion P1). This segmenter orchestrates chunking + AI handoff + DB around it.
46             self.person = PersonDetector(config)
47
48             @property
49             def name(self) -> str:
50                 return "yolo_hybrid"
51
52             @property
53             def description(self) -> str:
54                 return ("Two-stage pipeline: torchvision person detection + ByteTrack for fast "
55                         "candidate filtering, then an AI provider for high-precision semantic
56                         boundaries.")
```

```

57         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
58             self.person.lazy_load_model()
59
60             # Get sport profile config
61             sport_name = self.config.get("sport", "badminton")
62             try:
63                 sport_profile = sport_registry.get(sport_name)
64             except KeyError as e:
65                 logger.error(str(e))
66                 return []
67
68             # Get AI provider and model config
69             idx_cfg = self.config.get("indexing", {})
70             provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
71             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
72
73             # Get appropriate API key based on the provider
74             api_key_env_var = f"{provider_name.upper()}_API_KEY"
75             api_key = os.environ.get(api_key_env_var)
76             if not api_key:
77                 logger.error(
78                     f"{api_key_env_var} environment variable is not set! "
79                     f"To run the {provider_name} segmenter, set your API key. "
80                     f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
81                 )
82                 return []
83
84             try:
85                 provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
86             except KeyError as e:
87                 logger.error(str(e))
88                 return []
89
90             logger.info(f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}")
91
92             video_duration = get_video_duration(video_path)
93             if video_duration <= 0:
94                 logger.error("Invalid video duration.")
95                 return []
96
97             chunk_duration = idx_cfg.get("chunk_duration", 300)
98             chunk_overlap = idx_cfg.get("chunk_overlap", 30)
99             velocity_thresh = idx_cfg.get("yolo_velocity_threshold", 0.15) # Fraction of
frame width per second
100             merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
101             frame_skip = idx_cfg.get("yolo_frame_skip", 3)
102             tracker_config = idx_cfg.get("yolo_tracker", "bytetrack.yaml")
103             handoff_padding = idx_cfg.get("ai_handoff_padding",
idx_cfg.get("gemini_handoff_padding", 2.0))
104             ai_max_workers = idx_cfg.get("ai_max_workers", idx_cfg.get("gemini_max_workers",
5))
105             log_candidates = idx_cfg.get("log_yolo_candidates", True)
106             store_candidates = idx_cfg.get("store_yolo_candidates", True)
107
108             # Snapshot of the params that produced these detections ? stored alongside each
109             # candidate so a fine-tuning run can reproduce / correlate against them.
110             detection_params = {
111                 "velocity_thresh": velocity_thresh,
112                 "merge_gap": merge_gap,
113                 "frame_skip": frame_skip,

```

```

114         "tracker": tracker_config,
115         "detector_model": idx_cfg.get("detector_model", _DEFAULT_DETECTOR),
116         "detector_score_threshold": idx_cfg.get("detector_score_threshold", 0.5),
117         "min_action_window_duration": idx_cfg.get("min_action_window_duration",
118 2.0),
119     }
120     chunks_dir = os.path.join("output", "chunks_yolo")
121     os.makedirs(chunks_dir, exist_ok=True)
122
123     step = chunk_duration - chunk_overlap
124     chunk_starts = []
125     t = 0.0
126     while t < video_duration:
127         chunk_starts.append(t)
128         t += step
129
130     num_chunks = len(chunk_starts)
131     logger.info(f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
chunks.")
132
133     all_candidate_windows = []
134
135     t_start = time.time()
136     prefetch_workers = idx_cfg.get("yolo_prefetch_workers", 2)
137
138     if num_chunks > 1 and prefetch_workers > 0:
139         # --- PIPELINED PROCESSING ---
140         # GPU inference must stay sequential (single GPU), but ffmpeg chunk
141         # extraction is pure I/O. We pre-extract upcoming chunks in background
142         # threads so extraction of chunk N+1 overlaps with inference on chunk N.
143         logger.info(f"Running detection Phase 2 with pipelined extraction
({prefetch_workers} prefetch workers)...")
144
145         extract_executor =
concurrent.futures.ThreadPoolExecutor(max_workers=prefetch_workers)
146
147         def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
148             chunk_end = min(cs + chunk_duration, video_duration)
149             actual_dur = chunk_end - cs
150             chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
151             success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
152             return (ci, cs, chunk_path, success)
153
154         # Submit all extractions upfront ? they'll queue and run as workers free up
155         extract_futures = [
156             extract_executor.submit(_extract_chunk, ci, cs)
157             for ci, cs in enumerate(chunk_starts)
158         ]
159
160         # Process results in order: wait for extraction, then run inference on GPU
161         for future in extract_futures:
162             ci, chunk_start, chunk_path, success = future.result()
163             chunk_end = min(chunk_start + chunk_duration, video_duration)
164             logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
165
166             if not success:
167                 logger.warning(f"Chunk {ci+1} extraction failed. Skipping.")
168                 continue
169
170             windows = self.person.extract_action_windows_from_chunk(
171                 chunk_path, chunk_start, velocity_thresh, merge_gap,
172                 frame_skip=frame_skip, chunk_index=ci
173             )

```

```

174         logger.info(f"Found {len(window)} candidate action windows in chunk
{ci+1}.")
175         all_candidate_windows.extend(window)
176
177         try:
178             os.remove(chunk_path)
179         except Exception:
180             pass
181
182         extract_executor.shutdown(wait=False)
183     else:
184         # --- SEQUENTIAL PROCESSING (single chunk or prefetch disabled) ---
185         for ci, chunk_start in enumerate(chunk_starts):
186             chunk_end = min(chunk_start + chunk_duration, video_duration)
187             actual_dur = chunk_end - chunk_start
188             chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
189
190             logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
191             success = extract_video_chunk(video_path, chunk_start, actual_dur,
chunk_path)
192             if not success:
193                 continue
194
195             window = self.person.extract_action_windows_from_chunk(
196                 chunk_path, chunk_start, velocity_thresh, merge_gap,
197                 frame_skip=frame_skip, chunk_index=ci
198             )
199             logger.info(f"Found {len(window)} candidate action windows in chunk
{ci+1}.")
200             all_candidate_windows.extend(window)
201
202             try:
203                 os.remove(chunk_path)
204             except Exception:
205                 pass
206
207         # Deduplicate overlapping candidate windows across chunks (chunks overlap by
208         # chunk_overlap, so the same rally can surface in two adjacent chunks).
209         def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
210             fa, fb = a["active_frame_count"], b["active_frame_count"]
211             total = fa + fb or 1
212             return {
213                 "start": a["start"],
214                 "end": max(a["end"], b["end"]),
215                 "active_frame_count": fa + fb,
216                 "peak_velocity": max(a["peak_velocity"], b["peak_velocity"]),
217                 # Frame-weighted mean so longer sub-windows dominate proportionally.
218                 "mean_velocity": (a["mean_velocity"] * fa + b["mean_velocity"] * fb) /
total,
219                 "track_id_count": max(a["track_id_count"], b["track_id_count"]),
220                 "chunk_index": a["chunk_index"],
221             }
222
223         if all_candidate_windows:
224             all_candidate_windows.sort(key=lambda w: w["start"])
225             merged_windows = [all_candidate_windows[0]]
226             for current in all_candidate_windows[1:]:
227                 last = merged_windows[-1]
228                 if current["start"] <= last["end"]: # Overlap
229                     merged_windows[-1] = _merge_stats(last, current)
230                 else:
231                     merged_windows.append(current)
232             all_candidate_windows = merged_windows
233

```



```

234         logger.info(f"Phase 2 tracking completed in {time.time() - t_start:.1f}s.")
235         logger.info(f"Total candidate windows: {len(all_candidate_windows)}")
236
237     # Per-rally console log (final, full-video timestamps) for video cross-
verification.
238     if log_candidates:
239         for w in all_candidate_windows:
240             logger.info(
241                 f"[rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
242                 f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']} "
243                 f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f} "
244                 f"tracks={w['track_id_count']}"
245             )
246
247     # Persist raw candidates for the fine-tuning dataset. Map window index ->
candidate_id
248     # so confirmed AI segments can be linked back in Phase 4.
249     candidate_ids = [None] * len(all_candidate_windows)
250     if store_candidates:
251         for i, w in enumerate(all_candidate_windows):
252             stats = {
253                 "peak_velocity": w["peak_velocity"],
254                 "mean_velocity": w["mean_velocity"],
255                 "active_frame_count": w["active_frame_count"],
256                 "track_id_count": w["track_id_count"],
257             }
258             # Use peak velocity (capped at 1.0) as a coarse generic confidence
score.
259             confidence = min(1.0, w["peak_velocity"])
260             candidate_ids[i] = self.db.add_candidate_segment(
261                 video_id, detector="yolo_hybrid",
262                 start_time=w["start"], end_time=w["end"],
263                 chunk_index=w["chunk_index"], confidence=confidence,
264                 stats=stats, detection_params=detection_params,
265             )
266
267     try:
268         os.rmdir(chunks_dir)
269     except Exception:
270         pass
271
272     # Phase 3: AI Provider Handoff
273     gemini_segments = []
274     gemini_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
275     os.makedirs(gemini_dir, exist_ok=True)
276
277     logger.info(f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows...")
278
279     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
280         w_start = max(0, window["start"] - handoff_padding)
281         w_end = min(video_duration, window["end"] + handoff_padding)
282         w_dur = w_end - w_start
283
284         w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
285         # Re-encode short handoff clips to avoid keyframe-boundary drift
286         success = extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True)
287         if not success:
288             return []
289
290         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
291         segments, metrics = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur, label=f"Handoff {wi+1}")

```

```

292
293     # Coordinate mapping ? shift local timestamps back to full-video time
294     valid_segments = []
295     for segment in segments:
296         try:
297             segment["start_time"] = float(segment["start_time"]) + w_start
298             segment["end_time"] = float(segment["end_time"]) + w_start
299             # Stamp the source candidate so Phase 4 can link
prediction->confirmation.
300             segment["_candidate_id"] = candidate_ids[wi]
301             valid_segments.append(segment)
302         except (ValueError, TypeError, KeyError) as e:
303             logger.warning(f"Dropping segment with unparseable timestamps:
{segment} ? {e}")
304
305         try:
306             os.remove(w_path)
307         except OSError:
308             pass
309
310     return valid_segments
311
312     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
313         futures = [executor.submit(process_candidate_window, wi, w) for wi, w in
enumerate(all_candidate_windows)]
314         # Iterate in submission order to preserve temporal ordering
315         for future in futures:
316             gemini_segments.extend(future.result())
317
318         try:
319             os.rmdir(gemini_dir)
320         except OSError:
321             pass
322
323     logger.info(f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic
play segments.")
324
325     # Phase 4: DB Population
326     segments_data = []
327     for segment in gemini_segments:
328         try:
329             start = float(segment["start_time"])
330             end = float(segment["end_time"])
331             dur = end - start
332
333             shot_count = segment.get("shuttle_exchange_count")
334             if shot_count is None:
335                 shot_count = max(3, int(dur / 0.8))
336             else:
337                 try:
338                     shot_count = int(shot_count)
339                 except (ValueError, TypeError):
340                     shot_count = max(3, int(dur / 0.8))
341
342             metadata = {
343                 "shot_count": shot_count,
344                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
345                 "detector": f"yolo-hybrid-{model_name}"
346             }
347
348             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
349             if segment_id:

```

```

350             # Link back to the candidate that produced this segment. A candidate
351             # may yield multiple AI segments (one-to-many); confirmed_segment_id
keeps
352             # the last one ? the link table records "this candidate was
confirmed".
353             candidate_id = segment.get("_candidate_id")
354             if candidate_id is not None:
355                 self.db.link_candidate_to_segment(candidate_id, segment_id)
356             segments_data.append({
357                 "id": segment_id,
358                 "start_time": start,
359                 "end_time": end,
360                 "duration": dur,
361                 "metadata": metadata
362             })
363         except Exception as e:
364             logger.warning(f"Skipping invalid segment struct: {e}")
365
366         return segments_data
367
368     # Register the segmenter
369     segmenter_registry.register(HybridYoloSegmenter)
370

```

33. assistant (2026-06-18T01:29:00.235Z)

Let me examine the remaining validators and the gemini segmenter. Let me also look closely at a candidate issue: in `\_split\_overlong\_groups`, the `hard\_cap` midpoint chop uses `min(range(1, len(g)), key=...)`. If `split\_i` lands at `len(g)` somehow... actually `range(1, len(g))` excludes `len(g)`, so `g[split\_i:]` is non-empty. But if `split\_i == 1`, then `g[:1]` is a single element - the while loop checks `len(g) < 2` and emits. OK. But there's a potential infinite loop concern: if `mid` always maps to the same `split\_i`. Let me verify. Actually if both pieces always make progress (each strictly smaller), it terminates. With `split\_i` in `[1, len(g)-1]`, both `g[:split\_i]` and `g[split\_i:]` are strictly smaller than `g`. Good.

Let me read the remaining validators (blur, format, relevance, resolution_fps, base) and gemini.

34. user (2026-06-18T01:29:01.657Z)

```

1     import cv2
2     from backend.pipeline.validators.base import VideoValidator, ValidationResult
3     from typing import Dict, Any
4
5     class BlurValidator(VideoValidator):
6         @property
7         def name(self) -> str:
8             return "blur_check"
9
10        @property
11        def description(self) -> str:
12            return "Samples frames across the video and checks if the footage is excessively
blurry or out of focus."
13
14        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
15            cap = cv2.VideoCapture(video_path)
16            if not cap.isOpened():
17                return ValidationResult(
18                    validator_name=self.name,
19                    passed=False,
20                    message="Failed to open video file for blur analysis."
21                )
22
23            total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
24            if total_frames <= 0:

```

```

25         cap.release()
26         return ValidationResult(
27             validator_name=self.name,
28             passed=False,
29             message="Invalid frame count."
30         )
31
32     # Sample 15 frames evenly distributed across the video
33     num_samples = 15
34     step = max(1, total_frames // num_samples)
35
36     variances = []
37
38     for i in range(num_samples):
39         frame_idx = i * step
40         cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
41         ret, frame = cap.read()
42         if not ret:
43             continue
44
45         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
46         # Calculate the Laplacian variance (standard focus metric)
47         laplacian_var = cv2.Laplacian(gray, cv2.CV_64F).var()
48         variances.append(laplacian_var)
49
50     cap.release()
51
52     if not variances:
53         return ValidationResult(
54             validator_name=self.name,
55             passed=False,
56             message="Could not read frames to perform blur verification."
57         )
58
59     avg_variance = sum(variances) / len(variances)
60
61     details = {
62         "avg_laplacian_variance": round(avg_variance, 2),
63         "min_laplacian_variance": round(min(variances), 2),
64         "max_laplacian_variance": round(max(variances), 2),
65         "sampled_frames_count": len(variances)
66     }
67
68     val_cfg = config.get("validation", {})
69     min_blur_threshold = val_cfg.get("min_blur_threshold", 100.0)
70
71     if avg_variance < min_blur_threshold:
72         return ValidationResult(
73             validator_name=self.name,
74             passed=False,
75             message=f"Video quality rejected: Focus check failed. Average sharpness
score of {round(avg_variance, 1)} is below the minimum required threshold of
{min_blur_threshold}.",
76             details=details
77         )
78
79     return ValidationResult(
80         validator_name=self.name,
81         passed=True,
82         message="Video sharpness meets focus requirements.",
83         details=details
84     )
85

```

```

1      from backend.pipeline.validators.base import VideoValidator, ValidationResult
2      from typing import Dict, Any
3
4      class ResolutionFPSValidator(VideoValidator):
5          @property
6          def name(self) -> str:
7              return "resolution_fps_check"
8
9          @property
10         def description(self) -> str:
11             return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate
(e.g. 30fps) requirements."
12
13         def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
14             meta = context.get("ffprobe_meta")
15             if not meta or not meta.get("streams"):
16                 return ValidationResult(
17                     validator_name=self.name,
18                     passed=False,
19                     message="Video metadata is missing. Run format_check validator first."
20                 )
21
22             stream = meta["streams"][0]
23             width = int(stream.get("width", 0))
24             height = int(stream.get("height", 0))
25
26             # Parse frame rate (e.g. "30/1" or "30000/1001")
27             fps_str = stream.get("r_frame_rate", "0/1")
28             try:
29                 num, den = map(int, fps_str.split("/"))
30                 fps = num / den if den != 0 else 0
31             except ValueError:
32                 fps = 0.0
33
34             details = {
35                 "width": width,
36                 "height": height,
37                 "fps": round(fps, 2)
38             }
39
40             # Check bounds from config
41             val_cfg = config.get("validation", {})
42             min_w = val_cfg.get("min_resolution_width", 1280)
43             min_h = val_cfg.get("min_resolution_height", 720)
44             min_fps = val_cfg.get("min_fps", 29.9)
45
46             if width < min_w or height < min_h:
47                 return ValidationResult(
48                     validator_name=self.name,
49                     passed=False,
50                     message=f"Low resolution: Found {width}x{height}, but minimum is
{min_w}x{min_h}.",
51                     details=details
52                 )
53
54             if fps < min_fps:
55                 return ValidationResult(
56                     validator_name=self.name,
57                     passed=False,
58                     message=f"Low frame rate: Found {round(fps, 2)} FPS, but minimum is
{min_fps} FPS.",
59                     details=details
60                 )
61

```

```

62         return ValidationResult(
63             validator_name=self.name,
64             passed=True,
65             message="Video resolution and frame rate meet minimum standards.",
66             details=details
67         )
68

```

36. user (2026-06-18T01:29:02.934Z)

```

1     import cv2
2     import numpy as np
3     from backend.pipeline.validators.base import VideoValidator, ValidationResult
4     from typing import Dict, Any
5
6     class RelevanceValidator(VideoValidator):
7         @property
8         def name(self) -> str:
9             return "relevance_check"
10
11        @property
12        def description(self) -> str:
13            return "Checks if the video content is relevant to badminton by scanning for
14            court color palettes and geometric features."
15
16        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
17        -> ValidationResult:
18            cap = cv2.VideoCapture(video_path)
19            if not cap.isOpened():
20                return ValidationResult(
21                    validator_name=self.name,
22                    passed=False,
23                    message="Failed to open video file for relevance validation."
24                )
25
26            total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
27            if total_frames <= 0:
28                cap.release()
29                return ValidationResult(
30                    validator_name=self.name,
31                    passed=False,
32                    message="Invalid frame count."
33                )
34
35            # Sample 10 frames across the video
36            num_samples = 10
37            step = max(1, total_frames // num_samples)
38
39            green_ratios = []
40
41            # Get sport profile config
42            sport_name = config.get("sport", "badminton")
43            from backend.sports import sport_registry
44            try:
45                sport_profile = sport_registry.get(sport_name)
46                sport_rel_cfg = sport_profile.get_relevance_config()
47            except KeyError:
48                sport_rel_cfg = {}
49
50            val_cfg = config.get("validation", {})
51            rel_cfg = val_cfg.get("relevance", {})
52
53            # Read bounds from sport profile with fallback to user config/defaults
54            hsv_lower = np.array(rel_cfg.get("court_green_lower",
55            sport_rel_cfg.get("court_green_lower", [35, 40, 40])))

```

```

53         hsv_upper = np.array(rel_cfg.get("court_green_upper",
sport_rel_cfg.get("court_green_upper", [85, 255, 255])))
54         min_court_ratio = rel_cfg.get("court_pixels_min_ratio",
sport_rel_cfg.get("court_pixels_min_ratio", 0.05))
55
56         for i in range(num_samples):
57             frame_idx = i * step
58             cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
59             ret, frame = cap.read()
60             if not ret:
61                 continue
62
63             # Downscale frame for speed
64             h, w = frame.shape[:2]
65             small_frame = cv2.resize(frame, (320, 240))
66
67             # Convert to HSV color space
68             hsv = cv2.cvtColor(small_frame, cv2.COLOR_BGR2HSV)
69
70             # Mask court green pixels
71             mask = cv2.inRange(hsv, hsv_lower, hsv_upper)
72             green_pixels = cv2.countNonZero(mask)
73             total_pixels = mask.shape[0] * mask.shape[1]
74
75             ratio = green_pixels / total_pixels
76             green_ratios.append(ratio)
77
78         cap.release()
79
80         if not green_ratios:
81             return ValidationResult(
82                 validator_name=self.name,
83                 passed=False,
84                 message="Failed to extract frames for relevance checking."
85             )
86
87         avg_green_ratio = sum(green_ratios) / len(green_ratios)
88
89         details = {
90             "avg_green_court_ratio": round(avg_green_ratio, 4),
91             "max_green_court_ratio": round(max(green_ratios), 4),
92             "min_green_court_ratio": round(min(green_ratios), 4),
93             "green_threshold": min_court_ratio
94         }
95
96         # If a certain amount of court color is visible, consider it relevant
97         if avg_green_ratio < min_court_ratio:
98             return ValidationResult(
99                 validator_name=self.name,
100                 passed=False,
101                 message=f"Video rejected: Relevance check failed. The scene does not
contain a recognizable {sport_name} court (court area found: {round(avg_green_ratio * 100, 2)}%
vs min required {round(min_court_ratio * 100, 2)}%).",
102                 details=details
103             )
104
105         return ValidationResult(
106             validator_name=self.name,
107             passed=True,
108             message=f"Video identified as relevant to {sport_name} ({sport_name} court
structures found).",
109             details=details
110         )
111

```

37. assistant (2026-06-18T01:29:06.056Z)

Now let me look at the gemini segmenter and the wasb_infer.py, and the format/blur validators. Let me also examine the `\_select\_for\_handoff` person path more carefully and look at `tracknet\_hybrid.py`.

38. user (2026-06-18T01:29:07.219Z)

```
1     import os
2     import time
3     import logging
4     import subprocess
5     import concurrent.futures
6     from typing import List, Dict, Any, Tuple
7     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
8     from backend.utils.video import get_video_duration, extract_video_chunk
9     from backend.sports import sport_registry
10    from backend.providers import provider_registry
11    from backend.reporting import run_signals
12
13    logger = logging.getLogger(__name__)
14
15
16    def _chunk_workdir(video_id: str) -> str:
17        """Per-video chunk staging dir under output/chunks/<video_id>.
18
19        Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
20        otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
21        in-flight work. A per-video subdir means each run only ever creates/cleans its own.
22        """
23        return os.path.join("output", "chunks", video_id)
24
25
26    class GeminiPlaySegmenter(BasePlaySegmenter):
27        @property
28        def name(self) -> str:
29            return "gemini"
30
31        @property
32        def description(self) -> str:
33            return "High-precision AI-powered semantic play segment detection using Google Gemini Multimodal API."
34
35        def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
36            """
37            Processes a video to detect play segments using Google Gemini Multimodal Video
API,
38            populates SQLite, and indexes detailed events.
39            """
40            # Get sport profile config
41            sport_name = self.config.get("sport", "badminton")
42            try:
43                sport_profile = sport_registry.get(sport_name)
44            except KeyError as e:
45                logger.error(str(e))
46            return []
47
48            # Get AI provider and model config
49            idx_cfg = self.config.get("indexing", {})
50            provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
51            model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
52
```



```

53         # Get appropriate API key based on the provider
54         api_key_env_var = f"{provider_name.upper()}_API_KEY"
55         api_key = os.environ.get(api_key_env_var)
56         if not api_key:
57             logger.error(
58                 f"{api_key_env_var} environment variable is not set! "
59                 f"To run the {provider_name} segmenter, set your API key. "
60                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
61             )
62         return []
63
64         try:
65             provider = provider_registry.get(provider_name, api_key=api_key,
66 model_name=model_name)
67         except KeyError as e:
68             logger.error(str(e))
69             return []
70
71         # A: Check for debug_duration to trim the video first
72         debug_duration = self.config.get("indexing", {}).get("debug_duration")
73         video_to_upload = video_path
74         is_trimmed = False
75         temp_trimmed_path = None
76
77         if debug_duration:
78             logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
79             temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
80             os.makedirs("output", exist_ok=True)
81             if os.path.exists(temp_trimmed_path):
82                 try:
83                     os.remove(temp_trimmed_path)
84                 except Exception:
85                     pass
86
87             cmd = [
88                 "ffmpeg", "-y",
89                 "-ss", "0",
90                 "-t", str(debug_duration),
91                 "-i", video_path,
92                 "-c", "copy",
93                 temp_trimmed_path
94             ]
95             logger.debug(f"Running command: {' '.join(cmd)}")
96             try:
97                 subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
98                 logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
99                 video_to_upload = temp_trimmed_path
100                 is_trimmed = True
101             except Exception as e:
102                 logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")
103
104
105         # Helper to clean up local temp files
106         def cleanup_temp_files():
107             if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
108                 try:
109                     os.remove(temp_trimmed_path)
110                 except Exception:
111                     pass
112

```

```

113         # 1. Determine actual video duration
114         video_duration = get_video_duration(video_to_upload)
115         if video_duration > 0:
116             logger.info(f"Detected video duration: {video_duration:.1f}s")
117         else:
118             logger.warning("Could not detect video duration. Prompt will not include
duration context.")
119
120         # 2. Decide whether to use chunked processing
121         idx_cfg = self.config.get("indexing", {})
122         chunk_duration = idx_cfg.get("chunk_duration", 300)    # 5 minutes default
123         chunk_overlap = idx_cfg.get("chunk_overlap", 30)       # 30 seconds overlap
124         use_chunking = video_duration > 0 and video_duration > chunk_duration
125         # Re-encode + downscale chunks for upload (0 = legacy stream-copy). Stream-
copied
126         # 4K chunks (~1.3GB/90s) exceed the provider's MAX_UPLOAD_MB and are refused, so
127         # every chunk of a 4K source gets skipped; Gemini analyzes at low res anyway.
128         ai_scale_width = int(idx_cfg.get("ai_chunk_scale_width", 1280) or 0)
129
130         ai_segments = []
131         # Per-video chunk workdir (concurrency isolation): a concurrent run for another
132         # video keeps its own dir, so cleanup here never removes another run's chunks.
133         chunks_dir = _chunk_workdir(video_id)
134
135         if use_chunking:
136             # --- CHUNKED PROCESSING ---
137             step = chunk_duration - chunk_overlap
138             chunk_starts = []
139             t = 0.0
140             while t < video_duration:
141                 chunk_starts.append(t)
142                 t += step
143             num_chunks = len(chunk_starts)
144             logger.info(
145                 f"Video is {video_duration:.1f}s ? splitting into {num_chunks}
overlapping chunks "
146                 f"({chunk_duration}s each, {chunk_overlap}s overlap).")
147             )
148
149             os.makedirs(chunks_dir, exist_ok=True)
150
151             def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list,
dict]:
152                 chunk_end = min(chunk_start + chunk_duration, video_duration)
153                 actual_chunk_dur = chunk_end - chunk_start
154                 chunk_label = f"Chunk {ci+1}/{num_chunks}"
155                 chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
156
157                 # Extract chunk with ffmpeg
158                 logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
159                 success = extract_video_chunk(
160                     video_to_upload, chunk_start, actual_chunk_dur, chunk_path,
161                     reencode=ai_scale_width > 0,
162                     scale_width=ai_scale_width if ai_scale_width > 0 else None,
163                 )
164                 if not success:
165                     logger.error(f"Failed to extract {chunk_label}. Skipping.")
166                     return chunk_label, [], {}
167
168                 # Measure chunk size
169                 try:
170                     chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
171                 except Exception:
172                     chunk_size_mb = 0.0

```

```

173
174         # Get prompt from sport profile
175         prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
176
177         # Process this chunk through the provider
178         chunk_segments, metrics = provider.analyze_video(
179             chunk_path, prompt, chunk_duration=actual_chunk_dur,
label=chunk_label
180         )
181         metrics["file_size_mb"] = chunk_size_mb
182         metrics["segments_found"] = len(chunk_segments)
183
184         # Offset timestamps back to full-video coordinates
185         for segment in chunk_segments:
186             if "start_time" in segment:
187                 try:
188                     segment["start_time"] = float(segment["start_time"]) +
chunk_start
189
190                     except (ValueError, TypeError):
191                         pass
192             if "end_time" in segment:
193                 try:
194                     segment["end_time"] = float(segment["end_time"]) +
chunk_start
195
196                     except (ValueError, TypeError):
197                         pass
198
199             logger.info(f"{chunk_label} returned {len(chunk_segments)} play
segments.")
200
201         # Clean up chunk file
202         try:
203             os.remove(chunk_path)
204         except Exception:
205             pass
206
207         return chunk_label, chunk_segments, metrics
208
209     # Run chunks in parallel. Workers come from indexing.ai_max_workers (same
knob
210     # the hybrids honor; was hardcoded 5). ai_max_workers=1 = serial ? needed to
211     # gently ramp a cold/prepaid Gemini project, whose burst throttling returns
a
212     # misleading "credits depleted" 429 under 5 simultaneous uploads.
213     all_metrics = {}
214     max_workers = max(1, int(self.config.get("indexing",
215     {}).get("ai_max_workers", 5)))
216     logger.info("Starting parallel processing with up to %d concurrent
workers...", max_workers)
217     t_chunking_start = time.time()
218     with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as
executor:
219         futures = [executor.submit(process_single_chunk, ci, start) for ci,
start in enumerate(chunk_starts)]
220         for future in concurrent.futures.as_completed(futures):
221             label, segments, metrics = future.result()
222             ai_segments.extend(segments)
223             if metrics:
224                 all_metrics[label] = metrics
225
226     # Oversize refusals must be LOUD: each one is a time range with NO analysis,
227     # so "0 segments / success" here would be a silent failure, not "no
rallies".
228     oversize_skips = sum(1 for m in all_metrics.values() if
m.get("oversize_skipped"))

```

```

226         if oversize_skips:
227             logger.error(
228                 f"{oversize_skips}/{num_chunks} chunks exceeded the provider upload
cap and were "
229                 f"SKIPPED WITHOUT ANALYSIS ? segments are missing for those time
ranges. "
230                 f"Set indexing.ai_chunk_scale_width (default 1280) so chunks are re-
encoded/"
231                 f"downscaled before upload."
232             )
233             run_signals.incr(video_id, "oversize_clips_skipped", oversize_skips)
234
235         # Print Summary Log
236         summary_lines = [
237             "",
238             "=" * 60,
239             "                                GEMINI CHUNKING SUMMARY",
240             "=" * 60,
241             f" Total time: {time.time() - t_chunking_start:.1f}s",
242             f" Total chunks: {num_chunks}",
243             f" Total play segments found: {len(ai_segments)}",
244             "-" * 60,
245             f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments'}}",
246             "-" * 60,
247         ]
248         for i in range(num_chunks):
249             lbl = f"Chunk {i+1}/{num_chunks}"
250             m = all_metrics.get(lbl, {})
251             sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
252             upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
253             prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
254             gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
255             seg = m.get('segments_found', 0) if m else 0
256             summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} |
{gen:<8} | {seg}")
257         summary_lines.append("=" * 60)
258         logger.info("\n".join(summary_lines))
259
260         # Clean up local temp files
261         cleanup_temp_files()
262         try:
263             os.rmdir(chunks_dir)
264         except Exception:
265             pass
266
267         logger.info(f"All chunks processed. {len(ai_segments)} total play segments
before deduplication.")
268
269     else:
270         # --- SINGLE-FILE PROCESSING (short videos) ---
271         try:
272             file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
273         except Exception:
274             file_sz = 0.0
275
276         # Get prompt from sport profile
277         prompt = sport_profile.get_prompt(chunk_duration=video_duration)
278         ai_segments, metrics = provider.analyze_video(
279             video_to_upload, prompt, chunk_duration=video_duration,
label="SingleFile"
280         )
281         if metrics.get("oversize_skipped"):
282             logger.error(
283                 f"The video ({file_sz:.0f}MB) exceeded the provider upload cap and

```

```

was "
284             f"SKIPPED WITHOUT ANALYSIS ? 0 segments here does NOT mean 'no
rallies'."
285         )
286         run_signals.incr(video_id, "oversize_clips_skipped", 1)
287         summary_lines = [
288             "",
289             "=" * 50,
290             "            GEMINI SINGLE FILE SUMMARY",
291             "=" * 50,
292             f" File Size: {file_sz:.1f} MB",
293             f" Upload time: {metrics.get('upload_time', 0):.1f}s",
294             f" API Processing time: {metrics.get('process_time', 0):.1f}s",
295             f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
296             f" Play segments found: {len(ai_segments)}",
297             "=" * 50,
298         ]
299         logger.info("\n".join(summary_lines))
300         cleanup_temp_files()
301
302     if not ai_segments:
303         return []
304
305     # Populate SQLite DB with parsed play segments
306     segments_data = []
307     logger.info(f"Processing {len(ai_segments)} play segments through
validation...")
308
309     def parse_time(val) -> float:
310         """Converts a timestamp value to total float seconds.
311         Handles float/int pass-through, plain numeric strings, and
312         colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
313         when unexpected formats are encountered."""
314         if isinstance(val, (int, float)):
315             return float(val)
316         if isinstance(val, str):
317             val = val.strip()
318             if ":" in val:
319                 # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal
seconds.
320
321                 # Convert it, but warn so the issue is visible in logs.
322                 parts = val.split(":")
323                 parts.reverse()
324                 total = 0.0
325                 for i, p in enumerate(parts):
326                     try:
327                         total += float(p) * (60 ** i)
328                     except ValueError:
329                         pass
330                 logger.warning(f"Timestamp '{val}' was in colon-separated format
(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
331                 return total
332             try:
333                 return float(val)
334             except ValueError:
335                 logger.warning(f"Could not parse timestamp string '{val}' as a
number. Defaulting to 0.0.")
336                 return 0.0
337             logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
338             return 0.0
339
340     # 7. Parse timestamps and build candidate segment list
341     idx_cfg = self.config.get("indexing", {})
342     min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)

```

```

342         max_segment_duration = 90.0 # Singles play segments almost never exceed this
343
344         candidates = []
345         for idx, segment in enumerate(ai_segments):
346             start = parse_time(segment.get("start_time", 0.0))
347             end = parse_time(segment.get("end_time", 0.0))
348             if start >= end:
349                 logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s)")
350                 continue
351                 candidates.append({
352                     "idx": idx,
353                     "start": start,
354                     "end": end,
355                     "raw": segment
356                 })
357
358         # 8. Post-hoc validation pass
359         logger.info(f"Running validation on {len(candidates)} candidate play
segments...")
360         validated = []
361         rejected = []
362
363         for c in candidates:
364             label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"
365
366             # A: Reject segments that start beyond the video
367             if video_duration > 0 and c["start"] >= video_duration:
368                 rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
369                 continue
370
371             # B: Clamp end_time to video duration
372             if video_duration > 0 and c["end"] > video_duration:
373                 logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
374                 c["end"] = video_duration
375
376             dur = c["end"] - c["start"]
377
378             # C: Reject segments shorter than min_segment_duration
379             if dur < min_segment_duration:
380                 rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
381                 continue
382
383             # D: Flag suspiciously long segments
384             if dur > max_segment_duration:
385                 logger.warning(
386                     f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
387                     f"This may be a detection error. Keeping but flagging."
388                 )
389
390             validated.append(c)
391
392         # E: Resolve overlapping segments (keep the longer one)
393         if len(validated) > 1:
394             # Sort by start time
395             validated.sort(key=lambda r: r["start"])
396             non_overlapping = [validated[0]]
397             for curr in validated[1:]:
398                 prev = non_overlapping[-1]
399                 # Check if current segment overlaps with the previous one
400                 if curr["start"] < prev["end"]:
```

```

401         prev_dur = prev["end"] - prev["start"]
402         curr_dur = curr["end"] - curr["start"]
403         kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
prev)
404         kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
{kept['end']:.2f}s]"
405         dropped_label = f"Segment #{dropped['idx']+1}
[{dropped['start']:.2f}s - {dropped['end']:.2f}s]"
406         rejected.append((dropped_label, f"overlaps with {kept_label} ? kept
the longer segment"))
407         if kept == curr:
408             non_overlapping[-1] = curr # Replace previous with current
409         else:
410             non_overlapping.append(curr)
411         validated = non_overlapping
412
413         # Log validation summary
414         if rejected:
415             reject_lines = [f" ? {label}: {reason}" for label, reason in rejected]
416             logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
"\n".join(reject_lines))
417             logger.info(f"{len(validated)} play segments passed validation (out of
{len(ai_segments)} from Gemini).")
418
419         stitch_cfg = self.config.get("stitching", {})
420         begin_pad = stitch_cfg.get("begin_padding", 0.5)
421         end_pad = stitch_cfg.get("end_padding", 0.5)
422
423         total_highlight_dur = 0.0
424         for c in validated:
425             total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad
426
427         mins = int(total_highlight_dur // 60)
428         secs = int(total_highlight_dur % 60)
429         logger.info(
430             f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
431             f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin +
{end_pad}s end padding per segment)"
432         )
433
434         # 9. Populate SQLite DB with validated play segments
435         for c in validated:
436             start = c["start"]
437             end = c["end"]
438             segment = c["raw"]
439             dur = end - start
440
441             # Extract shot_count from Gemini or compute estimate if not present
442             shot_count = segment.get("shuttle_exchange_count")
443             if shot_count is None:
444                 shot_count = max(3, int(dur / 0.8)) # 1 shot every 0.8 seconds as
fallback estimate
445             else:
446                 try:
447                     shot_count = int(shot_count)
448                 except Exception:
449                     shot_count = max(3, int(dur / 0.8))
450
451             # Only store metrics we actually know ? no fabricated data
452             metadata = {
453                 "shot_count": shot_count,
454                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
455                 "detector": model_name
456             }

```

```

457
458         segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
459
460         if segment_id:
461             segments_data.append({
462                 "id": segment_id,
463                 "start_time": start,
464                 "end_time": end,
465                 "duration": dur,
466                 "metadata": metadata
467             })
468
469         return segments_data
470
471     # Register the segmenter
472     segmenter_registry.register(GeminiPlaySegmenter)
473

```

39. assistant (2026-06-18T01:29:11.883Z)

Now let me look at the format validator and wasb_infer. Let me also re-examine the `motion.py` `Failure` return path: when `cap` fails to open, it returns `Failure(...)` without calling `cap.release()`. That's minor since cap failed to open. But more interesting ? let me reconsider the `\_select\_for\_handoff` person path and `detections\_over\_windows` edge cases.

Let me check `wasb\_infer.py` and the format validator.

40. user (2026-06-18T01:29:12.857Z)

```

1     import subprocess
2     import json
3     import os
4     from backend.pipeline.validators.base import VideoValidator, ValidationResult
5     from typing import Dict, Any
6
7     class FormatValidator(VideoValidator):
8         @property
9         def name(self) -> str:
10             return "format_check"
11
12         @property
13         def description(self) -> str:
14             return "Verifies the video is a valid, uncorrupted MP4 container and extracts
metadata."
15
16         def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
17             if not os.path.exists(video_path):
18                 return ValidationResult(
19                     validator_name=self.name,
20                     passed=False,
21                     message=f"File not found: {video_path}"
22                 )
23
24             cmd = [
25                 "ffprobe",
26                 "-v", "error",
27                 "-select_streams", "v:0",
28                 "-show_format",
29                 "-show_streams",
30                 "-of", "json",
31                 video_path
32             ]
33

```



```

34         try:
35             result = subprocess.run(cmd, capture_output=True, text=True, check=True)
36             info = json.loads(result.stdout)
37
38             if not info.get("streams") or not info.get("format"):
39                 return ValidationResult(
40                     validator_name=self.name,
41                     passed=False,
42                     message="Failed to retrieve valid video streams from the file."
43                 )
44
45             # Store the metadata in context for subsequent validators to utilize
46             context["ffprobe_meta"] = info
47
48             stream = info["streams"][0]
49             fmt = info["format"]
50
51             details = {
52                 "format_name": fmt.get("format_name"),
53                 "duration": float(fmt.get("duration", 0)),
54                 "size_bytes": int(fmt.get("size", 0)),
55                 "codec_name": stream.get("codec_name"),
56             }
57
58             # Check if container name includes mp4
59             if "mp4" not in fmt.get("format_name", "").lower():
60                 return ValidationResult(
61                     validator_name=self.name,
62                     passed=False,
63                     message=f"Invalid format: Found '{fmt.get('format_name')}', but MP4
is required.",
64                     details=details
65                 )
66
67             return ValidationResult(
68                 validator_name=self.name,
69                 passed=True,
70                 message="Video format is valid MP4 container.",
71                 details=details
72             )
73
74         except subprocess.CalledProcessError as e:
75             return ValidationResult(
76                 validator_name=self.name,
77                 passed=False,
78                 message="Video stream is corrupted or unreadable by ffprobe.",
79                 details={"error": e.stderr}
80             )
81         except Exception as e:
82             return ValidationResult(
83                 validator_name=self.name,
84                 passed=False,
85                 message=f"Unexpected error running ffprobe: {str(e)}"
86             )
87

```

41. user (2026-06-18T01:29:13.950Z)

```

1     """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3     WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4     own building blocks (ImageDataset transform + detector + online tracker) to run on
5     any folder of extracted frames and emit a trajectory CSV in the exact shape the
6     indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7

```

```

8         Frame,Visibility,X,Y
9
10     It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11     imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12     (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23       records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24       most ``--flush-every`` windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26       un-cached window, and continue. Once every window is cached, the detector pass
27       is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28       the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29       not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36     python wasb_infer.py --frames_dir /path/frames --weights
37     ../pretrained_weights/wasb_badminton_best.pth.tar \
38     --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39     every 1000] [--fresh]
40
41     """
42     import os
43     import os.path as osp
44     import csv
45     import glob
46     import json
47     import argparse
48     import logging
49     from collections import defaultdict
50     from contextlib import contextmanager
51
52     logger = logging.getLogger(__name__)
53
54     CACHE_VERSION = 1
55     _DET_FILE = "det_raw.jsonl"
56     _MANIFEST_FILE = "manifest.json"
57
58     def _build_cfg(weights: str, sport: str):
59         """Compose the WASB eval config via Hydra, overriding model/weights/device."""
60         from hydra import compose, initialize
61         with initialize(config_path="configs", version_base=None):
62             cfg = compose(config_name="eval", overrides=[
63                 f"dataset={sport}",
64                 "model=wasb",
65                 f"detector.model_path={weights}",
66                 "runner.device=cuda",
67                 "runner.gpus=[0]", # this box has a single GPU; default config assumes
68
69     more
70         ])
71     return cfg
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

```

70     def _frame_id(path: str) -> int:
71         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
72         stem = osp.splitext(osp.basename(path))[0]
73         digits = "".join(ch for ch in stem if ch.isdigit())
74         return int(digits) if digits else -1
75
76
77     # ----- #
78     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
79     # ----- #
80     def _cache_paths(cache_dir: str):
81         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
82
83
84     def default_cache_dir(out_csv: str) -> str:
85         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
86         d = osp.dirname(osp.abspath(out_csv))
87         stem = osp.splitext(osp.basename(out_csv))[0]
88         return osp.join(d, stem + "_wasbcache")
89
90
91     def _atomic_write_text(path: str, text: str) -> None:
92         """Write then rename so a crash mid-write can't leave a half-written file."""
93         tmp = path + ".tmp"
94         with open(tmp, "w") as f:
95             f.write(text)
96             f.flush()
97             os.fsync(f.fileno())
98         os.replace(tmp, path)
99
100
101     def _det_plain(det) -> list:
102         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
103         xy = det["xy"]
104         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
105
106
107     def _dets_to_tracker(plain_list):
108         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
109         the tracker does vector math / np.linalg.norm on it)."""
110         import numpy as np
111         return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
112                 for p in plain_list]
113
114
115     def write_manifest(cache_dir: str, manifest: dict) -> None:
116         _, mpath = _cache_paths(cache_dir)
117         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
118
119
120     def read_manifest(cache_dir: str):
121         _, mpath = _cache_paths(cache_dir)
122         if not osp.exists(mpath):
123             return None
124         try:
125             with open(mpath) as f:
126                 return json.load(f)
127         except (json.JSONDecodeError, OSError):
128             return None
129
130
131     def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,

```

```

132             frames_in: int, frames_total: int) -> bool:
133         """A cache is reusable only if the run that produced it matches this run's
inputs."""
134         if not manifest or manifest.get("version") != CACHE_VERSION:
135             return False
136         return (
137             manifest.get("frames_dir") == osp.abspath(frames_dir)
138             and manifest.get("weights") == weights
139             and manifest.get("sport") == sport
140             and manifest.get("frames_in") == frames_in
141             and manifest.get("frames_total") == frames_total
142         )
143
144
145     def reset_cache(cache_dir: str) -> None:
146         """Drop any existing cache so the next run starts clean."""
147         det_jsonl, mpath = _cache_paths(cache_dir)
148         for p in (det_jsonl, mpath):
149             if osp.exists(p):
150                 os.remove(p)
151
152
153     def load_and_clean_cache(cache_dir: str):
154         """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
155         rebuild the per-frame detection accumulation, and rewrite the file to exactly that
156         prefix so a trailing half-written line from a crash can't corrupt future appends.
157
158         Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
159         [x, y, score, scale] candidates (accumulated across every window the frame is in,
160         in window order ? identical to a non-resumed run).
161         """
162         det_jsonl, _ = _cache_paths(cache_dir)
163         det_by_fid = defaultdict(list)
164         valid: list = []
165         if osp.exists(det_jsonl):
166             with open(det_jsonl, "r") as f:
167                 for line in f:
168                     s = line.strip()
169                     if not s:
170                         continue
171                     try:
172                         obj = json.loads(s)
173                     except json.JSONDecodeError:
174                         break # truncated trailing line (crash mid-
flush)
175                     if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
176                         break
177                     valid.append(s)
178                     for fid, plain in obj["f"]:
179                         det_by_fid[int(fid)].extend([list(p) for p in plain])
180                     # Guarantee a clean append boundary by rewriting only the good prefix.
181                     _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
182         return len(valid), det_by_fid
183
184
185     def _append_windows(fh, objs) -> None:
186         """Append window records as JSONL and durably flush (bounded loss on crash)."""
187         for o in objs:
188             fh.write(json.dumps(o, separators=(",", ":")) + "\n")
189         fh.flush()
190         os.fsync(fh.fileno())
191
192
193     def _atomic_write_trajectory(out_csv: str, rows) -> None:
194         os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)

```

```

195     tmp = out_csv + ".tmp"
196     with open(tmp, "w", newline="") as f:
197         w = csv.writer(f)
198         w.writerow(["Frame", "Visibility", "X", "Y"])
199         w.writerows(rows)
200         f.flush()
201         os.fsync(f.fileno())
202     os.replace(tmp, out_csv)
203
204
205     # ----- #
206     # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
207     # ----- #
208     _STREAM_FRAME_FMT = "{:08d}.png"
209
210
211     def synthetic_frame_path(index: int) -> str:
212         """Stable, index-encoding frame name used by the streaming path.
213
214         It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
215         downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
216         frames came from disk or from the in-memory stream. The streaming image loader
217         inverts this name back to an index to fetch the decoded frame.
218         """
219         return _STREAM_FRAME_FMT.format(int(index))
220
221
222     def build_windows(frames: list, frames_in: int):
223         """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
224         + checkpoint). Pure list math, identical for the disk and streaming paths.
225
226         Returns a list of windows, each a list of ``frames_in`` consecutive paths
227         (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
228         caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
229         ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
230         """
231         return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]
232
233
234     class SequentialFrameStore:
235         """In-memory sliding buffer over a forward-only frame reader, sized for the
236         detector's sliding-window access pattern (peak O(window) frames, not O(video)).
237
238         The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
239         ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
240         in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
241         start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
242         boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
243         far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
244         earliest index any not-yet-finished window can still ask for) and decode each source
245         frame exactly once via the forward-only ``reader(index)``.
246         """
247
248         def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
249             self._reader = reader
250             self._total = int(total)
251             self._window = max(1, int(window))
252             self._buf: dict = {}
253             # On a resumed run the detector's first needed frame is `start_index`; begin
254             # pulling there so the reader can skip (seek past) the already-cached prefix
255             # instead of decoding 0..start_index-1 just to throw them away.
256             self._next = int(start_index) # next index not yet pulled from the reader
257             self._max_seen = int(start_index) - 1
258             self._floor = int(start_index) # lowest index we still keep (never evict
below)

```

```

259
260     def get(self, index: int):
261         index = int(index)
262         if index < self._floor:
263             raise KeyError(
264                 f"frame {index} already evicted (below floor {self._floor}); the access
265                 "
266                 f"pattern must stay within a window of the highest frame seen")
267         if index >= self._total:
268             raise KeyError(f"frame {index} out of range (total {self._total})")
269         # Pull forward from the reader until the requested index is buffered.
270         while self._next <= index:
271             self._buf[self._next] = self._reader(self._next)
272             self._next += 1
273         if index > self._max_seen:
274             self._max_seen = index
275         # Evict frames older than the earliest a still-running window could re-request:
276         # once we've seen index `m`, no future window starts before `m - (window-1)`.
277         new_floor = max(self._floor, self._max_seen - (self._window - 1))
278         for k in range(self._floor, new_floor):
279             self._buf.pop(k, None)
280         self._floor = new_floor
281         return self._buf[index]
282
283     def _index_from_synthetic_path(path: str) -> int:
284         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
285
286         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
287         in lockstep with the on-disk naming scheme.
288         """
289         return _frame_id(path)
290
291     def _bgr_to_pil_rgb(frame_bgr):
292         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
293         EXACT
294         PIL RGB image WASB's disk path produces.
295
296         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
297         Image.open(PNG).convert('RGB')``;
298         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
299         COLOR_BGR2RGB)``
300         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
301         streamed
302         frame indistinguishable from the on-disk one to everything downstream.
303         """
304         import cv2
305         from PIL import Image
306         return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
307
308     def _make_streamed_read_image(store, real_read_image):
309         """Build the ``read_image`` replacement used during a streaming detector pass.
310
311         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
312         image,
313         matching ``read_image``'s real return type); any path that doesn't parse to an index
314         falls
315         through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
316         testable
317         without the WSL-only ``dataloaders`` package.
318         """
319         def _read_image(path, *args, **kwargs):
320             idx = _index_from_synthetic_path(path)

```

```

316         if idx < 0:
317             return real_read_image(path, *args, **kwargs)
318             return _bgr_to_pil_rgb(store.get(idx))
319     return _read_image
320
321
322     @contextmanager
323     def _streaming_read_image_patch(frame_loader, total: int, window: int = 1, start_index:
324     int = 0):
325         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
326         decoded
327         frames during the detector pass, byte-for-byte as it would over real PNGs.
328
329         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
330         ``cv2.imread``. ``read_image`` (``utils.utils``) does
331         ``Image.open(path).convert('RGB')``,
332         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
333         feed it a
334         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
335         byte-identical to the disk round-trip, and the shim never touches the filesystem so
336         the
337         ``osp.exists`` guard is sidestepped for synthetic stream paths.
338
339         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
340         actually
341         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
342         in that
343         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
344         imported
345         reference).
346
347         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
348         wrapped
349         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
350         on a
351         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
352         The
353         original reader is restored on exit.
354         """
355         from dataloaders import dataset_loader as _dl
356         store = SequentialFrameStore(frame_loader, total, window=window,
357         start_index=start_index)
358         real_read_image = _dl.read_image
359         # Rebind read_image in the consuming module for the duration of the detector pass;
360         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
361         B010-clean.)
362         _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
363         ignore[assignment]
364         try:
365             yield store
366         finally:
367             _dl.read_image = real_read_image # type: ignore[assignment]
368
369
370     # ----- #
371     # Inference
372     # ----- #
373     def run(frames, weights: str, out_csv: str, sport: str = "badminton",
374             limit: int = 0, batch_size: int = 8, num_workers: int = 4,
375             log_every_batches: int = 50, cache_dir: "str | None" = None,
376             flush_every: int = 1000, fresh: bool = False,
377             frame_loader=None, source_key: "str | None" = None) -> str:
378         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
379
380         Two equivalent ways to supply pixels (output is bit-identical between them):

```

```

367
368     * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
369     globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
370     ``cv2.imread``.
371     * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
372     frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
373     pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
374     reader, which returns a PIL RGB image) over the DataLoader pass so every other
375     line of
376     ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
377     identical
378     to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
379     ``cvtColor(bgr, BGR2RGB)``), verified byte-identical). Streaming forces
380     ``num_workers=0``
381     (the in-process shim + buffer can't cross worker processes).
382
383     ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
384     frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
385     after reboot still matches.
386     """
387     import time
388     import torch
389     from torch.utils.data import DataLoader
390     from detectors import build_detector
391     from trackers import build_tracker
392     from dataloaders import build_img_transforms, build_seq_transforms
393     from dataloaders.dataset_loader import ImageDataset
394     from utils import Center
395
396     streaming = frame_loader is not None
397
398     cfg = _build_cfg(weights, sport)
399     frames_in = int(cfg["model"]["frames_in"])
400     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
401     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
402
403     if streaming:
404         # `frames` is already the ordered list of (synthetic) frame paths.
405         if not isinstance(frames, (list, tuple)):
406             raise SystemExit("streaming mode requires `frames` to be a list of frame
407             paths")
408         frames = list(frames)
409         frames_dir = source_key or "stream"
410     else:
411         frames_dir = frames
412         frames = sorted(glob.glob(osp.join(frames_dir, "*.png"))) +
413         glob.glob(osp.join(frames_dir, "*.jpg"))
414         if limit:
415             frames = frames[:limit]
416         if len(frames) < frames_in:
417             where = frames_dir if not streaming else "video stream"
418             raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
419
420     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
421     checkpoint).
422     samples = []
423     for window in build_windows(frames, frames_in):
424         annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
425         for p in window]
426         samples.append({"frames": window, "annos": annos})
427     windows_total = len(samples)
428
429     # ---- cache setup / resume decision ----- #
430     if cache_dir is None:
431         cache_dir = default_cache_dir(out_csv)

```



```

424 os.makedirs(cache_dir, exist_ok=True)
425 det_jsonl, _ = _cache_paths(cache_dir)
426 cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
427 manifest = None if fresh else read_manifest(cache_dir)
428 resume = manifest_compatible(
429     manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
430     frames_in=frames_in, frames_total=len(frames),
431 )
432 if resume:
433     windows_done, det_by_fid = load_and_clean_cache(cache_dir)
434     logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
435 else:
436     if manifest is not None:
437         logger.info("cache incompatible with this run -> starting fresh")
438         reset_cache(cache_dir)
439         windows_done, det_by_fid = 0, defaultdict(list)
440
441 base_manifest = {
442     "version": CACHE_VERSION,
443     "frames_dir": cache_key,
444     "weights": weights,
445     "sport": sport,
446     "frames_in": frames_in,
447     "frames_total": len(frames),
448     "windows_total": windows_total,
449     "windows_done": windows_done,
450 }
451 write_manifest(cache_dir, base_manifest)
452
453 # ---- detector pass over the un-cached windows ----- #
454 remaining = samples[windows_done:]
455 if remaining:
456     _, transform_test = build_img_transforms(cfg)
457     try:
458         _, seq_transform_test = build_seq_transforms(cfg)
459     except Exception:
460         seq_transform_test = None
461     ds = ImageDataset(cfg, remaining, input_wh, output_wh,
462                      transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
463     # Streaming forces single-process loading: the in-memory frame store + the
464     # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
465     loader_workers = 0 if streaming else num_workers
466     loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=loader_workers)
467     detector = build_detector(cfg)
468
469     from contextlib import ExitStack
470
471     t0 = time.time()
472     done = windows_done
473     win_idx = windows_done
474     log_every = max(1, log_every_batches)
475     flush_n = max(1, flush_every)
476     pending = []
477     logger.info(f"detecting on {len(remaining)} remaining windows "
478               f"(total {windows_total}, batch={batch_size},
workers={loader_workers}, "
479               f"source={'video-stream' if streaming else 'frames-dir'})...")
480     det_f = open(det_jsonl, "a")
481     try:
482         with ExitStack() as stack:
483             stack.enter_context(torch.no_grad())

```

```

484         # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
485         # in-memory decoded frame for that synthetic path; every other line of
486         # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
487         # The detector's first needed frame is `windows_done` (window i starts
at
488         # frame i), so prime the store there for a resume.
489         if streaming:
490             stack.enter_context(
491                 _streaming_read_image_patch(frame_loader, len(frames),
492                                             window=frames_in,
start_index=windows_done))
493         for bi, batch in enumerate(loader):
494             imgs, _hms, trans = batch[0], batch[1], batch[2]
495             img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
496             batch_results, _ = detector.run_tensor(imgs, trans)
497             nb = imgs.shape[0]
498             for ib in range(nb):
499                 frames_payload = []
500                 for ie in sorted(batch_results[ib].keys()):
501                     p = img_paths[ie][ib]
502                     fid = _frame_id(p)
503                     plain = [_det_plain(d) for d in batch_results[ib][ie]]
504                     frames_payload.append([fid, plain])
505                     det_by_fid[fid].extend(plain)
506                 pending.append({"w": win_idx, "f": frames_payload})
507                 win_idx += 1
508             done += nb
509             if len(pending) >= flush_n or done >= windows_total:
510                 _append_windows(det_f, pending)
511                 pending = []
512                 base_manifest["windows_done"] = done
513                 write_manifest(cache_dir, base_manifest)
514             if (bi + 1) % log_every == 0 or done >= windows_total:
515                 el = time.time() - t0
516                 new_done = done - windows_done
517                 rate = new_done / el if el > 0 else 0.0
518                 eta = (windows_total - done) / rate if rate > 0 else 0.0
519                 logger.info(f"{done}/{windows_total} "
520                             f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
521                             f"elapsed {el:.0f}s | ETA {eta:.0f}s")
522             if pending:
523                 _append_windows(det_f, pending)
524                 base_manifest["windows_done"] = done
525                 write_manifest(cache_dir, base_manifest)
526         finally:
527             det_f.close()
528             logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
529     else:
530         logger.info("detector cache already complete -> tracker-only re-run")
531
532     # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
533     tracker = build_tracker(cfg)
534     tracker.refresh()
535     rows = []
536     for fid in sorted(det_by_fid.keys()):
537         r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
538         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
539
540     _atomic_write_trajectory(out_csv, rows)
541     visible = sum(1 for _, v, *_ in rows if v)
542     logger.info(f"wrote {len(rows)} rows -> {out_csv}")

```

```

543     logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
544     return out_csv
545
546
547 def extract_frames(video_path: str, frames_dir: str) -> str:
548     """Decode a video to PNG frames named by source frame index (00000000.png ...).
549
550     Idempotent: if the dir already holds the expected number of frames (matching the
551     container's reported frame count) we skip re-decoding ? re-extraction after a
552     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
553     extracting with ffmpeg on the host and passing --frames_dir.
554     """
555     import cv2
556     os.makedirs(frames_dir, exist_ok=True)
557     cap = cv2.VideoCapture(video_path)
558     if not cap.isOpened():
559         raise SystemExit(f"cannot open video: {video_path}")
560     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
561     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
562     if expected > 0 and existing >= expected:
563         cap.release()
564         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
565         return frames_dir
566     i = 0
567     while True:
568         ok, frame = cap.read()
569         if not ok:
570             break
571         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
572         i += 1
573     cap.release()
574     logger.info(f"extracted {i} frames -> {frames_dir}")
575     return frames_dir
576
577
578 def _probe_frame_count(video_path: str) -> int:
579     """Container-reported frame count via cv2 (0 if unknown). Used to size the
580     stream."""
581     import cv2
582     cap = cv2.VideoCapture(video_path)
583     if not cap.isOpened():
584         raise SystemExit(f"cannot open video: {video_path}")
585     n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
586     cap.release()
587     return n
588
589 def make_video_frame_loader(video_path: str):
590     """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
591
592     ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
593     long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
594     detector's sliding-window order); it reads sequentially and only uses a frame-seek
595     to
596     skip forward when a *resume* starts past the current position. Each ``cap.read()``
597     returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
598     would
599     yield (PNG is lossless), so the detector sees identical pixels to the disk path.
600
601     Returns the count hint from the container so the caller can build the frame list;
602     ``release`` closes the capture.
603     """
604     import cv2
605     cap = cv2.VideoCapture(video_path)
606     if not cap.isOpened():

```

```

605         raise SystemExit(f"cannot open video: {video_path}")
606     count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
607     state = {"pos": 0} # next index cap.read() will return
608
609     def frame_loader(index: int):
610         index = int(index)
611         if index < state["pos"]:
612             raise RuntimeError(
613                 f"streaming decode is forward-only; got index {index} after
{state['pos']}")
614         if index > state["pos"]:
615             # Forward skip only happens when resuming past cached windows. Seek once.
616             cap.set(cv2.CAP_PROP_POS_FRAMES, index)
617             state["pos"] = index
618         ok, frame = cap.read()
619         if not ok:
620             raise RuntimeError(f"failed to decode frame {index} from {video_path}")
621         state["pos"] += 1
622         return frame
623
624     def release():
625         cap.release()
626
627     return frame_loader, count_hint, release
628
629
630     def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str =
"badminton",
631                             limit: int = 0, batch_size: int = 8,
632                             log_every_batches: int = 50, cache_dir: "str | None" = None,
633                             flush_every: int = 1000, fresh: bool = False) -> str:
634         """Fast path: decode ``video_path`` on the fly and feed frames straight to the
635         detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
636         dominates a long clip). Output is bit-identical to the frames-on-disk path; see
637         ``run``'s docstring for why.
638         """
639         frame_loader, count_hint, release = make_video_frame_loader(video_path)
640         try:
641             if count_hint <= 0:
642                 raise SystemExit(
643                     f"could not determine frame count for {video_path}; cannot stream "
644                     f"({fall back to frame extraction with --frames_dir}")
645             frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
646             source_key = "video:" + osp.abspath(video_path)
647             return run(frame_list, weights, out_csv, sport, limit,
648                       batch_size=batch_size, num_workers=0,
649                       log_every_batches=log_every_batches, cache_dir=cache_dir,
650                       flush_every=flush_every, fresh=fresh,
651                       frame_loader=frame_loader, source_key=source_key)
652         finally:
653             release()
654
655
656     def main():
657         ap = argparse.ArgumentParser()
658         ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
659         ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
660         ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
661         ap.add_argument("--stream-video", action="store_true",
662                         help="FAST PATH: decode --video on the fly and feed frames straight
to "
663
664                         "the detector (no PNG extraction). Output-equivalent to the
disk path.")

```

```

664     ap.add_argument("--weights", required=True)
665     ap.add_argument("--out", required=True)
666     ap.add_argument("--sport", default="badminton")
667     ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
668     ap.add_argument("--batch-size", type=int, default=8)
669     ap.add_argument("--num-workers", type=int, default=4)
670     ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
671     ap.add_argument("--cache-dir", default=None,
672                     help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
673     ap.add_argument("--flush-every", type=int, default=1000,
674                     help="fsync the detector cache every N windows (bounds reboot
loss)")
675     ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
676     args = ap.parse_args()
677
678     if args.stream_video:
679         if not args.video:
680             raise SystemExit("--stream-video requires --video")
681         run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
682                             batch_size=args.batch_size,
683                             log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
684                             flush_every=args.flush_every, fresh=args.fresh)
685         return
686
687     frames_dir = args.frames_dir
688     if args.video:
689         frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
690     if not frames_dir:
691         raise SystemExit("provide --frames_dir or --video")
692     run(frames_dir, args.weights, args.out, args.sport, args.limit,
693         batch_size=args.batch_size, num_workers=args.num_workers,
694         log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
695         flush_every=args.flush_every, fresh=args.fresh)
696
697
698     if __name__ == "__main__":
699         main()
700

```

42. assistant (2026-06-18T01:29:27.170Z)

I've now read the bulk of the area. Let me focus on a few candidate bugs I noticed and verify them.

****Candidate 1:** ``gemini.py`` overlap resolution bug (line 392-411).****** When the kept segment is ``prev`` (the one already in ``non_overlapping``), but ``curr`` is dropped, ``prev`` is NOT extended to cover ``curr``. That's fine ? keeping the longer one. But there's a subtler issue: when ``kept == curr``, it replaces ``non_overlapping[-1] = curr``, but ``curr`` might not overlap with ``non_overlapping[-2]``. That's an existing tolerated design. Let me not flag design choices.

Let me check the ``_select_for_handoff`` person-cue path: ``min_players`` is compared to ``players_in_court``, but the person features are only added to ``window_stats`` if ``cue_flags.person`` is True. In ``_select_for_handoff``, the guard ``if person_on and min_players > 0`` ensures ``players_in_court`` exists. OK.

****Candidate 2:** In ``trajectory_hybrid.py`` line 278-279, the ``process_candidate_window`` uses ``window["start"]`` but the handoff iterates over ``handoff_indices`` and uses ``all_candidate_windows[wi]``. That's consistent.

Let me look more carefully at a real concern in `wasb\_infer.py` `load\_and\_clean\_cache` ? when resuming, `det\_by\_fid` accumulates from cache, but then the detector pass `remaining = samples[windows\_done:]` continues appending to `det\_by\_fid`. However there's a subtle issue: in the resume path the manifest records `windows\_done`, but `load\_and\_clean\_cache` returns `len(valid)` which is the count of contiguous valid lines. If `windows\_done` from manifest > `len(valid)` (because flush wrote manifest but the jsonl lost trailing lines), then `remaining = samples[windows\_done:]` would skip windows that were never actually cached. Let me check: `windows\_done, det\_by\_fid = load\_and\_clean\_cache(cache\_dir)` ? it uses the RETURN value `len(valid)`, not the manifest's `windows\_done`. So it's self-consistent. Good.

Let me look at the `detections\_over\_windows` `max\_end` computation and a possible issue with `\_split\_overlong\_groups` sort stability. Let me re-examine the `count\_net\_crossings`/`\_points\_in` for a potential issue, and check the test files to understand the expected contract for windows.

43. user (2026-06-18T01:29:28.596Z)

```
1      """TrackNet-based hybrid play segmenter.
2
3      Phase 2 (candidate generation) uses a TrackNetV4 *shuttle* trajectory computed
4      in WSL2 (backend/pipeline/detectors/tracknet_runner.py, ADR-001). Rationale: a
5      tiny, fast, motion-blurred shuttle is exactly what general detectors miss; a
6      purpose-built trajectory model localises it far more precisely, giving cleaner
7      candidate rallies than person-velocity heuristics.
8
9      The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
10     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
11     TrackNet runner and identity.
12     """
13     from typing import Any, Dict, Tuple
14
15     from backend.pipeline.segmenters.base import segmenter_registry
16     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
17     from backend.pipeline.detectors.base import DetectorRunner
18     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
19
20
21     class TrackNetHybridSegmenter(TrajectoryHybridSegmenter):
22         family = "tracknet"
23         display_label = "TrackNet"
24
25         @property
26         def name(self) -> str:
27             return "tracknet_hybrid"
28
29         @property
30         def description(self) -> str:
31             return ("Two-stage pipeline: TrackNetV4 shuttle-trajectory detection (WSL2) for
precise "
32                     "candidate rallies + AI provider for high-precision semantic
boundaries.")
33
34         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
35             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
36             return TrackNetRunner(cfg), {
37                 "weights_path": cfg.weights_path,
38                 "queue_length": cfg.queue_length,
39             }
40
41
42     segmenter_registry.register(TrackNetHybridSegmenter)
43
```

44. user (2026-06-18T01:29:29.985Z)

```

20: def extract_video_chunk(video_path: str, start_time: float, duration: float,
21-                          output_path: str, reencode: bool = False,
22-                          scale_width: Optional[int] = None) -> bool:
23-     """Extracts a chunk from a video file.
24-
25-     Args:
26-         reencode: If True, re-encodes with libx264 (slower but frame-accurate).
27-                   If False, uses stream-copy (fast but cuts at nearest keyframe).
28-                   Use True for short clips where keyframe drift matters.
29-         scale_width: If set (requires reencode), downscale to this width (height auto,
30-                      even). Useful for high-res sources headed to an API that analyses at
31-                      low res anyway ? shrinks uploads with no analysis loss.
32-     """
33-     cmd = [
34-         "ffmpeg", "-y",
35-         "-ss", str(start_time),
36-         "-t", str(duration),
37-         "-i", video_path,
38-     ]
39-     if reencode:
40-         if scale_width:
41-             cmd += ["-vf", f"scale={int(scale_width)}:-2"]
42-             cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
43-         else:
44-             cmd += ["-c", "copy"]
45-     cmd.append(output_path)
46-     try:
47-         subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
48-         return True
49-     except Exception as e:
50-         logger.error(f"Failed to extract chunk {start_time}s - {start_time+duration}s: {e}")
51-     return False

```

45. assistant (2026-06-18T01:29:41.108Z)

Now let me carefully analyze a specific concern: the `\_finalize` function in `tracknet\_runner.py` and the `\_trim\_inactive\_tail`. Let me re-examine a potential bug in `trajectory\_to\_action\_windows`.

After R4 trim (`\_trim\_inactive\_tail`), a group could be reduced to a single element (if `last\_dense == 0`). Then `\_finalize(g)` with a 1-element group works fine (start == end). But then the final filter `(w["end"] - w["start"]) >= min\_window\_duration` would drop it. OK.

But there's a subtle issue with R5 `\_blob\_fallback\_chop`: it runs AFTER R4 trim. The R4 trim can produce groups where `len(g) < 2`. In `\_blob\_fallback\_chop`, the check `if len(g) < 2 or ...` handles that. Fine.

Now, let me look at a more interesting concern. In `gate\_windows`, `\_points\_in` uses `start \* fps <= p.frame <= end \* fps`. The window `(s, e)` start/end are in **seconds** (from `trajectory\_to\_action\_windows` which divides frame by fps and adds chunk_start). But `p.frame` is the raw frame index. If `chunk\_start != 0`, the window's start in seconds includes `chunk\_start`, but `p.frame` does not. However, in the trajectory hybrid, `chunk\_start=0.0` always (whole-video pass). So in practice it's fine. The fusion segmenter passes `points` (whole-video) and windows with chunk_start=0. OK.

Let me dig into the `\_select\_for\_handoff` person path in fusion: line 125 `start\_f, end\_f = round(cw["start"] \* fps), round(cw["end"] \* fps)`. The `points` filtered `shuttle\_in = [p for p in points if start\_f <= p.frame <= end\_f]`. That's consistent with frame indices. Good.

Let me examine the histogram normalization concern in `scene\_cut.py` and `\_moving\_stats` reflect padding edge case. Let me look closer at `\_moving\_stats` when `len(env)` is small relative to `win`.

46. user (2026-06-18T01:29:43.026Z)

```

91     def _moving_stats(env: np.ndarray, win: int) -> Tuple[np.ndarray, np.ndarray]:
92         """Sliding-window mean and std (reflect-padded, via cumsum). win forced odd."""
93         if win % 2 == 0:
94             win += 1
95         pad = win // 2
96         p = np.pad(env.astype(np.float64), pad, mode="reflect")
97         c1 = np.cumsum(np.insert(p, 0, 0.0))
98         c2 = np.cumsum(np.insert(p * p, 0, 0.0))
99         s1 = c1[win:] - c1[:-win]
100        s2 = c2[win:] - c2[:-win]
101        mean = s1 / win
102        var = np.maximum(s2 / win - mean * mean, 0.0)
103        return mean[:len(env)], np.sqrt(var)[:len(env)]
104
105
106    def detect_hits(samples: np.ndarray, sr: int, k: float = 2.5, win_frames: int = 41,
107                  min_gap_sec: float = 0.08, band: Optional[Tuple[float, float]] = None)
-> List[HitEvent]:
108        """Detect impact transients: local maxima of the onset envelope that exceed an
109        adaptive threshold (moving mean + k*std), keeping the strongest within
110        `min_gap_sec`.
111        `k` is the sensitivity knob (lower = more hits); `band` band-limits the onset flux
112        to the shuttle-click frequencies (e.g. (3000, 9000))."""
113        env, hop_sec = onset_envelope(samples, sr, band=band)
114        if len(env) < 3:
115            return []
116        mean, std = _moving_stats(env, win_frames)
117        thr = mean + k * std
118        is_peak = (env[1:-1] >= env[:-2]) & (env[1:-1] > env[2:]) & (env[1:-1] > thr[1:-1])
119        peak_idx = np.nonzero(is_peak)[0] + 1
120        if peak_idx.size == 0:
121            return []
122        # enforce min inter-peak gap: greedily keep strongest first
123        min_gap = max(1, int(round(min_gap_sec / hop_sec)))
124        order = peak_idx[np.argsort(-env[peak_idx])]
125        taken = np.zeros(len(env), dtype=bool)
126        kept = []
127        for i in order:
128            lo, hi = max(0, i - min_gap), min(len(env), i + min_gap + 1)
129            if taken[lo:hi].any():
130                continue
131            taken[i] = True
132            kept.append(i)
133        kept.sort()
134        return [HitEvent(t=float(i * hop_sec), strength=float(env[i])) for i in kept]
135

```

47. assistant (2026-06-18T01:29:50.368Z)

There's a real concern in `\_moving\_stats`: `np.pad(..., mode="reflect")` requires that the array length be `> pad` for reflect mode (reflect cannot pad more than `len-1` elements). With `win\_frames=41` default, `pad=20`. `detect\_hits` guards `len(env) < 3` returns early, but if `3 <= len(env) <= 20`, then `np.pad` with `mode="reflect"` and `pad=20 > len(env)-1` raises a `ValueError`. Let me verify numpy's behavior.

Actually, numpy's `reflect` mode raises when pad width exceeds the dimension for `reflect`... let me verify precisely. Actually numpy allows reflect padding larger than the array in recent versions? Let me check. In older numpy, `np.pad` with `reflect` and `pad > len-1` raises `ValueError: Reflect... cannot reflect more than...`? Actually no? `reflect` mode in numpy does NOT raise for `pad > len`; it reflects repeatedly. Let me verify this in the environment.

48. user (2026-06-18T01:29:52.787Z)


```
[5. 4. 3. 2. 1. 2. 3. 4. 5. 4. 3. 2. 1. 2. 3. 4. 5. 4. 3. 2. 1. 2. 3. 4.
 5. 4. 3. 2. 1. 2. 3. 4. 5. 4. 3. 2. 1. 2. 3. 4. 5. 4. 3. 2. 1.]
```

49. assistant (2026-06-18T01:30:25.067Z)

numpy `reflect` handles `pad > len` fine. Not a bug. Wait ? the dev container has numpy 2.4.6. Good, so `\_moving\_stats` is safe.

Now let me reconsider the `detections\_over\_windows` function. `max\_end = max(ef for \_, ef in windows)`. If a window has `ef` beyond the actual frame count, the loop `while frame\_idx <= max\_end` exits via `if not cap.grab(): break`. Fine.

Let me look more carefully at a potential real bug in `gemini.py` overlap resolution. Consider sorted validated by start. When `kept == prev`, `curr` is dropped but `non\_overlapping[-1]` stays `prev`. The next `curr2` is compared against `prev` (still `non\_overlapping[-1]`). But if `curr` was dropped and `curr` extended further than `prev`, a `curr2` that overlaps `curr` but not `prev` would be incorrectly added as non-overlapping. That's a logic imperfection but it's a known "keep the longer one" heuristic ? the merged output may still have overlaps. This is design, the stitcher merges overlaps later anyway. Not a crash. I'll drop it.

Let me look at a concrete potential index bug: `gemini.py` line 407-408: `if kept == curr: non\_overlapping[-1] = curr`. The comparison `kept == curr` compares dicts. `kept` is either `prev` or `curr` (same object refs from the ternary). Dict `==` compares by value. If `prev` and `curr` happen to be equal dicts (same start/end/idx/raw), then `kept == curr` is True even when `kept` is actually `prev`. But they have different `idx`, so they'd never be value-equal. Edge case, not realistic. Drop.

Now let me look harder at the `\_split\_overlong\_groups` `hard\_cap` midpoint selection for a subtle infinite-loop / off-by-one. The `min(range(1, len(g)), key=lambda i: abs(g[i][0] - mid))`. If all elements have the same frame value... no, frames are strictly increasing (sorted, and active frames appended in order with `p.frame`). Actually frames could have duplicates? `active.append((p.frame, velocity))` ? p.frame increases monotonically since points are sorted, but could two visible points share a frame? In a trajectory CSV, frames are unique per row. After sort, frames are non-decreasing. Duplicates unlikely. If `split\_i` could be such that `g[split\_i] == g` (i.e., `split_i == len(g)`)? No, range stops at `len(g)-1`. So both pieces strictly smaller. Terminates. Fine.

Let me check the `extract\_action\_windows\_from\_chunk` `max\_gap\_frames` uses `fps` but active_frames store `frame\_idx` (raw, not frame-skip-adjusted). The grouping `af[0] - group[-1][0] <= max\_gap\_frames` where `max\_gap\_frames = int(merge\_gap \* fps)`. `frame_idx` increments by 1 each loop even on skipped frames (`frame_idx += 1` in skip branch). So `frame_idx` is the real frame index. Good consistency.

Let me now look at the `motion.py` resource leak: when `cap.isOpened()` fails it returns `Failure` without `cap.release()`. cv2 VideoCapture that failed to open doesn't hold a resource really. Minor, drop.

Let me re-examine `\_merge\_overlapping` in compiler more carefully and the `compile\_highlights` skipped-segment edge: After merge, the loop validates `start >= video\_duration` skips. But `padded\_start = max(0.0, start - begin\_padding)` and `padded\_end = end + end\_padding`. If `start` was clamped to 0 (negative), fine.

Let me look at the actual structured concern around `point\_in\_polygon` ? the `x\_cross` computation has a guard `if yj != yi else xi`, but the outer condition `(yi > y) != (yj > y)` already guarantees `yi != yj` when crossing (since one is `> y` and the other isn't, they can't be equal). So `yj == yi` only when both on same side ? condition False ? branch not entered. The `else xi` is dead but harmless. Not a bug.

Let me focus and verify one more promising area ? the `\_axis\_estimate` cache keyed by `id(points)`. Since `points` is a list created once per video and reused, `id()` is stable within a call. But `id()` can be reused after GC if a prior list was freed. Within a single `process\_video` call, `points` lives, so no reuse. Cross-call, the segmenter instance persists? Each video gets a fresh segmenter (`segmenter\_cls(db, config)` in main.py per request). Within one `process_video`, `points` is one object. Fine.

Let me look at the test for net crossings / gate to check the contract, and re-examine `\_points\_in` with float frame comparison. Let me check whether `p.frame` could be non-integer.

Let me now look at one thing I want to verify: the `compile\_highlights` `attempts` retry loop. When `vf\_args` is set (watermark) and the first attempt fails with `CalledProcessError`, it logs and `continue`s to the second attempt (no vf). But notice line 320: after a successful subprocess.run, if the clip is missing/empty, it sets `last\_exc = None` and `break`. The retry-without-watermark only happens on `CalledProcessError`, NOT on empty-output. So if the watermark encode "succeeds" (returncode 0) but produces an empty clip, it won't retry without watermark. Minor logic, but that's an edge, and producing empty on success is rare. Not a clear bug.

Let me verify a more concrete suspicion: in the `compile\_highlights`, the `cmd` variable used in the error message at line 328 (`''.join(cmd)`). On the empty-output path (`produced=False`, `last\_exc=None`), it goes to the `else` branch (line 332). On the CalledProcessError-all-attempts-failed path, `cmd` holds the last attempted command. Fine.

Let me look at `\_blob\_fallback\_chop` once more for the `forced += len(pieces) - 1` ? if a blob splits into N pieces, that's N-1 cuts. Reasonable.

Let me check the `relevance.py` `h, w = frame.shape[:2]` ? `h, w` are computed but unused (resize to fixed 320x240). Dead but not a bug.

Let me verify the most promising candidate so far ? `count\_net\_crossings` and gate semantics ? by checking the test file, and also examine whether there is a unit mismatch in `\_points\_in` between fps-scaled seconds and frames.

50. user (2026-06-18T01:30:26.726Z)

```
test_audio_features.py
test_calibrate_wasb.py
test_compiler.py
test_config_wasb.py
test_export_wasb_segments.py
test_fusion_compare.py
test_fusion_features.py
test_fusion_golden_telemetry.py
test_fusion_hybrid.py
test_hit_detector.py
test_person_detector.py
test_personalization_serving.py
test_promotion.py
test_rally_gate.py
test_tracknet_hybrid.py
test_tracknet_runner.py
test_trajectory_hybrid_equivalence.py
test_validators_base.py
test_wasb_cache.py
test_wasb_runner.py
```

51. assistant (2026-06-18T01:31:07.133Z)

Let me look at the compiler test to understand edge cases tested, and specifically check the `\_merge\_overlapping` / watermark behavior. But more importantly let me reconsider a subtle bug in `compiler.\_watermark\_filter`: the `font\_size\_ratio` divisor computation `divisor = max(1, round(1.0 / ratio))`. If ratio is e.g. 0.035, divisor = round(28.57) = 29. fontsize=h/29. Fine. If `ratio` is set to a value > 1 (config error), `1.0/ratio < 1`, round could be 0 or 1, max(1,...) protects. If ratio is 0, `wm.get("font\_size\_ratio")` or 0.035 ? `0` or 0.035 ? 0.035 (because 0 is falsy). Good, protected.

Let me re-examine the `\_ff\_quote` and the concat list file escaping. In `compile\_highlights`, the concat list uses `safe\_path = clip.replace("", "\\")` ? proper bash-style escaping for the concat demuxer. Fine.

Let me dig into a concrete reachable bug. Let me reconsider `detections\_per\_frame`: `tracker = self.make\_tracker(fps / step)`. `make\_tracker` does `frame\_rate = max(1, int(round(fps)))`. So

`fps/step` could be small. Fine.

Let me re-examine `in\_court\_ratio` vs `players\_in\_court` comparison in `\_select\_for\_handoff`. `min\_players` is `int(...)`. `players\_in\_court` returns a median (could be float). `st.get("players\_in\_court", 0.0) < min\_players`. If median is 2.0 and min_players is 2, `2.0 < 2` is False ? kept. Good.

Now let me carefully look at the `gemini.py` `parse\_time` MM:SS reversal handling for `"H:MM:SS.ms"`. Actually let me focus on something I can definitively prove. Let me re-examine `\_split\_overlong\_groups` sort at the end: `out.sort(key=lambda grp: grp[0][0])`. But in `hard\_cap/gap-split`, pieces are pushed to `work` and popped LIFO, then appended to `out`, finally sorted. The final sort ensures temporal order. But there's a concern: when a group is split, `work.append(g[best\_i])` then `work.append(g[best\_i:])`. Since `work` is LIFO, the second half is processed first. But final sort fixes order. Good.

Let me verify the windows produced retain correct order for the `forced\_flags` zip in `trajectory\_to\_action\_windows`. After `\_split\_overlong\_groups` (sorted), then R4 trim (per-group, order preserved), then R5 `\_blob\_fallback\_chop`. In R5, `out` is built by iterating `groups` in order and extending with sorted pieces. So order preserved, and `flags` aligned 1:1 with `out`. Then `zip(groups, forced\_flags)`. Good.

But wait ? when R5 is OFF (default), `forced\_flags = [False] \* len(groups)`. If R5 is ON, `\_blob\_fallback\_chop` returns new `groups` and `flags`. The `zip(groups, forced\_flags)` then zips correctly. Good.

Let me look at the actual test for the trajectory equivalence and check `\_finalize` when a group becomes empty. Can a group ever be empty? `\_trim\_inactive\_tail` returns `group[:last\_dense+1]` where `last\_dense >= 0`, so at least 1 element, or returns `group` whole. `\_split\_overlong\_groups` pieces always `len>=1`. `\_blob\_fallback\_chop` pieces `len>=1`. So `\_finalize(g)` always has `group[0]` and `group[-1]`. Good.

Let me check one more thing in `extract\_action\_windows\_from\_chunk`: `track\_ids = set().union(\*[ids for \_, \_, ids in group])`. If `group` is non-empty (guaranteed), this works. Good.

Let me now reconsider the `**person_detector `detections_over_windows`** `frame_idx % 256 == 0`` progress and the `max\_end`. Let me check whether `windows` could contain `start\_frame > end\_frame`. The caller passes `(start\_f, end\_f)`. If a window is degenerate... in fusion it's `round(cw["start"]\*fps), round(cw["end"]\*fps)` with end > start guaranteed (min_window_duration filter). OK.

I've done a thorough pass. Let me now zoom into one thing that looks genuinely suspicious ? the `**gemini.py` non-overlapping kept==prev case extends nothing**`, but more concretely, let me check `_merge_overlapping` in compiler handles the case correctly vs. what tests expect. Let me instead look at the `compiler` `skipped_segments` reporting line 339: `len(segments)` ? but `segments` was reassigned to merged. That's correct count.

Let me re-read `parse\_time` for the colon path: `"1:30" ? parts ["1","30"]` reversed `["30","1"]` ? $30*1 + 1*60 = 90$. Correct.

Let me look at a genuinely concerning detail in `compiler.compile\_highlights`: when a segment's `end` is clamped to `video\_duration` (line 261), but then `padded\_start = max(0, start - begin\_padding)` and `padded\_end = end + end\_padding`, then `padded\_end` clamped. But `start` was checked `start >= video\_duration` ? skip. However, after clamping end to video_duration, could `padded\_start >= padded\_end` ? `padded\_start = start - 0.5`, `padded\_end = min(video\_duration, end+1.0)` where `end = video\_duration`. So `padded\_end = video\_duration`, `padded\_start = start - 0.5`. Since `start < video\_duration` and `start < end`, `padded\_start < video\_duration = padded\_end`. `trim_duration > 0`. Good.

Let me verify one last real candidate: in `wasb\_runner.\_stage\_video` and `tracknet`, the cleanup. Actually, let me carefully re-examine the `**R4 `_trim_inactive_tail` two-pointer**`. The condition `while frames[lo] <= frames[i] - trail\_frames`. This advances `lo` while the oldest frame in window is at or before `frames[i] - trail\_frames`. The window is `(frames[i]-trail\_frames, frames[i]]`. `win\_count = i - lo + 1`. This counts frames in the trailing window. The density `fast\_sum / win\_count`. The bug potential: `fast\_sum` tracks

``fast[lo..i]``. When ``lo`` advances past, ``fast_sum -= fast[lo]`` then ``lo += 1``. Correct sliding window. Looks right.

Now let me make sure I haven't missed a genuine off-by-one in the streaming ``SequentialFrameStore``. Let me re-examine the eviction: ``new_floor = max(self._floor, self._max_seen - (self._window - 1))``. For `window=3`, after seeing index `m`, `floor = m-2`. The next window could start at `m-2` (window starting at frame `m-2` spans `m-2,m-1,m`). But could a window start before `m-2` after `m` is seen? Window `i` accesses frames `i..i+window-1`. The max frame seen is at window `i` where `i+window-1 = m`, so `i = m-window+1 = m-2`. The next window is `i+1 = m-1`, accessing `m-1,m,m+1`. So no future window asks for `< m-1`. `Floor = m-2` keeps `m-2` unnecessarily but never evicts something needed. Safe (conservative). Actually wait ? could window `i = m-2` still be in-flight (not yet fully read) when index `m` is requested? The `DataLoader` reads window `i`'s frames `i, i+1, ..., i+window-1` in order. It reaches `i+window-1 = m` last. At that point window `i` is done. So when `m` is seen, window `i` (`m-2`) is complete; window `i+1` (`m-1`) is next, needs `m-1`. `Floor m-2` is safe. Good.

I've been thorough. Let me do a final targeted check on one thing: the ``person_detector.detections_over_windows`` builds ``trackers = [self.make_tracker(...) for _ in range(n)]`` ? one per window, holding state. For 172 windows that's 172 trackers held in memory simultaneously. That's a memory concern but not a bug.

Let me check whether there's a genuine bug in how ``frames_per_win`` and ``trackers`` indexing aligns. Both indexed by ``wi``. Good.

Let me make one final verification ? look at the test for ``_select_for_handoff`` and confirm the never-empty fallback. Actually let me re-examine: in ``_select_for_handoff``, when ``min_crossings > 0`` but person OFF, and ALL windows fail crossings ? ``keep`` empty ? fallback returns all. But persisted candidates already stored. Fine, intended.

Let me run the existing pipeline tests quickly to confirm baseline green and that I understand contracts, then finalize.

52. user (2026-06-18T01:31:09.546Z)

..... [100%]
52 passed in 0.27s

53. assistant (2026-06-18T01:31:19.192Z)

Tests pass. Now let me reconsider one thing I glossed over that could be a real bug: in ``compiler.compile_highlights``, the `**video-duration` clamping interacts with ``_merge_overlapping``. But more concretely, let me re-examine the watermark retry logic precisely once more, because the ``last_exc = None # ffmpeg succeeded but produced nothing`` followed by ``break`` at line 320-321 only runs after the FIRST attempt in the ``attempts`` list. Let me trace the empty-output-with-watermark case:

```
- `attempts = [vf_args, []]`  
- First iteration (`attempt_vf = vf_args`): `subprocess.run` succeeds (returncode 0, no exception). Check `os.path.exists and getsize > 0` ? False (empty). So skip the `produced=True` block. Then line 320 `last_exc = None`, line 321 `break`.
```

So with watermark producing an empty file on success, it breaks out without trying the no-watermark attempt. Then ``produced=False, last_exc=None`` ? "empty output clip" skip. That's a minor degradation, not a crash. The watermark failing usually returns non-zero (the common case is handled). I'll leave it.

Now, let me reconsider the genuinely interesting candidate: `**`gemini.py`` overlap-resolution drops a segment that doesn't actually overlap the kept one.** Actually let me re-read lines 397-410 once more very carefully because there's a real logic bug pattern here.

54. user (2026-06-18T01:31:20.129Z)

```
392         # E: Resolve overlapping segments (keep the longer one)  
393         if len(validated) > 1:
```

```

394         # Sort by start time
395         validated.sort(key=lambda r: r["start"])
396         non_overlapping = [validated[0]]
397         for curr in validated[1:]:
398             prev = non_overlapping[-1]
399             # Check if current segment overlaps with the previous one
400             if curr["start"] < prev["end"]:
401                 prev_dur = prev["end"] - prev["start"]
402                 curr_dur = curr["end"] - curr["start"]
403                 kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
prev)
404                 kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
{kept['end']:.2f}s]"
405                 dropped_label = f"Segment #{dropped['idx']+1}
[{dropped['start']:.2f}s - {dropped['end']:.2f}s]"
406                 rejected.append((dropped_label, f"overlaps with {kept_label} ? kept
the longer segment"))
407                 if kept == curr:
408                     non_overlapping[-1] = curr # Replace previous with current
409             else:
410                 non_overlapping.append(curr)
411         validated = non_overlapping

```

55. assistant (2026-06-18T01:31:39.563Z)

This is a known greedy heuristic; the downstream stitcher's `\_merge\_overlapping` collapses any residual overlaps, so this isn't a correctness bug that produces wrong output ? it's a tolerated approximation. Not flagging.

I've now done a comprehensive trace of the entire `backend/pipeline/` area. Let me do one final check on something that's nagging me about the `gemini.py` `parse\_time` ? the colon path doesn't guard against an empty `total` accumulator producing wrong values, but that's just resilience.

Let me verify the one thing that could be a genuine reachable crash: `format.py` line 48 `stream = info["streams"][0]` ? guarded by `if not info.get("streams")` at line 38, which catches empty list (`[]` is falsy). Good.

And `resolution\_fps.py` line 23 `stream = meta["streams"][0]` ? guarded by `if not meta or not meta.get("streams")` at line 15. Good.

Let me reconsider `pts\_continuity.py`: `parts = line.split(",")` and `if len(parts) == 2`. But the ffprobe output `packet=pts\_time,flags` with `csv=p=0` produces `pts\_time,flags`. However, the `flags` field from ffprobe can contain a trailing comma or the flags string itself? ffprobe packet flags is like `K\_\_` or `K\_`. No comma inside. Good. But `pts\_time` could be `N/A` for some packets ? `float("N/A")` raises `ValueError`, caught by the inner try. Good.

After this exhaustive review, I have not found a clear-cut, high-confidence reachable defect that a maintainer would unambiguously call a bug. The code in this area has been heavily hardened. Let me do a final scan of one area I haven't fully traced ? the `\_shot\_count\_from` and the `min(1.0, cw["peak\_velocity"])` confidence (peak_velocity could exceed 1.0, capped ? fine; could it be negative? No, it's a distance/positive).

Let me make absolutely sure about one numeric edge: in `trajectory\_to\_action\_windows`, `confidence=min(1.0, cw["peak\_velocity"])` is in `trajectory\_hybrid`. peak_velocity = max of velocities, all positive. Fine.

One genuinely worth-flagging item: let me re-examine `\_get\_video\_duration` returning a value from `float(result.stdout.strip())`. If ffprobe outputs `N/A` (some streams), `float("N/A")` raises `ValueError`, caught by `except Exception` ? returns 0.0. Fine.

I'm confident the area is clean. Let me reconsider whether the `compiler` empty-output-with-watermark not retrying is worth flagging as low severity ? it's a real but very narrow logic gap (success+empty file is essentially never produced by ffmpeg). I'll keep precision high and report only what I can firmly stand behind.

Let me do one final targeted look at `detections\_over\_windows` ? `trackers = [self.make\_tracker(fps / step) for \_ in range(n)]` is created even when `n` could be 0? No ? line 330 guards `if not cap.isOpened() or n == 0: return`. So $n \geq 1$ here. Good.

Based on my thorough trace, I did not find a defect meeting the precision bar. Returning an empty candidate list is the honest answer.

56. user (2026-06-18T01:31:40.217Z)

Structured output provided successfully

57. assistant (2026-06-18T01:31:54.235Z)

I traced the full `backend/pipeline/` area and found no defect meeting the precision bar. Returning an empty candidates list.

Files reviewed and the specific things I traced (all turned out correct or tolerated-by-design):

- `backend/pipeline/stitcher/compiler.py` ? `\_parse\_time`, `\_merge\_overlapping`, segment validation/clamping, the watermark `-vf` retry loop, ffmpeg concat list escaping. The only narrow gap (a watermark encode that returns rc=0 but writes an empty clip won't retry without the watermark) is unreachable in practice ? ffmpeg doesn't succeed-with-empty-output ? so not a real bug.
- `backend/pipeline/detectors/tracknet\_runner.py` ? `trajectory\_to\_action\_windows` and helpers `\_split\_overlong\_groups` (hard-cap midpoint chop terminates; both halves strictly smaller), `\_trim\_inactive\_tail` (two-pointer sliding window correct), `\_blob\_fallback\_chop` (flags aligned 1:1 with groups). `\_finalize` never sees an empty group.
- `backend/pipeline/detectors/rally\_gate.py` + `fusion\_features.py` ? net-crossing/point-in-polygon math; unit alignment between window seconds and frame indices holds because the trajectory pass uses `chunk\_start=0`.
- `backend/pipeline/detectors/person\_detector.py` ? `detections\_per\_frame` / `detections\_over\_windows` seek+grab logic, per-window tracker indexing, `max\_end` loop guard.
- `backend/pipeline/detectors/wasb\_infer.py` ? `SequentialFrameStore` eviction floor (conservative, never evicts a still-needed frame), resumable cache `load\_and\_clean\_cache` (uses returned contiguous count, not the manifest's, so self-consistent).
- `backend/pipeline/segmenters/{trajectory\_hybrid,fusion\_hybrid,gemini,yolo\_hybrid,motion}.py` ? handoff index/candidate-id alignment, the gemini overlap-resolution greedy heuristic (residual overlaps are collapsed downstream by the stitcher, so not a correctness defect), `motion.py`'s `Result` return (deliberate, handled in `main.py`).
- `backend/pipeline/validators/\*.py` ? guarded `streams[0]`/`format` access, ffprobe parse fallbacks.
- `backend/pipeline/audio/hit\_detector.py` ? `\_moving\_stats` reflect-padding is safe under numpy 2.4.6 for `pad > len-1` (verified by running it).
- `backend/pipeline/personalization/serving.py` ? overlay/min-N fallback logic.

Pipeline-related tests (`test\_compiler.py`, `test\_rally\_gate.py`, `test\_tracknet\_runner.py`) pass green, consistent with my reading of the contracts.